

16 · Systems Engineering (Systemanalyse / Requirements Engineering)

Modul: W3WI_101.2 · Klausurteil **Systems Engineering** (früher: *Systemanalyse*) **Dozent:** Prof. Dr. Michael Bächle („BäM“) **Studiengang:** DHBW Ravensburg · Data Science und Künstliche Intelligenz / Wirtschaftsinformatik – Data Science **Klausur:** 120 Minuten, **100 Punkte, KEINE Hilfsmittel**, Antworten nur auf den Klausurblättern, geheftet abgeben (*ältere Kurse: 70–72 Minuten, 60 Punkte*)

Quellenlage dieser Analyse

Vorlesungsfoliensätze (die Kernquelle)

LE	Titel	Datei	Seiten
LE 1	Black Swans — Warum Softwareprojekte scheitern	01.RE-BlackSwansOhneLoesung.pdf	14
LE 2	Softwaretechnik — Ingenieursmäßige Systementwicklung	02.RE-Softwaretechnik.pdf	14
LE 3	Prozessmodelle — „Elefanten isst man am Stück“	03.RE-Prozessmodelle.pdf	30
LE 4	Agiles Software Engineering — „Elefanten isst man scheibchenweise“	04.RE-AgileSE.pdf + 04.1AgileBegriffe.pdf + 04.2AgileFluencyModell.pdf	20+12+12
LE 5	Essenzielle Modellierung — Paradigma der ereignisorientierten Zerlegung	06.RE-ERM.pdf	18
LE 6	Erhebungstechniken — Ermittlung der Anforderungen	07.RE-Erhebungstechniken.pdf	31
LE 7	Use Cases — Modellierung der Anwendungsfälle	09.RE-UC.pdf	35
LE 8	Aktivitätsdiagramme, BPMN	10.RE-AD-BPMN.pdf	11
LE 9	Entscheidungstabellen — stark regelbehafte Anforderungen	11.RE-ET.pdf	17
LE 10	Anforderungen formulieren — gute Anforderungssätze	12.RE-Anforderungen.pdf	20
Exkurs	Spezifikationsvorlagen — Inhalte einer Spezifikation	13.RE-Spezifikationsvorlagen.pdf	6

Alte Klausuren (5 Stück — die wichtigste Prüfungsvorbereitung!)

Datei	Kurs	Datum	Dauer	Punkte
WWIDS19.Klausur.SA.pdf	WWIDS119	18.06.2020	70 min	60
WWIDS120.Klausurteil.SA.pdf	WWIDS120	22.09.2021	70 min	60
WWIDS121.WKL.Klausurteil_SA.pdf	WWIDS121	27.06.2022	70 min	60
WWIDSx22.WKL.Klausurteil.pdf	WWIDS222 (WKL)	15.01.2024	72 min	60
WDSx23.Systems-Engineering.Klausur.pdf	WDSx23	18.12.2024	120 min	100 ← neuestes Format!

Weitere Materialien

Datei	Inhalt
00.ATMFallstudie.DSKI.pdf	Fallstudie AIM („All-Inclusive-Maschine“) mit 7 User Stories + 4 Arbeitsaufträge
ET01.txt	Übungsaufgabe Entscheidungstabelle „Geld einzahlen“
01_Mertens.Fehlschlaege.pdf	Mertens: Fehlschläge bei IT-Großprojekten (FISCUS , RUBIN, Tiergartentunnel, DiPlaZ, eGK, Toll Collect, Hartz IV)
Use Cases Lessons Learned.pdf	Studierendenpräsentation mit 3 Lessons Learned zu BPMN/UC
bäm.pdf	Follow-Up-Beitrag „BSDL“ mit Systemanalyse-AHA-Momenten
IEEE-STD-830-1998.pdf · srs_template.doc · Volere_templateV901.doc	Spezifikationsvorlagen
scrumhandbook.pdf (66 S.) · boehm-tuerner-balancing.pdf · Brooks.pdf · The_New_New_Product_Development_Game.pdf · SpotifyScaling.pdf · Shore_SAFc.pdf	Vertiefungsliteratur zu Agilität
BABOKv2.0.pdf (271 S.)	Business Analysis Body of Knowledge (Referenz, nicht prüfungsrelevant im Detail)

 **Hinweis zur Quellenlage:** Einige Dateien sind **reine Bild-Scans ohne Textebene** und konnten daher nicht ausgewertet werden: Balzert_Kap20.pdf , Balzert_Kap23.pdf , Cockburn.Rules.pdf , ET-Musterlösung.pdf , Shore_Kap_1.pdf , Bergsmann.Recht0001.pdf , Entscheidungsmatrix_Ermittlungstechniken.pdf . Inhalte, die ausschließlich dort stehen (z. B. die exakte Liste „fünf Voraussetzungen für den Einsatz agiler Methoden“ aus Balzert Kap. 23), sind unten mit  als **ergänzt/rekonstruiert** gekennzeichnet.

 Die neueste Klausur verweist auf „**BSDS**“ — das Lehrbuch (Bächle/Daurer/...: *Einführung in die Wirtschaftsinformatik* bzw. das Systems-Engineering-Skript). Es liegt dem ZIP nicht bei; die entsprechenden Inhalte sind aber vollständig in den Folien enthalten.



TEIL 0 — DAS KLAUSURFORMAT (zuerst lesen!)

0.1 ★★ Die Struktur ist über alle 5 Klausuren fast identisch

Neues Format (WDSx23, 120 min, 100 Punkte)

Aufgabe	Punkte	Typ
1	36	Wissensfragen (a)–(j) — 10 Teilaufgaben
2	6	Kano-Modell erläutern
3	8	Arten von Anforderungen + Beispiele
4	10	Entscheidungstabelle erstellen (konsolidiert & widerspruchsfrei)
5	5	Fehlerhafte Entscheidungstabelle korrigieren
6	15	Formale Fehler in einem Product Use Case finden (5 Stück + Begründung)
7	20	(a) Product Use Case modellieren (15 P.) + (b) BPMN-Modell (5 P.)
Σ	100	

Altes Format (60 Punkte, 70 min)

Aufgabe	Punkte	Typ
1	20	Wissensfragen (a)–(h/j)
2	10	10 Single-Choice-Fragen (⚠ falsche Antwort = –1 Punkt!)
3	10	Entscheidungstabelle erstellen
4	20	(a) Product Use Case (15 P.) + (b) Aktivitätsdiagramm (5 P.)
Σ	60	

🔑 **Die 40-Punkte-Regel:** In **jeder** Klausur sind **mindestens 30–35 % der Punkte** die große Modellieraufgabe (**Product Use Case + AD/BPMN**), und **10–15 %** die **Entscheidungstabelle**. Wer diese **beiden Techniken sicher beherrscht**, hat die Klausur **fast bestanden**.

0.2 ★★ Häufigkeitsanalyse: Welche Themen kommen wie oft?

Thema	2020	2021	2022	2024	2024/12	Σ
Product Use Case modellieren	✓	✓	✓	✓	✓	5/5
Entscheidungstabelle	✓	✓	✓	(✓ SC)	✓✓	5/5
Aktivitätsdiagramm / BPMN	✓ AD	✓ AD	✓ AD	✓ Muster	✓ BPMN	5/5
Coffee-Break-Test (SC)	✓	✓	✓	✓	—	4/5
„Who has the ball“ (SC)	—	✓	✓	✓	—	3/5
Lastenheft-Definition (SC)	✓	✓	✓	✓	—	4/5
Essenz / essentielle Anforderung	✓	✓	✓	✓	✓	5/5
FISCUS	✓	✓	—	—	✓	3/5
Agiles Manifest	✓	—	✓	✓	—	3/5
Scrum (Master/PO/Komponenten)	✓	✓	✓	✓	✓	5/5
Prozessmodelle (V-Modell, evolutionär/inkrementell)	✓	—	✓	✓	—	3/5
Prototypen (3 Arten)	✓	—	—	✓	—	2/5
Akteure / Arten von Akteuren	—	✓	✓	—	✓	3/5
Ereignisarten / ERM	✓	✓	✓	—	✓	4/5
QS analytisch/konstruktiv	✓	—	—	✓	✓	3/5
INVEST	—	—	—	✓	✓	2/5
DoD / DoR	—	—	—	✓	✓	2/5
Dimensionen klassisch vs. agil	—	—	—	✓	✓	2/5
include/extend (SC)	✓	✓	✓	—	—	3/5
Kano-Modell	—	—	—	—	✓	1/5
Arten von Anforderungen	—	—	—	—	✓	1/5

🔑 **Fazit:** Die absoluten **Muss-Themen** sind **Product Use Case, Entscheidungstabelle, AD/BPMN, Essenz, Scrum**. Das neueste Format hat **Kano-Modell, Anforderungsarten, INVEST, DoD/DoR und LeSS (Area Product Owner)** hinzugenommen.

0.3 ⚠ Die Single-Choice-Falle

*„Es ist immer nur eine der angegebenen Antworten korrekt! **Eine falsche Antwort führt jeweils zu 1 Punkt Abzug!**“*

🔑 **Strategie:** Nur ankreuzen, wenn Sie sich sicher sind. **Nicht ankreuzen = 0 Punkte, falsch ankreuzen = –1 Punkt.** Bei 50 % Sicherheit ist der Erwartungswert 0 — bei weniger lieber leer lassen.

TEIL A — LE 1: BLACK SWANS · WARUM SOFTWAREPROJEKTE SCHEITERN

A.1 ★ Der Begriff „Black Swan-Projekt“

Definition (nach Taleb 2008): Ein **höchst unwahrscheinliches Ereignis**, welches im Wesentlichen **drei Charakteristika** aufweist:

1. **Unvorhersehbarkeit**
2. **Massive Auswirkungen**
3. **Seltenes Auftreten**

📖 Taleb, N. N. (2008): *Der Schwarze Schwan: Die Macht höchst unwahrscheinlicher Ereignisse*, 2. Aufl., Carl Hanser: München.

A.2 📝 Fallbeispiel Levi Strauss & Co. (2003–2008)

Aspekt	Fakten
Ausgangslage	150-jähriges Jubiläum; internationale Expansion in über 110 Länder → hochgradig fragmentierte und überlappende IT-Architektur (Baan in Europa, Mainframe-Eigenentwicklungen für Asien, Kanada, USA)
Zweiter Treiber	Sarbanes-Oxley Act (2002) → ein neuer Auditor deckte Mängel in den Buchhaltungsverfahren auf; Levi Strauss musste beinahe seine Abschlüsse neu erstellen
Entscheidung	Komplette Architektur mit SAP erneuern (Beratung: Deloitte)
Geplante Kosten	USD 5 Mio. (ohne Beratungshonorare)
Komplexitätsanstieg	Während der Implementierung in Asien forderte Schlüsselskunde Walmart , dass die <i>bald abzulösenden</i> Altsysteme mit dessen SCM-System interfacen
Go-Live USA	2008. Man versuchte, das Risiko durch Vorziehen von Q2-Aufträgen ins Q1 zu mildern
Katastrophe	Die drei US-Distributionszentren waren eine volle Woche offline
Schaden	USD 192,5 Mio. Ergebnisbelastung in Q2
Personelle Folge	CIO David Bergen (8 Jahre im Amt) trat zurück
Ironie	SAP blieb Hauptlieferant von Levi Strauss

📖 Flyvbjerg, B.; Budzier, A. (2011): *Double whammy – how ICT projects are fooled by randomness and screwed by political intent.*

🔑 **Merksatz:** Der Titel des Papers beschreibt die zwei Ursachen: „**fooled by randomness**“ (Zufall/Unvorhersehbarkeit) und „**screwed by political intent**“ (politische Absichten).

A.3 ★ Die empirische Studie von Capers Jones

 Jones, Capers (2008): *Applied Software Measurement. Global Analysis of Productivity and Quality*, 3rd ed., McGraw-Hill.

- **Umfang:** Studie über **8.300 Projekte** (1991–2008)
- **Ziel:** Quantitative Messung von Softwaremerkmalen („**software metrics**“)

Untersuchte Größen:

- Durchschnittsalter der Software
- **Durchschnittliche Lebensdauer einer Programmiersprache: 15 Jahre**
- Verteilung der Projektarten (Neuentwicklung etc.)
- Entwicklungskosten
- **Function-Point-Umrechnung:** Java/C++: 1 FP $\approx$ **53 LOC** · C: 1 FP $\approx$ **128 LOC** · VB: 1 FP $\approx$ **32 LOC**
- Anzahl ausgelieferter Fehler / ausgelieferter **schwerer** Fehler
- **Planungstreue**
- Produktivität von **Generalisten und Spezialisten**
- **Anteil nachgebesserter Anforderungen**

A.4 ★★ Die CHAOS-Studie (Standish Group)

Was ist die CHAOS-Studie? Der **CHAOS Report** der **Standish Group** (erste Version **1994**, seither immer wieder aktualisiert) beschäftigt sich mit den **Erfolgs- und Misserfolgsfaktoren in IT-Projekten**. Sie gehört zu den **bekanntesten und wichtigsten Langzeitstudien** im Bereich Projektmanagement; seit 1994 wurden **über 40.000 Einzelprojekte** wissenschaftlich untersucht.

Die drei Fragen des Dozenten (Webrecherche-Auftrag):

1. Wie hoch ist der Anteil der Projekte, die **zumindest teilweise nicht erfolgreich** waren?
2. Was sind die **Hauptgründe** dafür?
3. Was ist die **Kritik** an der CHAOS-Studie?

Antworten (ergänzt — die Folien enthalten sie bewusst nicht, „OhneLoesung“!)

(1) Anteil: Rund $\frac{2}{3}$ bis $\frac{3}{4}$ **aller Projekte** sind nicht vollständig erfolgreich. Die Standish-Klassifikation kennt drei Kategorien:

Kategorie	Bedeutung	Größenordnung (klassisch 1994 / später)
Successful	in Zeit, im Budget, mit allen Funktionen	ca. 16 % (1994) → ca. 29–35 %
Challenged	fertiggestellt, aber verspätet, teurer und/oder mit weniger Funktionen	ca. 53 % → ca. 50 %
Failed / Impaired	abgebrochen	ca. 31 % → ca. 19 %

(2) ★★ Die drei Hauptgründe (exakt so in Klausur 2022 gefragt: „Nennen Sie die drei Hauptpunkte für das Scheitern von Projekten in der CHAOS-Studie“):

1. **Fehlende / unvollständige Anforderungen** (*incomplete requirements*) — mangelnde Anforderungserhebung
2. **Fehlende Nutzer-/Anwendereinbindung** (*lack of user involvement*)
3. **Fehlende Management-Unterstützung / unzureichende Ressourcen** (*lack of executive support & resources*)

(3) Kritik an der CHAOS-Studie:

- **Definition von „Erfolg“** ist rein auf das **magische Dreieck (Zeit, Kosten, Umfang)** verengt — ein verspätetes Projekt mit hohem Geschäftswert gilt als „challenged“
- **Methodik intransparent** (Stichprobenziehung, Fragebogen nicht offengelegt)
- **Selektionsverzerrung**: gescheiterte Projekte werden überproportional gemeldet
- Kritik von **Eveleens & Verhoef (2010)**: die Zahlen seien statistisch nicht haltbar

A.5 ★★★★★ FISCUS — der deutsche Klassiker (3× in Klausuren!)

📖 Mertens, P.: *Fehlschläge bei IT-Großprojekten der Öffentlichen Verwaltung.*

Projektziel

Es sollte ein **bundeseinheitliches IT-System für die Steuererhebung** geschaffen werden. Bislang hatte **jedes Bundesland ein eigenes IT-Verfahren** — Hardware, Betriebssysteme, Programmiersysteme und Datenspeicherung (index-sequentiell vs. relational) variierten stark.
Erhoffter Nutzen: vor allem **Kostenvorteile bei der Weiterentwicklung der Software.**

★ Der Projektverlauf — viermal neu aufgesetzt!

Phase	Organisation	Technologie	Hauptproblem(e)
1	Dezentral	CASE (Computer-assisted Software Engineering)	Lange Verhandlungen über Einzelwünsche der Bundesländer
2	Dezentral	Objektorientierung	Request-Broker-Middleware enttäuschte , zu lange Rechen- und Antwortzeiten
3	Dezentral	Framework / IBM-San-Francisco-Werkzeug	Frühzeitige Aufgabe von San Francisco durch IBM , u. a. wegen Ablösung von Client-Server durch Web-Technologie
4	Zentral (FISCUS GmbH)	Rückkehr zu COBOL auf Mainframe und Unix	Unnachgiebigkeit der Bundesländer

💥 **Der Gesamtschaden wird auf ca. 5 Mrd. € geschätzt.** Beginn: **Anfang der 1990er Jahre.**

★★★★ Die 6 Diagnose-Gründe (auswendig — 2-3 reichen meist!)

1. **Völlig dezentrale Projektorganisation** mit **mehr als zehn Standorten** — jeder Standort hatte eine **eigene IT-Projektleitung**
2. **Schwache Koordinationsstelle**, von den **16 Bundesländern nach POLITISCHEN Gesichtspunkten besetzt**. Vermutung: Die Bundesländer befürchteten, durch zu starke IT-Vereinheitlichung **steuerliche Gestaltungsspielräume einzubüßen**
3. Der **Anwendungsrahmen war sehr Informatik-orientiert** (statt fachlich/steuerrechtlich)
4. **Nachträgliche Integration heterogener, historisch gewachsener Software** + politische Interessengegensätze → Erfolg nur bei **kompetenter, mit großen Vollmachten ausgestatteter ZENTRALER Projektleitung** möglich (**Peer-to-Peer-Strukturen eignen sich hier nicht**). Der Heterogenität mit **Framework-Technik** zu begegnen, war „**noch zu ambitiös**“
5. **Lange Laufzeit** → die IT entwickelte sich beträchtlich weiter, gewisse Methoden wurden **zeitweise überbewertet (Überforderung der Objektorientierung!)**
6. **Personalproblem**: Die Bundesländer stellten oft wenig interessiertes Personal ab — Zitat eines Interviewpartners: „**Es wurde disponibles statt gutes Personal abgestellt.**“

🔑 **Das Fazit von Mertens (wörtlich):** „Rückblickend stellt sich FISCUS als ein großes Vorhaben dar, für welches die **Projektziele angesichts von ‚Stakeholdern‘ mit sehr unterschiedlichen (politischen) Vorstellungen nicht rechtzeitig verbindlich abgeklärt** wurden.“

Exportchancen: minimal — es gibt **außerhalb Deutschlands nirgendwo ein auch nur entfernt vergleichbar komplexes Steuerrecht.**

⚠️★ Die Single-Choice-Falle zu FISCUS (Klausur 2020)

Frage: „Was war **KEIN** Grund dafür, dass FISCUS scheiterte?“ A. **Personalmangel** · B. Überforderung der Objektorientierung · C. Projektmanagementfehler

Antwort: A — Personalmangel. ⚠️ **Nicht die MENGE, sondern die QUALIFIKATION des Personals war das Problem** („disponibles statt gutes Personal“). B (Phase 2: OO/Request-Broker enttäuschte) und C (dezentrale Projektorganisation, schwache Koordination) waren sehr wohl Gründe.

★ Warum scheiterte FISCUS beim zweiten Anlauf? (Klausur 2021)

Phase 2 war die **Objektorientierung**: Die **Request-Broker-Middleware enttäuschte** — die **Rechen- und Antwortzeiten waren zu lang**. Die Technologie war der Aufgabe schlicht **nicht gewachsen** („Überforderung der Objektorientierung“).

A.6 Weitere von Mertens untersuchte Großprojekte

Als Vergleichsfälle behandelt Mertens: **FISCUS**, **RUBIN** (Nürnberger fahrerlose U-Bahn U3), **Tiergartentunnel**, **DiPlaZ**, **eGK** (elektronische Gesundheitskarte), **Toll Collect**, **Hartz IV**.

Übergreifende Problempunkte (Auszug aus Mertens' Vergleichstabelle):

- **Ungünstige Relation Zeit zur Vorbereitung vs. Realisierung** (Bestätigung: **stark**)
- **Komplexität unterschätzt**, Integration IT-Mechanik
- **Besonderheiten öffentlicher Aufträge**: potenzielle Auftragnehmer zögern, sich zu bewerben (z. B. **IBM schied aus FISCUS aus**; für **Galileo** kam kein Konsortium zustande)

Handlungsempfehlungen von Mertens (Auszug):

1. **Sorgfältigere Abstimmung der Projektparameter Zeitrahmen und Projektinhalt**
2. Bei **Föderalismus-Reformen IT-Aspekte beachten**, um Debakel wie bei FISCUS zu vermeiden
3. **Kluft zwischen wenig IT-spezialisierten Beamten und hochspezialisierten Beratern überbrücken**
— im öffentlichen Dienstrecht **Fachspezialistenlaufbahn** mit guten Aufstiegs-/Verdienstmöglichkeiten schaffen

▼ ? Selbsttest Teil A

1. Nennen Sie die 3 Charakteristika eines Black-Swan-Ereignisses.
2. Was war der Auslöser des Levi-Strauss-Projekts? Was war der Schaden?
3. Wie viele Projekte umfasste die Capers-Jones-Studie und über welchen Zeitraum?
4. Wie lange ist die durchschnittliche Lebensdauer einer Programmiersprache?
5. Nennen Sie die 3 Hauptgründe für Projektscheitern nach der CHAOS-Studie.
6. Nennen Sie 3 Kritikpunkte an der CHAOS-Studie.
7. Was war das Ziel von FISCUS und welchen Schaden verursachte es?
8. Nennen und erläutern Sie **zwei** Gründe für das Scheitern von FISCUS. (*Klausurfrage!*)
9. Warum scheiterte FISCUS im **zweiten** Anlauf?
10. Warum war **Personalmangel** kein Grund für das Scheitern von FISCUS?

TEIL B — LE 2: SOFTWARETECHNIK (SOFTWARE ENGINEERING)

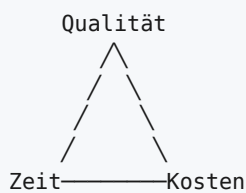
B.1 ★★ Die Definition (auswendig!)

Software-Technik ist die zielorientierte Bereitstellung und systematische Verwendung von Prinzipien, Methoden und Werkzeugen für die arbeitsteilige, ingenieurmäßige Entwicklung und Anwendung von umfangreichen Softwaresystemen.

Die Definition zerlegt (jedes Wort ist bedeutungstragend):

Baustein	Bedeutung
zielorientiert	→ das Magische Dreieck aus Kosten, Zeit und Qualität
Prinzipien	Grundsätze, die man seinem Handeln zugrunde legt
Methoden	planmäßig angewandte, begründete Vorgehensweisen
Werkzeuge	Tools zur Unterstützung
arbeitsteilig	mehrere Personen
ingenieurmäßig	systematisch, nachvollziehbar, nicht künstlerisch
umfangreiche Softwaresysteme	Komplexität ist das Kernproblem

Das Magische Dreieck



B.2 ★ Prinzipien

Prinzipien

- sind **Grundsätze**, die man seinem Handeln zugrunde legt
- sind **allgemein gültig, abstrakt und allgemeinsten Art**
- bilden eine **theoretische Grundlage**
- werden **aus Erfahrungen und Erkenntnissen hergeleitet** und durch sie **bestätigt oder widerlegt**

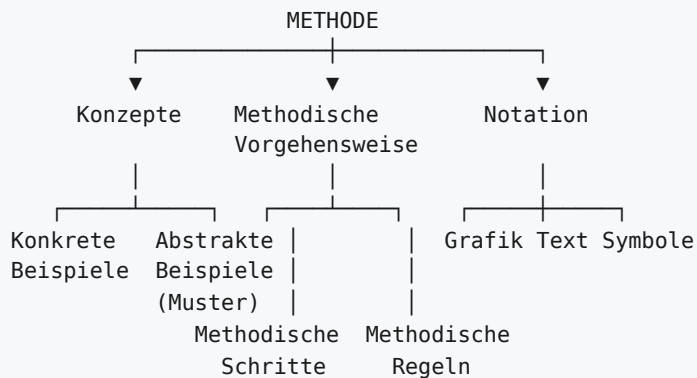
Die vier genannten Prinzipien (auswendig!):

1. **Prinzip der Abstraktion**
2. **Prinzip der Strukturierung**
3. **Prinzip der Hierarchisierung**
4. **Prinzip der Modularisierung**

B.3 ★ Methoden

Methoden sind **planmäßig angewandte, begründete Vorgehensweisen zur Erreichung von festgelegten Zielen**. In der Softwaretechnik wird der Begriff aber auch als **Oberbegriff von Konzepten, Notationen und methodischen Vorgehensweisen** verstanden.

Der Methodenbaum:



B.4 ★★ Methoden sind Zerlegungsstrategien

Zerlegungsstrategien haben die Aufgabe, die Komplexität des Systems in kleine handhabbare Teile zu zerlegen, die möglichst **STABIL** sind:

- einfach strukturierte Schnittstellen
- nur lokale Auswirkung von Änderungen (Entkopplung; **keine Seiteneffekte**)
- pragmatische Zerlegung, die nachvollziehbar ist

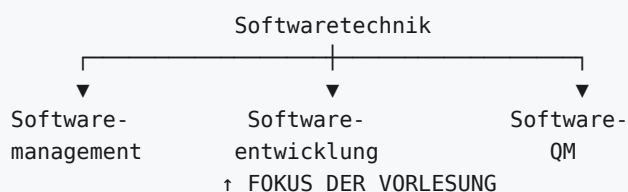
! Die Single-Choice-Frage dazu (Klausur 2024)

„Was ist eine korrekte Aufzählung von möglichen Arten bzw. Prinzipien der Zerlegung?“ **A.**

ereignisorientierte Zerlegung, funktionsorientierte Zerlegung ✓ B. klassische Zerlegung, objektorientierte Zerlegung C. agile Zerlegung, klassische Zerlegung

🔑 **Antwort A.** Die Vorlesung basiert auf dem **Paradigma der EREIGNISORIENTIERTEN Zerlegung** (siehe LE 5!). Es gibt keine „klassische“ oder „agile“ Zerlegung.

B.5 Teilaufgaben der Softwaretechnik



B.6 ★ Jede Sichtweise führt zu eigenen Modellen

Sicht	Modelle
Daten	ERM, OLAP-DM, DD (Data Dictionary), SA
Funktionen	Funktionsbaum, Geschäftsprozess, DFD (Datenflussdiagramm), Regeln, UML
Dynamik	Petri-Netz, Kontrollstrukturen, Sequenzdiagramm, SA/RT
GUI	Maskengenerator, Grafik-Editor

B.7 ★★ Die 6 Komplexitätsarten (auswendig!)

Komplexitätsart	Beispiel
Komplexität der Funktionen	Büroanwendungen: ca. 1000–1500 Funktionen
Komplexität der Daten	Datenbanksysteme
Komplexität der Algorithmen	komplexe numerische Berechnungen
Komplexität des zeitabhängigen Verhaltens	Betriebssysteme, Prozesssteuerungen
Komplexität der Systemumgebung (embedded systems)	Motormanagement, Kraftwerkssteuerungen
Komplexität der GUI	CAD-Systeme, Grafikprogramme

B.8 ★ Basiskonzepte je Komplexitätsart

Komplexität ...	Basiskonzepte	Anwendungsbereich
...der Daten	Data Dictionary, ERM , (Zustandsautomat), Klassendiagramm	Betriebswirtschaftlich
...der Funktionen	Klassendiagramm, Geschäftsprozesse , Datenflussdiagramm, Funktionsbaum	Technisch-wissenschaftlich
...der Algorithmen	Pseudocode, Programmablaufplan, Struktogramm, Entscheidungstabelle	Echtzeit
...der Systemumgebung	Datenflussdiagramm	
...des zeitabhängigen Verhaltens	Petri-Netz, Zustandsautomat, Aktivitätsdiagramm , Sequenzdiagramm, Kollaborationsdiagramm	
...der GUI	grafische Spezifikation, (Petri-Netz), (Zustandsautomat), Kontrollstrukturen	Mensch-Computer-Interaktion

🔑 **Merken:** Entscheidungstabellen gehören zur **Komplexität der ALGORITHMEN**, Aktivitätsdiagramme zur **Komplexität des ZEITABHÄNGIGEN VERHALTENS**, ERM zur **Komplexität der DATEN**.

B.9 ★ Einsatz der Basismethoden in Prozessmodell-Phasen

Phase	Methoden
Analyse	Funktionsbaum, Geschäftsprozess , Datenflussdiagramm, Zustandsautomat
Entwurf	Sequenzdiagramm, Petri-Netz, Kollaborationsdiagramm, ERM , Klassendiagramm, Entscheidungstabelle , Data Dictionary, Regeln
Implementierung (inkl. Feinentwurf)	Struktogramm, Pseudocode, Programmablaufplan

B.10 ★ Wie vergleicht man Methoden? (7 Kriterien)

? *Lohnt sich der Aufwand, eine bestimmte Methode zu lernen? Unterstützt die Methode die Problemlösung?*

1. Handhabbarkeit
2. Anwendungsbreite
3. Erlernbarkeit
4. Formalisierungsgrad
5. Werkzeugunterstützung
6. Unterstützung der Softwaretechnik (Prinzipien etc.)
7. Dokumentierbarkeit

B.11 ★★ Die semantische Lücke (semantic gap)

? Kann die Methode den *semantic gap* überwinden?

📝 **Beispiel:** SA (Structured Analysis) eignet sich **gut** für die Systemanalyse, ist aber **für den Systementwurf nicht einsetzbar**.

Phase	Analyse	Entwurf	Implementierung
Methode	SA	SD	C++

🔑 Die **semantische Lücke** ist der Bruch zwischen den Modellen zweier aufeinanderfolgender Phasen — eine Methode kann sie **nicht** allein überbrücken.

B.12 ★ Die 10 Forderungen an eine Methode (auswendig-Liste)

1. Beitrag zur Lösung des magischen Dreiecks
2. Methodenverbund muss möglich sein
3. Methode muss **mindestens für eine Softwareklasse** einsetzbar sein
4. **Gesicherte theoretische Erkenntnisse** als Basis
5. **Definierte und nachvollziehbare Arbeitsschritte und -ergebnisse**
6. **Dokumentierbarkeit und Überprüfbarkeit** der Ergebnisse
7. **Rechnerunterstützung muss möglich sein**
8. **Einfach erlernbar, lehrbar und praktizierbar**
9. **Softwaretechnik-Prinzipien müssen unterstützt werden**
10. **Aktive Unterstützung des Entwicklers:** *was soll getan werden, nicht: was soll nicht getan werden / ist verboten*

▼ ? Selbsttest Teil B

1. Geben Sie die Definition von Software-Technik wörtlich wieder.
2. Was ist das magische Dreieck?
3. Nennen Sie die 4 Prinzipien der Softwaretechnik.
4. Welche Aufgabe haben Zerlegungsstrategien? Nennen Sie die 3 Kriterien für „stabile“ Teile.
5. Nennen Sie die 6 Komplexitätsarten mit je einem Beispiel.
6. Zu welcher Komplexitätsart gehört die Entscheidungstabelle? Zu welcher das Aktivitätsdiagramm?
7. Was ist die semantische Lücke? Geben Sie das SA/SD-Beispiel wieder.
8. Nennen Sie 5 der 10 Forderungen an eine Methode.
9. Welche 3 Teilaufgaben hat die Softwaretechnik? Welche ist Fokus der Vorlesung?

TEIL C — LE 3: PROZESSMODELLE

C.1 ★ Was sind Prozessmodelle?

Jede Softwareentwicklung erfolgt in einem **organisatorischen Rahmen**, der **Aufbau-, insbesondere aber Ablauforganisation** eines Entwicklungsprojekts definiert. **Prozessmodelle beschreiben einen solchen Rahmen. Prozessmodelle bestehen aus definierten AKTIVITÄTEN, die definierte ERGEBNISSE (Produkte) liefern.**

ungetestetes Programm → Programmieren → getestetes Programm → Test

★★ Inhalt eines Prozessmodells (6 Punkte, auswendig!)

1. **Reihenfolge des Arbeitsablaufs**
2. **Jeweils durchzuführende Aktivitäten**
3. **Definition der Teilprodukte** einschließlich Layout und Inhalt
4. **Fertigstellungskriterien für Teilprodukte**
5. **Notwendige Mitarbeiterqualifikationen**
6. **Verantwortlichkeiten und Kompetenzen**

🔑 „Wir haben es also mit **Arbeitszerlegung und -synthese anhand von organisatorischen Referenzmodellen** zu tun!“

C.2 ★ Top-Ten-Risiken (nach Boehm) und Gegenmaßnahmen

#	Risiko	Maßnahmen
1	Personelle Defizite	Hochtalentierte Mitarbeiter einstellen; Teams zusammenstellen
2	Unrealistische Termin- und Kostenvorgaben	Detaillierte Kosten-/Zeitschätzungen mit mehreren Methoden ; Produkt an Kostenvorgaben orientieren; inkrementelle Entwicklung ; Wiederverwendung von Software; Anforderungen streichen
3	Entwicklung von falschen Funktionen	Benutzerbeteiligung ; Prototypen; frühzeitiges Benutzerhandbuch
4	Entwicklung der falschen GUI	Benutzerbeteiligung ; Prototypen; Aufgabenanalyse
5	„Vergolden“ (Gold Plating)	Anforderungen streichen; Prototypen; Kosten/Nutzen-Analyse; Entwicklung an Kosten orientieren
6	Kontinuierliche Anforderungsänderungen	Hohe Änderungsschwelle; inkrementelle Entwicklung (Änderungen auf später verschieben)
7	Defizite bei extern gelieferten Komponenten	Leistungstest; Inspektionen; Kompatibilitätsanalyse
8	Defizite bei extern erledigten Aufträgen	Prototypen; frühzeitige Überprüfung; Verträge auf Erfolgsbasis
9	Defizite in der Echtzeitleistung	Simulation; Leistungstest; Prototypen; Tuning
10	Überforderung der Softwaretechnik	Technische Analyse; Kosten/Nutzen-Analyse; Prototypen

🔑 **Beachte:** Risiko #10 „Überforderung der Softwaretechnik“ ist genau das, was bei **FISCUS** passierte (Objektorientierung, Framework-Technik)!

C.3 ★ Das WHISCY-Syndrom und „code & fix“

WHISCY = „Why isn't Sam coding yet??“

Konsequenz des WHISCY-Syndroms — das Prozessmodell „code & fix“:

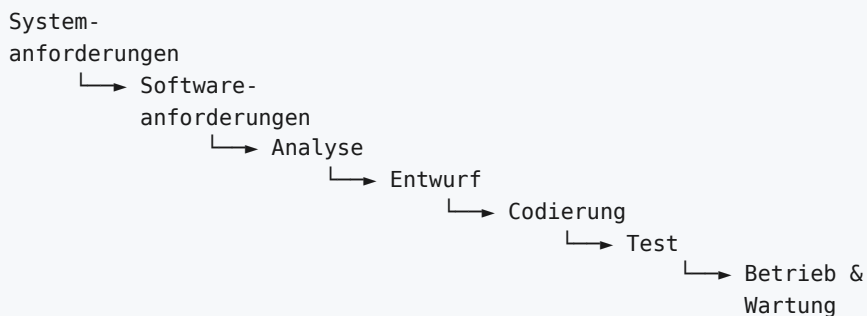
- ① Schreibe ein Programm
- ② Finde die Fehler und behebe sie
- ③ Wiederhole ① und ② bis das Programm stabil ist
- ④ Gehe nach den Schritten ① bis ③ davon aus, dass es ein tolles Programm geworden ist!

★★ Die 3 Nachteile von code & fix — und was daraus folgt

Nachteil	Folgerung
Nach der Behebung von Fehlern wird das Programm so umstrukturiert, dass weitere Fehlerbehebungen immer teurer werden	→ ENTWURFSPHASE notwendig
Anforderungen des Anwenders (fachliche Funktionen, Ergonomie) kommen zu kurz	→ ANALYSEPHASE notwendig
Fehler sind schwer zu finden , da Tests schlecht vorbereitet sind und Änderungen unzureichend durchgeführt werden	→ TESTPHASE notwendig

🔑 **Merksatz:** Aus den drei Nachteilen von code & fix leiten sich **genau die drei fehlenden Phasen** her — das ist die didaktische Herleitung des Wasserfallmodells!

C.4 ★★ Das Wasserfallmodell



★★ Die 6 Merkmale (auswendig!)

1. Jede Aktivität ist in der richtigen Reihenfolge und **VOLLSTÄNDIG** durchzuführen
2. **Am Ende jeder Aktivität steht ein fertiggestelltes DOKUMENT** → das Wasserfallmodell ist ein **dokumentengetriebenes** Prozessmodell
3. Der Entwicklungsablauf ist **SEQUENTIELL** — jede Aktivität muss beendet sein, bevor die nächste anfängt
4. Das Prozessmodell orientiert sich am **TOP-DOWN-Vorgehen**
5. Es ist **einfach, verständlich und benötigt wenig Managementaufwand**
6. **Anwender-/Auftraggeberbeteiligung ist NUR in der Definitionsphase** vorgesehen, die restlichen Phasen erfolgen intern

⚠ Die 4 Nachteile

1. Es ist **nicht immer sinnvoll, alle Entwicklungsschritte vollständig** abzuarbeiten
2. Es ist **nicht immer sinnvoll, alle Entwicklungsschritte sequentiell** abzuarbeiten
3. **Gefahr, dass die Dokumentation wichtiger wird als das eigentliche Produkt!**
4. **Risikofaktoren werden möglicherweise nicht erkannt**, da der einmal festgelegte Entwicklungsablauf **starr** durchgeführt wird

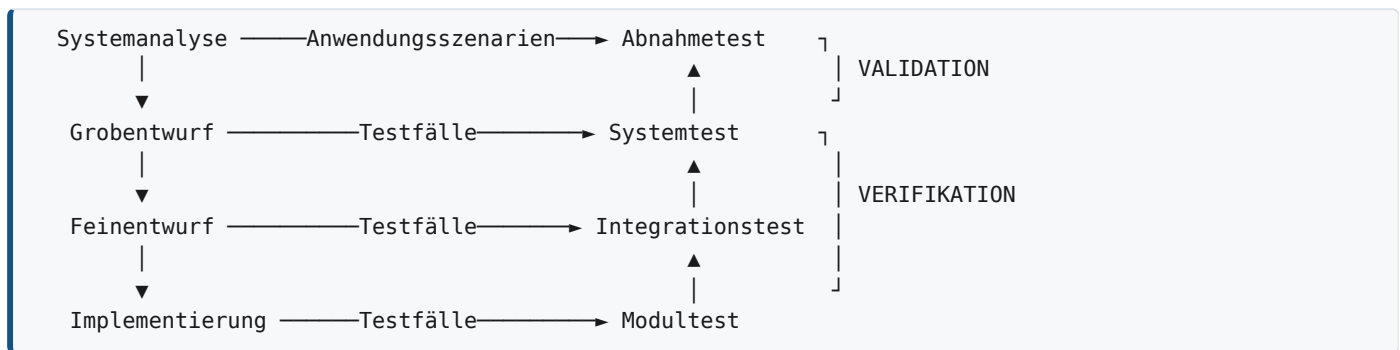
✅ **Trotzdem:** „Trotz dieser Nachteile hat das Wasserfallmodell **wesentlich zu einer plan-, steuer- und kontrollierbaren Softwareentwicklung** beigetragen (Projektmanagement!)“

C.5 ★★ Das V-Modell (Klausurfrage 2020: 4 Punkte!)

⚠ Es gibt ZWEI V-Modelle:

1. Das „klassische“ V-Modell als **Erweiterung des Wasserfallmodells** ← das ist in der Klausur gemeint!
2. Das **V-Modell 97 / V-Modell XT des Bundes** als Erweiterung des klassischen V-Modells (mit SE 1 ... SE 9 Submodellen)

★★ Das „klassische“ V-Modell



🔑 **Der Kerngedanke:** Jeder konstruktiven Phase auf dem linken Ast steht eine prüfende Phase auf dem rechten Ast gegenüber. Die **Testfälle** werden bereits in der jeweiligen Entwurfsphase definiert (Pfeile nach rechts).

★★★★ Verifikation vs. Validation (SEHR beliebte Frage!)

	VERIFIKATION	VALIDATION
Leitfrage	„Wird ein KORREKTES Produkt entwickelt?“	„Wird das RICHTIGE Produkt entwickelt?“
Was wird geprüft?	die formale Richtigkeit und Konsistenz der Transformation zwischen je zwei Ergebnissen	ob das IS den Anforderungen des Anwenders tatsächlich entspricht
Automatisierbar?	Liegen die Ergebnisse in formalisierter Form vor, ist u. U. eine computergestützte, vollautomatische Verifikation möglich	nein
Anwenderbeteiligung	nein	✅ Findet IMMER mit Anwenderbeteiligung statt
Position im V	Grobentwurf/Feinentwurf/Implementierung ↔ System-/Integrations-/Modultest	Systemanalyse ↔ Abnahmetest

🔑 **Der Merksatz:** „Formale Korrektheit (Verifikation) bedeutet **nicht**, dass ein IS den Anforderungen des Anwenders tatsächlich entspricht.“

C.6 ★★★★★ Die 2 Arten von QS-Maßnahmen (Klausurfrage 3×!)

Analytische QS-Maßnahmen (→ v. a. **PRODUKT**qualität)

Diagnostische Maßnahmen zur MESSUNG des erreichten Qualitätsniveaus eines Prüflings (Produkt/Prozess).

Analysierende Maßnahmen	Testende Maßnahmen
Verifikation	Dynamischer Test
Review	Simulation
Walk Through	Schreibtischtest
Inspektion	
(manuell)	(maschinell / manuell)

Konstruktive QS-Maßnahmen (→ v. a. **PROZESS**qualität)

Methoden, Sprachen, Werkzeuge, Richtlinien, Standards und Checklisten, die dafür sorgen, dass das entstehende Produkt bzw. der Erstellungsprozess A PRIORI bestimmte Eigenschaften aufweist.

Technische Maßnahmen	Organisatorische Maßnahmen
Methoden	Richtlinien
Programmiersprachen	Standards
Werkzeuge	Checklisten

★★ Die Merkregel

	analytisch	konstruktiv
Zeitpunkt	im Nachhinein (a posteriori)	im Vorhinein (a priori)
Ziel	MESSEN des Qualitätsniveaus	SICHERSTELLEN von Eigenschaften
Fokus	Produktqualität	Prozessqualität

📝★★ Anwendung auf Scrum (Klausurfrage 2020 & 2024!)

„Erläutern Sie analytische und konstruktive Qualitätssicherung bei Scrum.“

Scrum-Element	QS-Art	Begründung
Sprint Review	analytisch	Das fertige Inkrement wird geprüft/gemessen — Produktqualität
Sprint Retrospektive	★ KONSTRUKTIV	Es wird der PROZESS verbessert, damit künftige Inkremente a priori bessere Eigenschaften haben — Prozessqualität
Definition of Done	konstruktiv	Checkliste → a priori festgelegte Eigenschaften
Definition of Ready	konstruktiv	Checkliste für die Sprint-Aufnahme
Daily Scrum	konstruktiv	Prozessorganisation
Testgetriebene Entwicklung, Coding Standards, Pair Programming	konstruktiv	Methoden/Richtlinien
Unit-/Akzeptanztests	analytisch	testende Maßnahmen

⚠️★★ **DIE KLAUSURFRAGE (2024 SC + 2024/12 offen):** „Ist die Sprint Retrospektive eine konstruktive oder analytische QS-Maßnahme?“ → **KONSTRUKTIV**, denn sie zielt auf die **Verbesserung des PROZESSES** ab (Prozessqualität), nicht auf die Messung eines Produktzustandes. (In der SC-Version 2024 lautete die richtige Antwort entsprechend „B. der konstruktiven Qualitätssicherung“.)

C.7 ★★ Das Prototypen-Modell

! Die 4 Probleme traditioneller Prozessmodelle

1. Traditionelle Modelle verlangen eine **vollständige Spezifikation der Anforderungen zu PROJEKTBEGINN**
2. Oftmals ist eine **wechselseitige Kooperation zwischen Entwicklern und Anwendern** notwendig
3. Für manche Problemstellungen sind **alternative Lösungen** möglich, die **diskutiert** werden müssen
4. Die **Realisierbarkeit mancher Anforderungen** (z. B. Echtzeit) kann **nur praktisch erprobt** werden

Definition

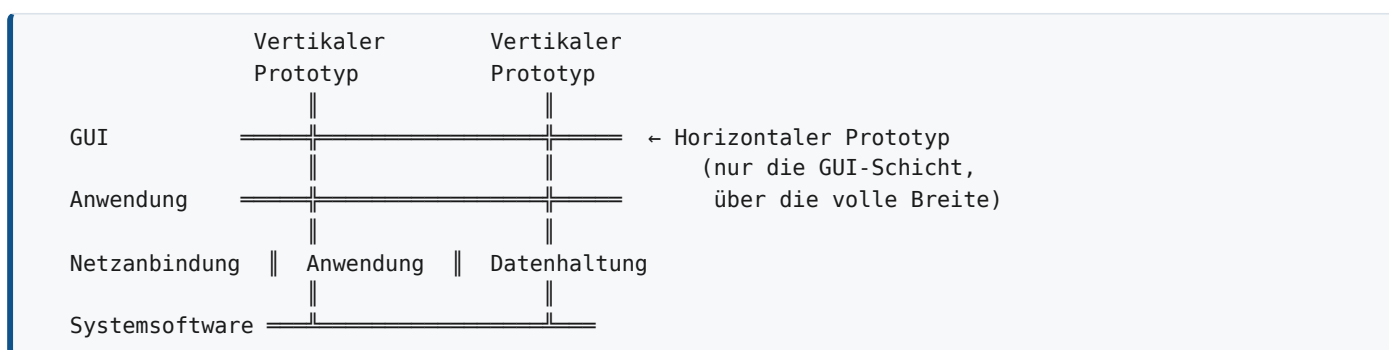
Das Prototypen-Modell unterstützt auf systematische Weise die frühzeitige Erstellung ablauffähiger MODELLE (Prototypen) des zukünftigen Informationssystems. Motto: „Redo until Right“

★★★★ Die 3 ARTEN von Prototypen (Klausurfrage 2× — auswendig!)

#	Art	Synonym	Zweck
1	Experimenteller Prototyp	Labormuster	Technische Machbarkeit prüfen — kann eine Anforderung überhaupt realisiert werden? (z. B. Echtzeitverhalten)
2	Explorativer Prototyp	Demonstrationsprototyp, „Prototyp i. e. S.“	Anforderungen erkunden und mit dem Anwender abstimmen — Demonstration/Akquise
3	Evolutionärer Prototyp	Pilotsystem	Wird zum Produkt weiterentwickelt — der Prototyp ist der Kern des späteren Systems

🔑 **Merkhilfe:** Experimentell = Erproben (Technik) · Explorativ = Erkunden (Anforderungen) · Evolutionär = Entwickeln (wird zum Produkt)

★ Horizontale vs. vertikale Prototypen



	Horizontaler Prototyp	Vertikaler Prototyp
Umfang	eine SCHICHT über die volle Breite (z. B. die komplette GUI, aber ohne Funktion dahinter)	eine Funktion durch ALLE Schichten hindurch (GUI → Anwendung → Datenhaltung)
Typischer Einsatz	Oberflächen-/Ergonomieabstimmung	technische Machbarkeit einer Kernfunktion

! Die Klausurfrage 2020 lautet: „Erläutern Sie die **drei Arten von Prototypen**, die es **NEBEN der Unterscheidung in vertikale und horizontale** Prototypen gibt.“ → Gemeint sind **experimentell, explorativ, evolutionär!**

Das Prototypen-Prozessmodell (Zuordnung der Prototypen zu den Phasen)

Phase	Ergebnis	Prototyp
Planen (Akquise)	Durchführbarkeitsstudie	Demo-PT
Definieren	Produktmodell	PT i. e. S. (explorativ)
Entwerfen	Produktarchitektur	Labormuster (experimentell)
Implementieren	Produkt	Pilotsystem (evolutionär)

C.8 ★★ Der Kreislauf des Verstehens (5 Schritte)

Merke: Der Kreislauf des Verstehens ist der **SCHLÜSSEL** zu einer erfolgreichen Systemanalyse, an deren Ende eine erfolgreiche Systemspezifikation steht! 📖 Beer, J.: *Systemspezifikation*, in: Siedersleben, J. (Hrsg.): *Softwaretechnik. Praxiswissen für Softwareingenieure*, 2. Aufl., München/Wien 2003, S. 21–48.

1. Der ANWENDER erzählt dem Systemanalytiker, wie er sich eine neue Funktion vorstellt.



2. Der SYSTEMANALYTIKER schreibt das, was er verstanden hat, in SEINER Darstellung nieder: Objektdiagramme, Ablaufpläne, Pseudocode etc.

- Die intellektuelle Leistung: die Vorstellungen des Anwenders STRUKTURIEREN, Komplexität ERKENNEN und DURCHDRINGEN, die KONSISTENZ sichern.



3. Seinen Lösungsvorschlag ÜBERSETZT der Systemanalytiker in die Darstellung des ANWENDERS in DESSEN Sprache (z. B. eine Dialogbeschreibung).



4. Dieses Dokument oder den Prototyp zeigt er dem Anwender. Er wird ÜBERRASCHT feststellen, dass er vieles NICHT oder NICHT RICHTIG verstanden hat, was ihm vorher völlig klar erschien.



5. Am Dokument für den Anwender werden jetzt Änderungen, Verbesserungen, Erweiterungen vorgenommen → und dann wieder bei Schritt 2 weitermachen.

💬 Das berühmte Zitat des typischen Anwenders: „I can't tell you what I want, but I'll know it when I see it!"

★ Bewertung des Prototypen-Modells

✓ Vorteile	✗ Nachteile
Reduzierung des Entwicklungsrisikos	Höherer Entwicklungsaufwand
Integration in andere Prozessmodelle möglich	Gefahr, dass „Wegwerf“-PT aus Zeitmangel doch Teil des Endsystems wird
Geeignete Werkzeuge vorhanden	Verträge für SE berücksichtigen vielfach PT nicht
Verbesserung der Planung und der Kreativität von Lösungen	Wird als Ersatz für fehlende Dokumentation betrachtet
Einbezug des späteren Anwenders	Grenzen und Restriktionen eines PT sind oft nicht bekannt

C.9 ★★★ Evolutionäres vs. inkrementelles Prozessmodell (Klausurfrage 2×!)

Das evolutionäre Prozessmodell

Ausgangspunkt sind die KERN- bzw. MUSS-Anforderungen des Auftraggebers:

1. Diese Anforderungen definieren den **Produktkern**
2. **NUR dieser Produktkern wird entworfen und entwickelt**
3. Das **Kernsystem, auch NULLVERSION genannt**, wird an den Auftraggeber ausgeliefert
4. Der Auftraggeber **sammelt Erfahrungen mit der Nullversion** und ermittelt daraus seine **Produktanforderungen für eine erweiterte Version**
5. Die Nullversion wird anschließend **erweitert**. Die neue Produktversion wird eingesetzt. Neue Anforderungen werden ermittelt ...

Die 4 Merkmale:

1. Das Informationssystem wird **allmählich und stufenweise** entwickelt
2. **Pflegeaktivitäten werden ebenfalls als Erstellung einer neuen Version betrachtet**
3. **Gut geeignet, wenn der Auftraggeber seine Anforderungen noch nicht vollständig überblickt**
4. Die Entwicklung ist **CODE-getrieben** — man konzentriert sich jeweils auf **lauffähige Teilprodukte**

Bewertung:

✓	✗
Auftraggeber erhält in kürzeren Zeitabständen ein einsatzfähiges Produkt (z. B. halbjährlich)	⚠ Gefahr, dass die Nullversion wichtige KERNanforderungen übersehen hat und deshalb die gesamte Architektur überarbeitet werden muss
Kann mit dem PT-Modell kombiniert werden (Pilotsystem-Entwicklung ≈ evolutionäre Entwicklung)	⚠ Gefahr, dass die Nullversion nicht flexibel genug weiterentwickelbar ist
Frühzeitiger Einsatz erlaubt Analyse der Auswirkungen auf die Geschäftsprozesse	
Entwicklung in kleineren Arbeitsschritten überschaubarer Größe → Korrekturen der Entwicklungsrichtung möglich	
Kein „Big Bang“-Endtermin , sondern viele Zwischentermine mit einsatzfähigen Zwischenergebnissen	

★★★ Das inkrementelle Prozessmodell — DER UNTERSCHIED

Die Nachteile des evolutionären Modells lassen sich dadurch beheben, dass **NICHT EIN PARTIELLES, SONDERN EIN VOLLSTÄNDIGES** Informationssystem modelliert wird. Analog zum evolutionären Modell wird dann nur ein **TEIL** der Anforderungen umgesetzt. Da bei der inkrementellen Vorgehensweise bereits ein **VOLLSTÄNDIGES ANALYSEMODELL** vorliegt, ist sichergestellt, dass die inkrementellen Erweiterungen zum bisherigen System **PASSEN**.

DIE MUSTERANTWORT (Klausurfrage: „Erläutern Sie den Unterschied“)

Beide Modelle liefern das System stufenweise in Versionen aus und setzen zunächst nur einen Teil der Anforderungen um. Der entscheidende Unterschied liegt in der DEFINITIONS-/ANALYSEPHASE:

- **Evolutionär:** Es wird **nur ein PARTIELLES Modell** erstellt — nämlich das des Produktkerns (Nullversion). Weitere Anforderungen ergeben sich **erst aus den Erfahrungen** mit der ausgelieferten Version. → **Risiko:** Kernanforderungen übersehen, Architektur muss später komplett überarbeitet werden.
- **Inkrementell:** Es wird **von Anfang an ein VOLLSTÄNDIGES Modell (Definition) des Gesamtsystems** erstellt; **danach** wird das System **in Inkrementen umgesetzt**. → **Vorteil:** die Erweiterungen **passen garantiert zum bisherigen System**, das Architekturrisiko sinkt.

 **Eine-Satz-Merkhilfe:** Evolutionär = „erst bauen, dann verstehen“ · Inkrementell = „erst vollständig verstehen, dann stückweise bauen“.

C.10 **DIE ZUSAMMENFASSUNGSTABELLE (Folie 30 — auswendig!)**

Modell	Primäres Ziel	Fokus	Benutzer-beteiligung	Merkmale
Wasserfallmodell	minimaler Managementaufwand	Dokumente	gering	sequentiell, volle Breite
V-Modell	maximale Qualität	Dokumente	gering	sequentiell, volle Breite, QS
Prototypen-Modell	Risikominimierung	Code	hoch	nur Teilsysteme (horizontal/vertikal)
Evolutionäres Modell	minimale Entwicklungszeit	Code	mittel	sofort: nur Kernsystem
Inkrementelles Modell	min. Entwicklungszeit + Risikominimierung	Code	mittel	volle Definition; dann zunächst Kernsystem

 **Diese Tabelle ist die Antwort auf jede Prozessmodell-Vergleichsfrage!**

▼ ? Selbsttest Teil C

1. Nennen Sie die 6 Inhalte eines Prozessmodells.
2. Was ist das WHISCY-Syndrom? Was folgt daraus?
3. Nennen Sie die 3 Nachteile von code & fix und die daraus abgeleiteten Phasen.
4. Nennen Sie die 6 Merkmale des Wasserfallmodells.
5. Erläutern Sie das klassische V-Modell. *(Klausurfrage, 4 Punkte!)*
6. Verifikation vs. Validation — Leitfragen und Unterschiede?
7. Was sind analytische, was konstruktive QS-Maßnahmen? Nennen Sie je 3 Beispiele.
8. Ist die Sprint Retrospektive analytisch oder konstruktiv? Begründen Sie!
9. Nennen und erläutern Sie die 3 Arten von Prototypen.
10. Was ist ein horizontaler, was ein vertikaler Prototyp?
11. Nennen Sie die 5 Schritte des Kreislaufs des Verstehens.
12. Welches Zitat beschreibt den typischen Anwender?
13. Erläutern Sie den Unterschied zwischen evolutionärem und inkrementellem Prozessmodell.
14. Was ist die „Nullversion“?
15. Ordnen Sie zu: Welches Modell hat welches primäre Ziel, welchen Fokus, welche Benutzerbeteiligung?

TEIL D — LE 4: AGILES SOFTWARE ENGINEERING

Motto der LE: „Elefanten isst man **scheibchenweise**“ (LE 3 hieß: „Elefanten isst man am Stück“)

D.1 ★ Die drei Leitfragen des Kapitels

Der Dozent strukturiert das gesamte Kapitel entlang **drei provokanter Fragen**:

1. **Hype, Mode, Zukunft?**
2. **Grenzerfahrungen des Software Engineering?**
3. **Zu dumm für ingenieurmäßiges Arbeiten?**

D.2 ★★ Schwergewichtige SE-Prozesse (Klausurfrage: „Warum schwergewichtig?“)

Das „**klassische**“ **Software Engineering** mit seinen Prozessen und Methoden wird als **SCHWERGEWICHTIG** bezeichnet.

★★★★ Die 6 Vorwürfe (auswendig!)

#	Vorwurf	Erläuterung
1	Rigider Formalismus	sinnlose Einhaltung von Standards
2	Dokumentenfixierung	„Schrackware statt Software“
3	Zu wenig Kommunikation	im Team und mit dem Kunden
4	Management nimmt sich zu wichtig	Konzentration auf Planung, Steuerung und Kontrolle ; Prozessbeherrschung wird mit Kompetenz verwechselt
5	Moving Targets nicht beherrschbar	laufende Änderungen der Kundenwünsche nicht erfassbar
6	Time to Market wird behindert	schnelle Verfügbarkeit des IS oftmals wichtiger als 100 % Realisierung aller Anforderungen

Die zwei Belege des Dozenten

(a) Innovative Systeme sind meistens nicht vollständig spezifizierbar:

- **Covisint** als Plattform: muss **evolutionär** weiterentwickelt werden
- Mitte der 90er Jahre waren die meisten **E-Business-Systeme zu neuartig**, um abschließend spezifiziert werden zu können

(b) ★ Eine vollständige, formale Spezifikation ist nicht immer wirtschaftlich sinnvoll — die Space-Shuttle-Rechnung:

Beim **Space Shuttle** steckt in **jeder LOC 1 Personenmonat Gesamtaufwand**. Umgerechnet auf ein typisches betriebliches IS:

- **100–200 KLOC** typische Größe
- **Kosten: 1 bis 2 Milliarden Euro**
- **Dauer: bei 1000 Mitarbeitern immer noch 16 Jahre** — und dabei ist **Brooks' Law** noch gar nicht berücksichtigt!

 *Brooks' Law: „Adding manpower to a late software project makes it later.“ (The Mythical Man-Month)*

★★★★ Fazit 1: Grenzerfahrungen? — die drei Merksätze

- **Formalismus bedeutet nicht Disziplin**
- **Prozess bedeutet nicht Fähigkeit**
- **Dokumentation bedeutet nicht Verständnis**

D.3 ★★ Leichtgewichtige SE-Prozesse

Leichtgewichtige SE-Prozesse fordern den Einsatz eines **MINIMALEN SATZES** an SE-Methoden und -Prozessen unter Konzentration auf das, was der Kunde will: **LAUFFÄHIGE SOFTWARE**.

★ Die Effektivitäts-/Effizienz-Formel (Klausur-Gold!)

Schwergewichtige Prozesse	= Effektivität (<i>doing the right things</i>)
Leichtgewichtige Prozesse	= Effektivität (<i>doing the right things</i>) UND Effizienz (<i>doing the things right</i>)

🔑 **Merke:** Leichtgewichtige SE-Prozesse beanspruchen für sich: „DOING THE RIGHT THINGS RIGHT!“

Definition Agile Softwareentwicklung

- **Agil** = beweglich, wendig, geschäftig
- Bei der agilen SE wird **hoher Wert auf die ÄNDERBARKEIT der Software** gelegt. Dabei sind **nicht die eingesetzten Verfahren** von Bedeutung, **sondern die erzielten Ergebnisse**.
- **Einziges Ziel und Erfolgskriterium ist das LAUFFÄHIGE IS!**

🔑 **Merke:** Agile SE = Leichtgewichtiges Software Engineering

D.4 ★★★★★ DAS AGILE MANIFEST (Klausurfrage 3× — auswendig, auch auf Englisch!)

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions	over	processes and tools
Working software	over	comprehensive documentation
Customer collaboration	over	contract negotiation
Responding to change	over	following a plan

That is, while there is value in the items on the right, we value the items on the LEFT more.

📖 Cockburn, A.: *Agile Software Development*, Boston et al. 2002, p. 213 · www.agilealliance.org

★ Deutsche Gegenüberstellung (Coldewey)

Schwergewichtiges SWE	Leichtgewichtiges SWE
Detaillierte Prozesse und Werkzeuge	Konzentration auf Kommunikation und Kollaboration
Umfangreiche Dokumente sind notwendig	Laufendes IS wichtiger als Dokumente
Detaillierte Verträge	Zusammenarbeit mit Kunden wichtiger als detaillierte Verträge
Konzentration auf Planung, Steuerung und Kontrolle	Fähigkeit, auf Änderungen zu reagieren, wichtiger als Planerfüllung

! ★ Die Single-Choice-Falle (Klausur 2020)

„Was ist **KEIN** Punkt des Agilen Manifests?“ A. „Individuals and interactions over processes and tools“ **B. „Team skills over financial incentives“** ✓ ← **das ist der Fake!** C. „Customer collaboration over contract negotiation“

⚠ **Der letzte Satz ist wichtig:** Die Werte auf der **rechten Seite haben SEHR WOHL einen Wert** — sie werden nur **geringer gewichtet**. Wer in der Klausur schreibt „Dokumentation ist unwichtig“, verliert Punkte!

D.5 ★★ Die 12 Prinzipien der agilen SE

1. **Kundenzufriedenheit** durch **schnelle und kontinuierliche Auslieferung lauffähiger Software**
2. **Änderungen begrüßen** und Änderungen ermöglichen
3. **Kontinuierliche Auslieferung** von Software **innerhalb kurzer Intervalle**
4. **Kunden und Entwickler arbeiten eng und TÄGLICH zusammen**
5. **Professionalität der MENSCHEN, nicht der Techniken**. Vertrauen in die Menschen, nicht in die Techniken
6. **Kommunikation findet primär FACE-TO-FACE statt**
7. **Lauffähige Software ist das Maß zur Messung des Fortschritts**, nicht die Dokumentation
8. **Nachhaltige SE** durch entsprechende organisatorische Maßnahmen (**Burnout verhindern; keine Überstunden langfristig zulassen**)
9. **Konzentration auf technische Qualität und gutes Design**
10. **So einfach wie möglich entwickeln**. Minimierung der Arbeit, die nicht notwendig gewesen wäre
11. **Selbstorganisation des Teams zulassen**
12. **Regelmäßige Reviewrunden des Teams** mit Rückkopplung in den SE-Prozess

★ Fazit 2: Zu dumm für Software Engineering?

„Agile Softwareentwicklung **ist** Software Engineering und eines Softwareingenieurs durchaus würdig. Ein **dummer** Softwareingenieur würde gar nicht auf die Idee kommen, von Standards, Richtlinien, Normen, Vorgaben etc. abweichen zu wollen, um **situativ selbständig zu entscheiden**.“

D.6 ★ Taxonomie agiler Verfahren (Coldewey)

Achsen: **Flexibilität (x) ↔ Detaillierungsgrad (y)** = Umfang, in dem das Verfahren das Vorgehen festlegt

Verfahren	Position
Extreme Programming (XP)	hoher Detaillierungsgrad
Feature Driven Development (FDD)	
Dynamic Systems Development Method (DSDM)	„Verfahren mit Startvorschlägen“
Crystal Methodenfamilie	„Effizienzregeln“
Scrum	
Adaptive Software Development (ASD)	hohe Flexibilität, „lernendes Team“

D.7 ★ Die Crystal-Methodenfamilie (Cockburn)

- „Crystal“ bedeutet nichts Besonderes — es ist lediglich eine METAPHER
- In Abhängigkeit von **Projektgröße** und **Kritikalität** des IS werden **unterschiedliche Methoden** eingesetzt
- Cockburn versteht Crystal **nicht als verbindlichen Methodensatz**, der strikt einzuhalten ist
- **Crystal ist lediglich ein BEISPIEL für die Entwicklung einer Methodenfamilie.** Es sollte **bestenfalls adaptiert** werden — dabei sind aber die **Prinzipien der agilen SE unbedingt einzuhalten**

★ Die Crystal-Matrix

Y-Achse: Möglicher Schaden (Kritikalität)

Code	Bedeutung
L	Loss of life — Verlust von Menschenleben
E	Essential money — „ Essenzielle “ finanzielle Verluste
D	Discretionary money — Mäßige finanzielle Verluste
C	Comfort — Verlust von Komfort

X-Achse: Teamgröße → 6 / 20 / 40 / 80 Personen → Farbnamen **Clear** / **Yellow** / **Orange** / **Red**

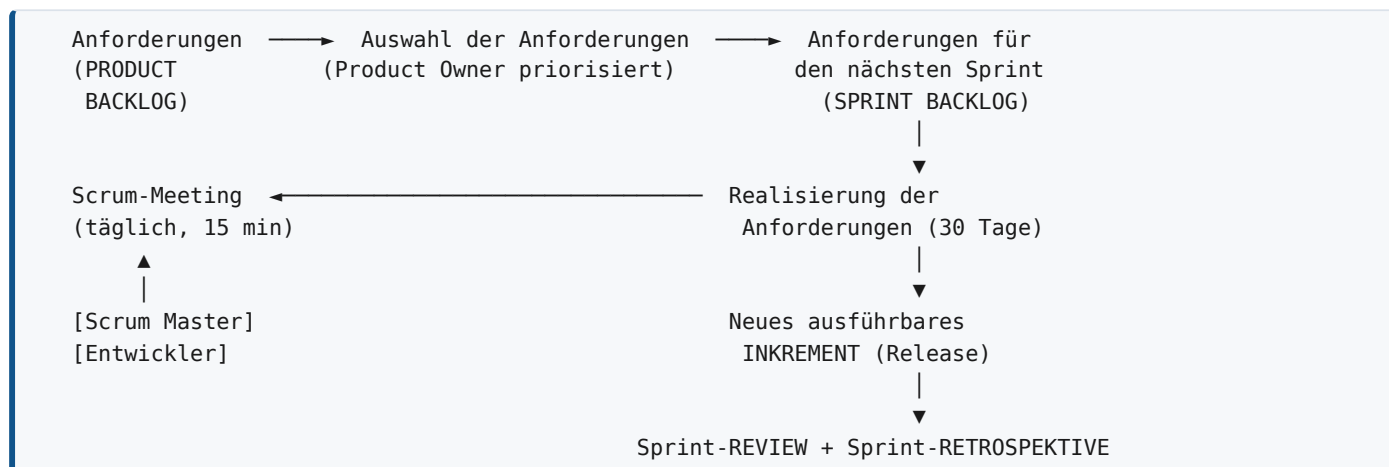
	Clear (6)	Yellow (20)	Orange (40)	Red (80)	
L	L6	L20	L40	L80	← Menschenleben
E	E6	E20	E40	E80	← essenzielles Geld
D	D6	D20	D40	D80	← mäßiges Geld
C	C6	C20	C40	C80	← Komfort

Beispiel: Crystal Clear (Kategorie D6)

- **Kategorie D6:** Teamgröße **ca. 6 MA**, lediglich **finanzieller Verlust** (schmerzlich, aber unkritisch)
- **1 Team, 1 Ort** (räumliche Einheit ist wichtig)
- **Rollen:** Sponsor, Senior Designer, Designer-Programmierer, Kunde. **Einer davon ist Projektkoordinator**
- Software wird **regelmäßig inkrementell alle 2–3 Monate** ausgeliefert
- **Fortschritt wird v. a. anhand ausgelieferter SW als Meilenstein** gemessen
- **Automatisierte Regressionstests** implementiert
- **Direkter Kundeneinbezug, zwei Kundenreviews pro Release**
- Typische Produkte/Dokumente: Releases, Zeitplan für Reviews, **Use Cases**, Designszeichnungen, GUI-Entwürfe, grobes Objektmodell, Quellcode, Migrationstools, Testfälle, Benutzerhandbuch
- **Vorlagen, Codingstandards, QS-Standards sind Sache des TEAMS**
- **Wichtigste Tools:** Compiler, KM-Tool (Konfigurationsmanagement), **Whiteboard**

D.8 ★★★★★ SCRUM (Klausurfrage in JEDER Klausur!)

Das Scrum-Prozessmodell



★★★★★ „Nennen und erläutern Sie die KOMPONENTEN von Scrum" (10 Punkte in Klausur 2021!)

A) Die 3 ROLLEN

Rolle	Aufgaben
Product Owner (PO)	Verantwortlich für den Product Backlog und dessen Priorisierung ; vertritt die Stakeholder/den Kunden ; maximiert den Business Value ; nimmt das Inkrement ab
Scrum Master (SM)	⚠ KEINE fachliche Leitung! Sorgt für die Einhaltung des Scrum-Prozesses , beseitigt Hindernisse (Impediments) , moderiert die Events, unterstützt das Team, schützt es vor Störungen von außen
Entwicklungsteam / Entwickler	Selbstorganisiert, crossfunktional , setzt die Anforderungen um, schätzt den Aufwand, ist gemeinsam für das Inkrement verantwortlich

B) Die 3 ARTEFAKTE

Artefakt	Beschreibung
Product Backlog	priorisierte Liste aller Anforderungen (Epics, User Stories) an das Produkt; lebendes Dokument , gepflegt vom PO
Sprint Backlog	die für diesen Sprint ausgewählten Backlog-Elemente + Umsetzungsplan; gehört dem Team
Inkrement (Release)	das am Sprintende lauffähige, potenziell auslieferbare Produktinkrement

C) Die 4-5 EVENTS (Zeremonien)

Event	Dauer	Zweck
Sprint Planning	zu Sprintbeginn	Auswahl der Anforderungen aus dem Product Backlog in das Sprint Backlog (Definition of Ready!)
Sprint	30 Tage (laut Folie; heute meist 1-4 Wochen)	Realisierung der Anforderungen
Daily Scrum / Scrum-Meeting	täglich, 15 Minuten	Synchronisation im Team, Hindernisse benennen
Sprint Review	am Sprintende	Inkrement vorstellen und abnehmen → ANALYTISCHE QS
Sprint Retrospektive	am Sprintende	PROZESS verbessern → KONSTRUKTIVE QS

D) Die 2 „Commitments“/Checklisten

- **Definition of Done (DoD)** — wann ist ein Backlog-Item **fertig**?
- **Definition of Ready (DoR)** — wann darf ein Item **in den Sprint aufgenommen** werden?

!☆☆ Die Scrum-Fallen in Single-Choice

Frage	Antwort	Begründung
„Welche Rolle spielt das Projektmanagement bei Scrum?“ (2020)	B. hilft dem Team	Scrum kennt keine klassische PL-Rolle; wo PM existiert, unterstützt es
„Was ist KEINE Aufgabe des Scrum Masters?“ (2021, 2022)	A. Die FACHLICHE LEITUNG des Teams ✓	SM leistet fachliche Unterstützung und fachliche Moderation — aber niemals die LEITUNG ! Das Team ist selbstorganisiert
„Was ist KEINE Aufgabe des Scrum Masters?“ (2024 — andere Optionen)	B. Die ARBEITSPLANUNG des Teams ✓	Die Arbeitsplanung macht das Team selbst (Selbstorganisation!). SM macht Moderation der Retrospektiven und Unterstützung bei Problemen

🔑 Die Merkregel für alle SM-Fragen: Alles, was nach **LEITEN**, **PLANEN** oder **ANWEISEN** klingt, ist **NICHT** Aufgabe des Scrum Masters. Alles, was nach **MODERIEREN**, **UNTERSTÜTZEN**, **HINDERNISSE BESEITIGEN** klingt, schon.

D.9 ☆☆☆ Skalierung: LeSS, SAFe, Spotify

Die Materialien nennen drei Skalierungsansätze:

- **LeSS** — Large Scale Scrum
- **SAFe** — Scaled Agile Framework (Big Picture)
- **Spotify Engineering Culture** (Henrik Kniberg) — Squads, Tribes, Chapters, Guilds

☆☆☆ Der Area Product Owner (Klausurfrage 2024/12, 2 Punkte!)

📖 In LeSS (Large Scale Scrum) wird bei sehr großen Produkten das Product Backlog in „Requirement Areas“ (Anforderungsbereiche) aufgeteilt.

Ein Area Product Owner (APO)

- ist der **Product Owner** für **GENAU EINEN Anforderungsbereich (Requirement Area)**
- **priorisiert und pflegt das Area Product Backlog** — einen fachlich zusammengehörigen Ausschnitt des Gesamt-Product-Backlogs
- **arbeitet mit mehreren Teams** dieses Bereichs (typisch: 4–8 Teams)
- **berichtet an / stimmt sich ab mit dem einen übergeordneten Product Owner**, der weiterhin das **Gesamtprodukt** verantwortet und die **Priorisierung ÜBER die Bereiche hinweg** vornimmt

🔑 **Kurzantwort für die Klausur (2 Punkte):** „Ein Area Product Owner ist in LeSS der Product Owner für einen einzelnen Anforderungsbereich (Requirement Area) eines großen Produkts. Er priorisiert und pflegt das Area Product Backlog für die Teams seines Bereichs und stimmt sich mit dem übergeordneten (Gesamt-)Product Owner ab.“

D.10 ★★ Fähigkeitstufen eines Entwicklungsteams (Cockburn)

Ebene	Charakteristik
3	Fähig, eine Methode ANZUPASSEN , d. h. auch deren Regeln zu BRECHEN , um mit unvorhergesehenen Situationen fertig zu werden
2	Fähig, eine Methode bei VORHERSEHBAREN Situationen anzupassen
1A	Nach Ausbildung fähig, größere Entwicklungsaktivitäten richtig abzuwickeln. Kann mit viel Erfahrung auf Ebene 2 gelangen
1B	Nach Ausbildung in der Lage, kleinere Teilschritte in Entwicklungsprozessen korrekt auszuführen. Kann mit Erfahrung auf 1A gelangen
-1	Hat vielleicht technische Fähigkeiten, ist aber unfähig oder unwillig, im Team mit gemeinsam vereinbarten Methoden zu arbeiten

D.11 ★★★ DIE 5 DIMENSIONEN DER METHODENAUSWAHL (Klausurfrage 2×!)

„Nennen und erläutern Sie drei Dimensionen für die Auswahl, ob man klassisch oder agil entwickeln sollte.“ (6 Punkte, Klausur 2024/12)

Der Dozent zeigt ein **Radar-/Spinnendiagramm mit 5 Achsen** (nach **Boehm & Turner: Balancing Agility and Discipline**):

#	Dimension	Skala (agil ↔ klassisch)	Agil ist besser, wenn ...	Klassisch ist besser, wenn ...
1	Personal (Skill-Level)	% Ebene 1B ↔ % Ebene 2 und 3 (40/30/20/10/0 bzw. 15/20/25/30/35)	viele Mitarbeiter auf Ebene 2 und 3 (können Methoden anpassen)	viele Mitarbeiter auf Ebene 1B (brauchen klare Vorgaben)
2	Kritikalität (Verlust bei Fehlern)	Komfort → Geld → viel Geld → Leben einzelner → Leben vieler	nur Komfort oder wenig Geld auf dem Spiel	Menschenleben auf dem Spiel
3	Dynamik (% Änderungen der Anforderungen)	1 / 5 / 10 / 30 / 50 %	hohe Änderungsrate (30–50 %)	stabile Anforderungen (1–5 %)
4	Teamgröße (Anzahl Mitarbeiter)	3 / 10 / 30 / 100 / 300	kleines Team (3–10)	großes Team (100–300)
5	Unternehmenskultur (% Belohnung von Initiative vs. Belohnung von Gehorsam)	10 / 30 / 50 / 70 / 90 %	Initiative wird belohnt (70–90 %)	Gehorsam wird belohnt (10–30 %)

!★ Die Single-Choice-Falle (Klausur 2024)

„Was ist **KEINE** Dimension der Methodenauswahl klassisch versus agil?“ **A. Innovationsgrad des Projekts** ✓ ← **das ist der Fake!** **B. Anzahl der Teammitglieder (= Teamgröße ✓)** **C. Professionalität der Teammitglieder (= Personal ✓)**

📖 „Nennen Sie drei der FÜNF Voraussetzungen für den Einsatz agiler Methoden" (Klausur 2022)

📖 Diese Liste stammt aus dem nicht-durchsuchbaren Balzert-Scan (Kap. 23). Die inhaltlich passende und mit den Folien konsistente Antwort sind die **fünf Dimensionen der Methodenauswahl** in ihrer „agilen" Ausprägung:

1. **Qualifiziertes Personal** — überwiegend Mitarbeiter der Ebenen 2 und 3, die Methoden situativ anpassen können
2. **Geringe Kritikalität** — bei Fehlern droht kein Verlust von Menschenleben
3. **Hohe Dynamik** — die Anforderungen ändern sich häufig (agil lohnt sich erst dann)
4. **Kleines Team** — wenige Mitarbeiter, idealerweise an einem Ort
5. **Passende Unternehmenskultur** — Initiative wird belohnt, nicht Gehorsam

(Ergänzend nennt Balzert häufig auch: **ständige Verfügbarkeit des Kunden** und **Bereitschaft zu Selbstorganisation**.)

📖★ „Nennen Sie VIER Bedingungen, unter denen die agile Softwareentwicklung UNGEEIGNET ist" (Klausur 2020)

Die **Umkehrung** der fünf Dimensionen:

1. **Hohe Kritikalität** (Menschenleben oder sehr viel Geld auf dem Spiel — z. B. Medizintechnik, Flugsteuerung)
2. **Große Teams** (100–300 Mitarbeiter, verteilte Standorte)
3. **Stabile Anforderungen** (nur 1–5 % Änderungen — dann ist Planbarkeit der größere Vorteil)
4. **Gehorsams-/Vorgabekultur** im Unternehmen (Initiative wird nicht belohnt)
5. (zusätzlich) **Überwiegend gering qualifiziertes Personal (Ebene 1B)**
6. (zusätzlich) **Regulatorische/vertragliche Pflicht zu vollständiger Dokumentation** (z. B. behördliche Abnahme, Festpreisverträge)

★ Fazit 3: Hype, Mode, Zukunft?

- **Hype** sicherlich bei Methoden wie **XP**
- **Mode** sicherlich, weil vielfach **alter Wein in neuen Schläuchen** (Crystal, ASD)
- **Zukunft** sicherlich, weil **Rückbesinnung auf wichtige Tugenden** (Crystal, ASD)

D.12 ★★★★★ AGILE BEGRIFFE: Epic, User Story, INVEST, DoD, DoR

Epic

Definition	Eine große, übergeordnete Anforderung, von der User Stories abgeleitet werden
Anwendung	Es werden nur inhaltliche Aspekte beschrieben. Aus Epics werden in nachfolgenden Verfeinerungen untergeordnete Epics, User Stories und/oder Use Cases
Mitwirkende	Programm-/Portfoliomanager, Produktmanager, Product Owner, Systemarchitekt
Eigenschaften	Umsetzung innerhalb eines Sprints NICHT möglich (zu groß dafür) · ausschließlich inhaltliche Anforderungssicht · nur grobe Kostenschätzung möglich

Beispiele des Dozenten:

EP #1: Der Geldautomat erlaubt es einem Kunden **24×7 seine WERTPAPIERgeschäfte** mit unserer Bank abzuwickeln. **EP #2:** Der Geldautomat erlaubt es einem Kunden **24×7 seine GELDTRANSAKTIONEN** mit unserer Bank abzuwickeln.

★★★★★ User Story

Definition	Eine Funktionalität, die entweder für einen User oder einen Käufer eines Systems oder einer Software von WERT ist
Anwendung	Repräsentiert eine funktionale Anforderung mit einem Detaillierungsgrad, der ausreicht, um als Basis für die Planung der nächsten Iteration zu dienen
Mitwirkende	Produktmanager, Product Owner, Mitarbeiter des Fachbereichs, Key User, Entwicklerteam, evtl. Systemarchitekt
Eigenschaften	Kurze funktionale Anforderungsbeschreibung aus BENUTZERSICHT, in Alltagssprache · Entwicklungsaufwand: 1–5 Personentage (= umsetzbar in einer Iteration/Sprint)

★★★★★ DIE USER-STORY-SCHABLONE (Klausurfrage 2024/12, 3 Punkte — auswendig!)

Als <i>Benutzerrolle/Systemrolle</i> will ich <i>Aktion/Story</i> sodass/weil <i>Grund für das Ziel/Nutzen</i>
--

Die drei Bestandteile erläutert:

1. **Als <Benutzerrolle>** — **WER** will etwas? Die **Rolle**, nicht die konkrete Person
2. **will ich <Aktion>** — **WAS** soll das System ermöglichen? Die eigentliche Funktionalität
3. **sodass/weil <Nutzen>** — **WARUM/WOZU?** Der **Geschäftswert** — dieser Teil verhindert Feature-Wildwuchs und ermöglicht Priorisierung

Beispiele des Dozenten (zu EP #1):

US #1: Als Bankkunde will ich jederzeit den Geldautomaten nutzen können, sodass ich von den Schalterzeiten unabhängig **Aktien kaufen** kann. **US #2:** Als Bankkunde will ich jederzeit den Geldautomaten nutzen können, sodass ich von den Schalterzeiten unabhängig **Aktien verkaufen** kann.

★★★★ INVEST — die Qualitätsregeln für User Stories (Klausurfrage 2×, 5–6 Punkte!)

Buchstabe	Begriff	Erläuterung (wörtlich vom Dozenten)
I	Independen t	Die US sind unabhängig voneinander . Jede US kann als eigenständiges Paket geschätzt, geplant und umgesetzt werden.
N	Negot iable	Die US sind verhandelbar (und verhandelt). Sie stellen aber keinen fixen Vertrag dar. Stellt sich im Verlauf der Umsetzung heraus, dass ein Detail anders einfacher oder benutzerfreundlicher umgesetzt werden kann, dann wird dies mit dem PO abgestimmt .
V	Valua ble	Jede US muss einen erkennbaren WERT für den Kunden liefern.
E	Estim atable	Der Aufwand für die US ist zumindest relativ abschätzbar (und geschätzt).
S	Small	Jede US muss innerhalb einer einzigen Iteration umsetzbar sein. Sie sollte daher, je nach Sprint-Dauer, eine Größe von maximal fünf Umsetzungstagen haben.
T	Testa ble	Jede US muss testbar sein.

★★★★ Definition of Done (DoD) vs. Definition of Ready (DoR) — (Klausurfrage 2×!)

	Definition of Done (DoD)	Definition of Ready (DoR)
Was ist es?	Eine Checkliste mit ZWEI SICHTEN	Eine Liste mit Kriterien
Wofür?	Wann ist ein umgesetztes Backlog Item (= US) FERTIG ?	Wann darf ein Product-Backlog-Element (meist eine US) in das SPRINT BACKLOG aufgenommen werden?
Zeitpunkt	Ende des Sprints (Sprint Review)	Sprint Planning (Eingangstor)
Die zwei Sichten	① Produktsicht : alle Eigenschaften , die ein umgesetztes Backlog Item erfüllen muss · ② Prozesssicht : alle Tätigkeiten , die dabei erledigt sein müssen	—

🔑 Merkbild: DoR = das Tor **IN** den Sprint · DoD = das Tor **AUS** dem Sprint.

★★★★ Die SMART-Regel für DoD-Kriterien

Die Kriterien für ein DoD sollten der SMART-Regel entsprechen:

Buchstabe	Begriff	Bedeutung
S	Spezifisch	eindeutig und präzise
M	Messbar	messbar und testbar
A	Akzeptiert	angemessen und von allen Beteiligten akzeptiert
R	Realistisch	Einhaltung und Erreichung muss möglich sein
T	Terminiert	bis wann, wie getestet und wie abgenommen

★ Beispiel-Kriterien einer DoR (12 Fragen des Dozenten)

1. Ist die Anforderung **vollständig beschrieben**?
2. Ist die User Story **verständlich und eindeutig formuliert**?
3. Ist die US **bewertet** (Priorität, Business Value, Risiko)?
4. Sind die relevanten **Akzeptanzkriterien** definiert?
5. Sind die **Risiken** für die US identifiziert und bewertet?
6. Wird die US **vom Tester als testbar eingestuft**?
7. Sind die **Testfälle** erstellt?
8. Ist geklärt, **wer die US abnimmt**?
9. Ist das **Entwicklerteam bereit** für die Umsetzung?
10. Erfüllt die US die **INVEST-Kriterien**?
11. Sind alle **Abhängigkeiten** der US zu anderen Anforderungen klar?

D.13 ★★ Qualitätskriterien für Anforderungen nach IEEE 830-1998 und IREB

Kriterium	Erläuterung
Verständlich und eindeutig	⚠ Keine Mehrdeutigkeiten durch Universalquantoren („jeder“, „nichts“, „immer“), passive Formulierungen , konjunktive und relativierende Formulierungen („meistens“, „manchmal“, „sollte“, „dürfte“)
Vollständig und angemessen	Alle relevanten Stakeholder soweit wie möglich beteiligen; Ausgleich zwischen den unterschiedlichen Vorstellungen finden
Abgestimmt	Abstimmung mit allen relevanten Stakeholdern (Review-Meetings, Sprint-Planning-Meeting)
Gültig und aktuell	Anforderungen können sich laufend verändern — deshalb auf diese Kriterien prüfen
Realisierbar	Nicht realisierbare Anforderungen werden begründet abgelehnt ; ggf. Machbarkeitsanalyse (z. B. experimenteller Prototyp)
Bewertet	Eine Anforderung ist dann korrekt , wenn sie die tatsächlichen Bedürfnisse der Stakeholder widerspiegelt ⇒ ESSENZ!
Konsistent	widerspruchsfrei in sich sowie über alle Anforderungen hinweg
Prüfbar	Jede Anforderung muss am fertigen Release prüfbar sein. Kriterien werden vorab festgelegt durch Akzeptanz-/Abnahmekriterien, DoD und Testfälle . 🔑 „Solange man für eine Anforderung nicht genau weiß, wie man sie später abnehmen und testen wird, ist sie NICHT REIF für die Umsetzung!“
Verfolgbar	Rückverfolgbarkeit der Beziehungen zwischen Anforderungen, User Stories, Use Cases, Epics etc. (<i>Traceability</i>)

D.14 ★ Das Agile-Fluency-Modell (Shore/Larsen)

Die 4 Zonen

Zone	Worum geht es?	Was verändert sich?
(Vor-agil)	Ausgangspunkt	—
① Focus ing	Agile Grundlagen: Konzentration auf Ergebnisse für das Geschäftsfeld, als Team zusammenarbeiten, den Auftrag in Besitz nehmen (Selbstorganisation) . Teams bringen immer zuerst das wertvollste Feature heraus und können die Richtung ändern . 🗝️ <i>Sie fokussieren sich auf die wertvollsten Anforderungen</i>	Veränderung hin zur Teamkultur
② Deliv ering	Technische Exzellenz: Quellcode jederzeit leicht änderbar ; hohe Codequalität (keine Zeit mit Fehlersuche); Verbesserung des Produktionszyklus (hohe Prozessqualität); zuverlässig geringe Fehlerzahl . Erfordert Investition in technische Fähigkeiten UND strukturelle Veränderungen (Test- und Betriebsfähigkeit ins Team integrieren). 🗝️ DevOps als Idee	Veränderung hin zu Teamfähigkeiten
③ Opti mizin g	Optimierung der agilen Prozesse: Anpassung der Organisationsstruktur ; Produkt- und Marktexperten dauerhaft in die Teams ; volle Verantwortung der Teams für Produktbudget und Produktplan . 🗝️ <i>Ein niemals endender Prozess von Experimenten, Lernen und Entdecken</i> . Skalierbare Modelle der Agilität werden hier relevant, z. B. SAFe	Veränderung bei den Organisationsstrukturen
④ Stren gthen ing	Mögliche Zukunft der Agilität: hochgradig spekulativ ; die Gesamtheit der Erfahrungen mit Agilität soll in organisatorische Verbesserungen eingebracht werden. 🗝️ Experimentelle Managementansätze wie Soziokratie	Veränderung in der Organisationskultur

★ Fluency ≠ Reife

Fluency ist eine Eigenschaft, die in einem TEAM entsteht, NICHT bei einem Individuum. Es bedeutet **nicht**, dass jedes Teammitglied jede Fähigkeit besitzt. Stattdessen braucht **das Team als Ganzes die Fähigkeit, immer die richtigen Personen zur richtigen Zeit einzusetzen.**

★ Die 4 Reifestufen je Zone (mit dem Retrospektiven-Beispiel)

Stufe	Definition	🗝️ Beispiel Retrospektive
1. Lernen	Das Team erlernt die Fähigkeiten der Zone	Das Team erlernt die Fähigkeit zur Retrospektive
2. Bewandert	Das Team kann eine Fähigkeit zeigen, wenn es sich darauf konzentriert	Das Team macht immer genau am Ende eines Sprints eine Retrospektive
3. Fluent (routiniert)	Solange das Team von einem COACH unterstützt wird , zeigt es die Fähigkeit automatisch, ohne darüber nachzudenken	Angeleitet vom Coach macht das Team auch Retrospektiven, sobald sich dafür ein Bedarf zeigt
4. Unabhängig fluent	Das Team zeigt die Fähigkeit automatisch, OHNE Coach	Das Team macht selbstständig Retrospektiven flexibel nach Bedarf

💬 **Der kritische Kommentar des Dozenten:** *, „Das ist das Problem mit den Methodengurus: Sie wollen letztlich (viel) Geld mit ihren Modellen und Methoden verdienen. **Modelle und Methoden sollten prinzipiell Aufgabe der Wissenschaft sein und allen zur Verfügung stehen.**“*

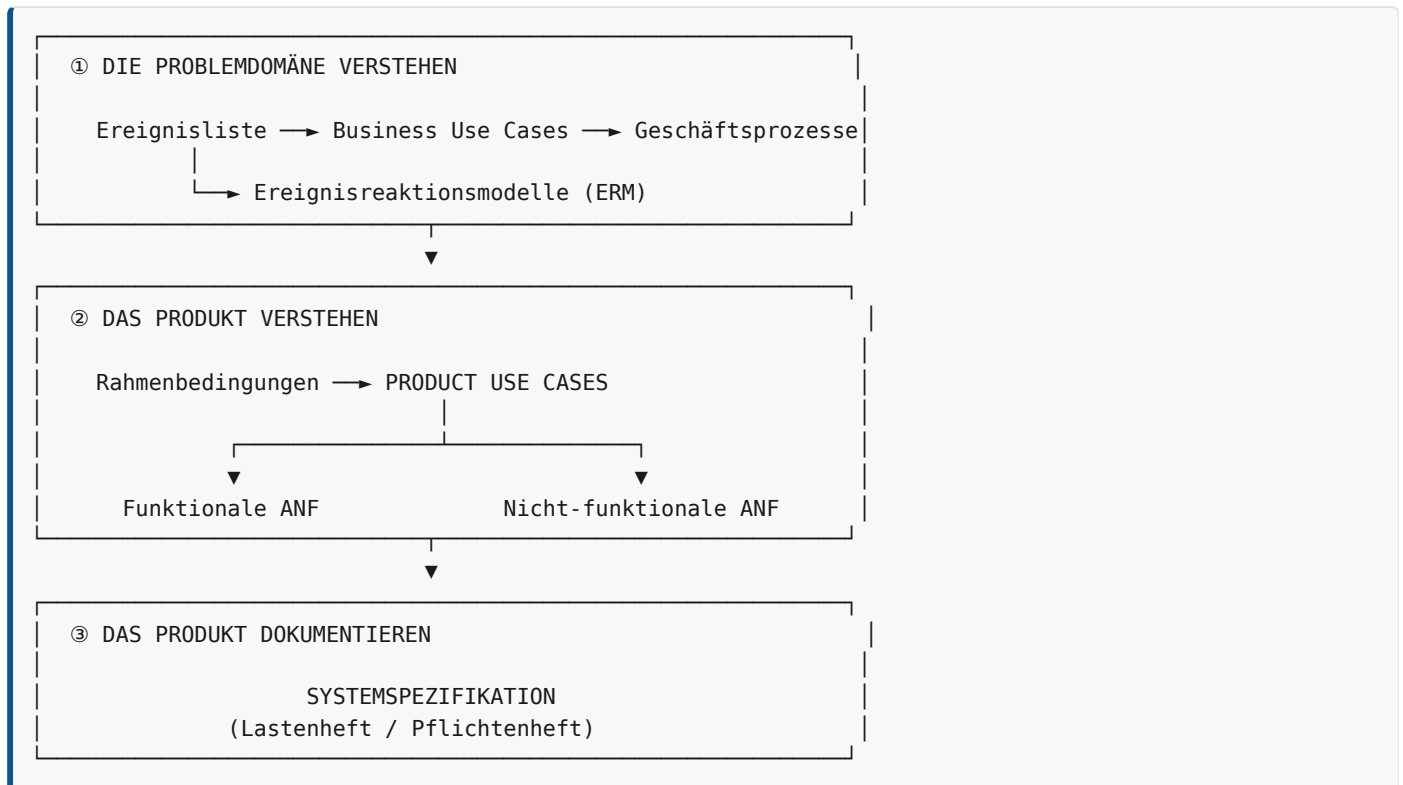
▼ ? Selbsttest Teil D

1. Nennen Sie die 6 Vorwürfe an schwergewichtige SE-Prozesse.
2. Warum wird klassische Softwareentwicklung als schwergewichtig bezeichnet? *(Klausurfrage!)*
3. Rechnen Sie die Space-Shuttle-Analogie nach: Was kostet ein 150-KLOC-System und wie lange dauert es bei 1000 MA?
4. Nennen Sie die drei Merksätze aus Fazit 1.
5. Wie lautet die Effektivitäts-/Effizienz-Formel für leicht- und schwergewichtige Prozesse?
6. Nennen Sie das agile Manifest — alle 4 Punkte, gerne auf Englisch. *(Klausurfrage, 4–8 Punkte!)*
7. Warum ist „Team skills over financial incentives“ kein Punkt des Manifests?
8. Nennen Sie 6 der 12 agilen Prinzipien.
9. Was bedeutet „Crystal“? Wie ist die Crystal-Matrix aufgebaut (L/E/D/C und 6/20/40/80)?
10. Nennen und erläutern Sie die Komponenten von Scrum. *(10 Punkte!)*
11. Was ist KEINE Aufgabe des Scrum Masters? Nennen Sie die Merkregel.
12. Was tut ein Area Product Owner? *(Klausurfrage, 2 Punkte)*
13. Nennen und erläutern Sie 3 Dimensionen der Methodenauswahl klassisch vs. agil. *(6 Punkte!)*
14. Nennen Sie 4 Bedingungen, unter denen agile Entwicklung ungeeignet ist.
15. Was ist ein Epic? Warum passt es nicht in einen Sprint?
16. Nennen und erläutern Sie die User-Story-Schablone. *(3 Punkte!)*
17. Erläutern Sie INVEST vollständig. *(5–6 Punkte!)*
18. Erläutern Sie DoD und DoR. *(4 Punkte!)* Welche Regel gilt für DoD-Kriterien?
19. Erläutern Sie SMART.
20. Nennen Sie 5 Qualitätskriterien für Anforderungen nach IEEE 830/IREB.
21. Nennen Sie die 4 Zonen des Agile-Fluency-Modells und was sich in jeder verändert.
22. Was bedeutet „fluent“ im Unterschied zu „bewandert“?

TEIL E — LE 5: ESSENZIELLE MODELLIERUNG · EREIGNISSE · ERM

E.1 ★★ ★ DAS RE-VORGEHENSMODELL (der rote Faden der gesamten Vorlesung!)

Dieses Diagramm steht auf **jedem** Foliensatz ab LE 5 — es zeigt, wo man gerade ist:



★ Der 4-Schritt der Systemmodellierung (Kopfzeile jedes Foliensatzes)

System
ABGRENZEN → IDENTIFIZIEREN → SPEZIFIZIEREN → FORMAL MODELLIEREN

E.2 ★★★★★ Der Zweck von Informationssystemen (Grundsatzaussage!)

„Systeme existieren, um **DIENTSTLEISTUNGEN FÜR DRITTE** anzubieten. Diese Dienste sind die **REAKTION** auf ein **EREIGNIS**, das **AUSSERHALB** des Systems aufgetreten ist.“

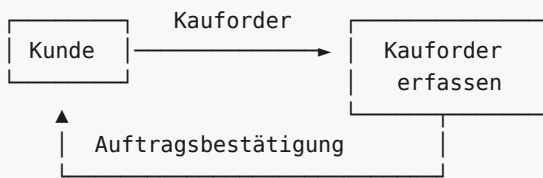
Beispiel für ein Ereignis: *Ein Kunde will 100 Apple-Aktien kaufen.*

NB: Ein **Kunde** ist jemand, der **bereits mit der Bank in Geschäftsbeziehung steht**.

Das ist das Paradigma der EREIGNISORIENTIERTEN ZERLEGUNG: Man zerlegt das System **nicht** nach Funktionen oder Daten, sondern **nach den Ereignissen, auf die es reagieren muss**.

E.3 ★★★★★ Ereignisse und Datenflüsse — die Kernregel

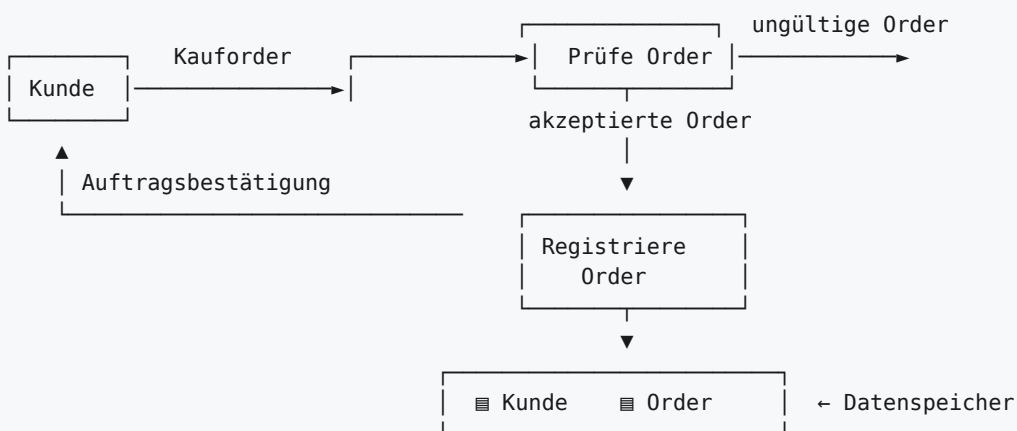
MERKE: Jedes Ereignis produziert einen **EINDEUTIGEN** Datenfluss. Durch den **INHALT** des Datenflusses identifiziert das System, **WELCHES** Ereignis eingetreten ist.



MERKE: Eine REAKTION ist die Zusammenfassung ALLER AKTIVITÄTEN, die auf ein Ereignis reagieren.

E.4 ★★★★★ Das Ereignisreaktionsmodell (ERM)

Das vollständige ERM für „Kunde gibt Kauforder“

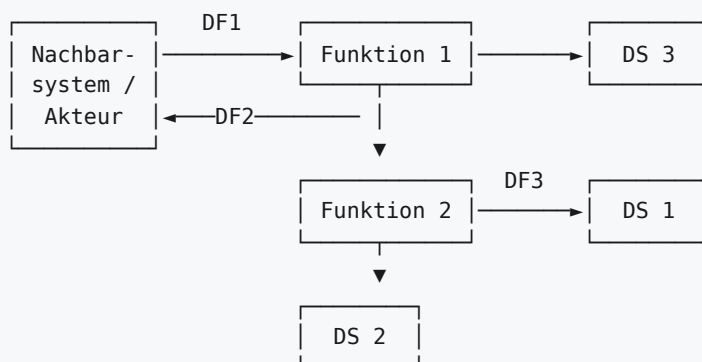


(Der hier relevante Systemausschnitt ist eine **Verfeinerung** von „Kauforder erfassen“.)

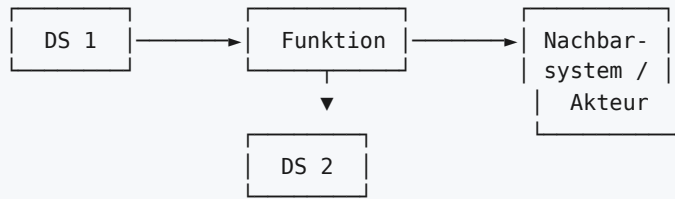
★★★★★ DIE KERNREGEL DES ERM (auswendig!)

MERKE: In einem Ereignisreaktionsmodell (ERM) wird die REAKTION des Systems IMMER von NACHBARSYSTEMEN und DATENSPEICHERN EINGEGRENZT!

MUSTER FÜR EIN EXTERNES EREIGNIS



MUSTER FÜR EIN ZEITLICHES EREIGNIS



Der Unterschied: Ein **externes** Ereignis wird von einem **Nachbarsystem AUSGELÖST** (DF kommt von links rein). Ein **zeitliches** Ereignis hat **keinen auslösenden Datenfluss** von außen — es startet direkt bei den **Datenspeichern** und **endet** in einem Nachbarsystem.

E.5 ★★ ★ DIE EREIGNISARTEN (Klausurfrage 4×!)

„Nennen Sie die Arten von Ereignissen, die im Requirements Engineering relevant sind.“ (2 Punkte)

Art	Definition	Namensregel
① EXTERNE Ereignisse	Werden von einem Nachbarsystem/Akteur außerhalb des Systems ausgelöst; erzeugen einen eingehenden Datenfluss	Name des Nachbarsystems + Grund für den Datenfluss
② ZEITLICHE Ereignisse	Werden durch das Eintreten eines vorgegebenen ZEITPUNKTES ausgelöst. Das Ereignis endet, wenn es einen oder mehrere Datenflüsse nach AUSSEN abgegeben hat	„Zeit für“ + auszuführende Tätigkeit

★★ ★ DIE SINGLE-CHOICE-FALLEN (3× in Klausuren!)

Klausur	Frage	Richtige Antwort
2020	„Was ist KEINE Ereignisart in der Systemanalyse?“ A. Personelle Ereignisse · B. Zeitliche Ereignisse · C. Externe Ereignisse	A. Personelle Ereignisse ✓
2021, 2022	„Was ist KEINE Ereignisart, die in der Systemanalyse betrachtet wird?“ A. Echte Ereignisse · B. Externe Ereignisse	A. Echte Ereignisse ✓

Merken: Es gibt **GENAU ZWEI** Ereignisarten — **EXTERNE** und **ZEITLICHE**. Alles andere ist erfunden!

Beispiele für Ereignisnamen

Art	Beispiel
Extern	„ Kunde gibt Kauforder “ (Nachbarsystem <i>Kunde</i> + Grund <i>gibt Kauforder</i>)
Zeitlich	„ Zeit für Börsenschluss “ (soll heißen: Bei Ende Börsenhandel noch offene Kaufordern löschen)
Zeitlich	„ Zeit für das Löschen abgelaufener Order “

Das zeitliche Ereignis „Zeit für Börsenschluss“:

Die Kauforder konnte nicht am gleichen Börsentag platziert werden. **Zeitliches Ereignis:** Ende des Börsentages. **Reaktion:** Order löschen, Kunden informieren.

```

■ Order → [ Lösche Order ] —gelöschte Order→ △ Kunde
■ Kunde →

```

⚠ **Wichtig:** „Ereignisse werden **im Kontext des Systems** gesucht.“

E.6 ★★ Die Ereignisliste

Die Ereignisliste dient der Sammlung **ALLER RELEVANTEN EREIGNISSE**, für die das System einen **DIENTST** ausführen können muss.

Ereignisname	Zugehörige Datenflüsse
Kunde gibt Kauforder	Kauforder (I) Auftragsbestätigung (O) Ungültige Order (O)
Zeit für Börsenschluss	Gelöschte Order (O)
...	...

I = Input des Systems (eingehender Datenfluss) · **O = Output** des Systems (ausgehender Datenfluss)

★★ Zusammenhang Ereignisliste ↔ ERM

MERKE: Zu jedem **NICHT-TRIVIALEN** Ereignis aus der Ereignisreaktionsliste wird ein **EIGENES** Ereignisreaktionsmodell erstellt!

⚠ Die Single-Choice-Falle (Klausur 2020)

„Ereignisreaktionsmodelle modelliert man...“ A. immer · B. nur bei Bedarf → B. **NUR BEI BEDARF** ✓
(nämlich nur für **nicht-triviale** Ereignisse!)

⚠ ★ Die Single-Choice-Falle zum auslösenden Datenfluss (2× in Klausuren!)

„Welche Aussage zum auslösenden Datenfluss einer Ereignisreaktion ist **FALSCH**?“ A. „Das Nachbarsystem, das ein Ereignis auslöst, muss auch immer dasjenige sein, das die Reaktion erhält“ ✓ ← **das ist die falsche Aussage!** B. „Jedes auslösende externe Ereignis erzeugt einen eindeutigen Datenfluss in das System“ (richtig!) C. „Datenspeicher sind immer systemintern und stellen keine Nachbarsysteme dar“ (richtig!)

🔑 **Begründung zu A:** Ein Ereignis kann von Nachbarsystem X ausgelöst werden und die Reaktion an Nachbarsystem Y gehen (z. B. Kunde bestellt → Rechnung geht an die Buchhaltung). **Auslöser ≠ zwingend Empfänger.**

⚠ Die Single-Choice-Falle zum Nachbarsystem (Klausur 2020)

„Welche Aussage ist richtig?“ A. Ein Nachbarsystem ist immer auch ein Akteur B. Ein Akteur ist immer auch ein Nachbarsystem **C. Akteur und Nachbarsystem können identisch sein** ✓

🔑 **Begründung:** Nachbarsystem ist ein Begriff aus der **ERM-/Datenfluss-Welt** (Problemdomäne), Akteur ein Begriff aus der **Use-Case-Welt** (Produktsicht). Sie **überlappen sich häufig, sind aber nicht deckungsgleich** — z. B. ist ein Datenspeicher weder Akteur noch Nachbarsystem, und ein Nachbarsystem, das nur Daten empfängt, ist nicht zwingend Akteur eines PUC.

E.7 ★★★★★ ESSENZ — der zentralste Begriff der Vorlesung (5×/5 Klausuren!)

„Denken Sie daran — **Essenz, Essenz, Essenz!!!**“ (der Dozent wiederholt diese Folie zweimal im selben Foliensatz)

★★★★★ Die Definition

Wir sind immer noch in der **PROBLEMDOMÄNE** und wollen das **WAHRE PROBLEM** verstehen (nicht die IT-Probleme). Die **ESSENTIELLE SICHT** zeigt das System als eine Menge von **IMPLEMENTIERUNGSUNABHÄNGIGEN** Anforderungen.

📝 Das Beispiel des Dozenten:

Im Ereignis *Kunde gibt Kauforder* wird die Order **zuerst geprüft und anschließend registriert**.

- **Funktionen:** *prüfe Order, registriere Order*
- **Daten:** *Kundendaten, Orderdaten* — die Daten werden anschließend von **anderen Ereignissen** benötigt (Bsp.: *Zeit für Börsenschluss*)
- ★ „WIE das Ereignis **TECHNISCH** funktioniert, ist uninteressant (**NICHT ESSENTIELL**).“

📝 ★★★★★ DIE KLAUSURAUFGABE (2× wörtlich identisch, 4 Punkte!)

Frage: *„Ihr Auftraggeber formuliert folgende Anforderung im Gespräch mit Ihnen: **Die Kontaktaufnahme mit dem Kunden durch unseren First-Level-Support findet über Microsoft Teams statt**‘. Ist diese Anforderung essentiell? Begründen Sie Ihre Antwort.“*

✓ MUSTERANTWORT

NEIN, diese Anforderung ist NICHT essentiell.

Begründung:

- **Essentiell** ist eine Anforderung nur dann, wenn sie **implementierungsUNabhängig** ist — sie beschreibt **WAS** und **WOFÜR** etwas getan werden muss, nicht **WIE** bzw. **WOMIT**.
- „**Microsoft Teams**“ ist eine **konkrete technische Lösung / ein konkretes Produkt** — also eine **Implementierungsentscheidung**, keine fachliche Notwendigkeit.
- Die **essentielle** Formulierung derselben Anforderung lautet: „**Der First-Level-Support muss mit dem Kunden Kontakt aufnehmen können**“ (ggf. ergänzt um essentielle Qualitätsanforderungen wie „synchron“, „mit Bildübertragung“, „nachvollziehbar dokumentiert“).
- Ob dies über **Teams, Zoom, Telefon, E-Mail oder ein Ticketsystem** geschieht, ist eine **Entscheidung der Lösungsdomäne** und gehört **nicht in die Problem-domäne**.
- ⚠ **Gefahr bei nicht-essentieller Formulierung:** Man **verbaut sich vorzeitig Lösungsalternativen** und macht das System **abhängig von einem Produkt**, das sich ändern kann. Die Anforderung wäre außerdem bei einem Wechsel des Werkzeugs **hinfällig**, obwohl sich das fachliche Bedürfnis nicht geändert hat.

(Falls „**Microsoft Teams**“ tatsächlich zwingend vorgegeben ist — z. B. wegen einer Konzernvorgabe — dann ist es keine funktionale Anforderung, sondern eine **RANDBEDINGUNG (technische Restriktion)** und gehört als solche dokumentiert.)

!★ Die Single-Choice-Falle zur Essenz (2× in Klausuren!)

Stimmt diese Aussage: „Für die Essenzbildung müssen **Ablaufreihenfolgen und Abhängigkeiten in den Tätigkeiten der Problem-domäne** betrachtet werden, die **KEINEN fachlichen Hintergrund** haben.“ A. Die Aussage ist richtig B. Die Aussage ist **FALSCH** ✓

🔑 **Begründung: Genau umgekehrt!** Bei der Essenzbildung werden gerade die Ablaufreihenfolgen und Abhängigkeiten **ENTFERNT, die KEINEN fachlichen Hintergrund haben** (also nur technisch/organisatorisch bedingt sind). Betrachtet und **beibehalten** werden nur die Abhängigkeiten, die **fachlich zwingend** sind.

📝 „Erläutern Sie, was man unter ‚Essenz‘ versteht“ (Klausur 2020, 2 Punkte)

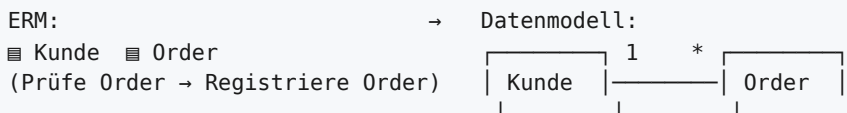
Die Essenz eines Systems ist die Menge aller Anforderungen, die das System erfüllen muss, **unabhängig von jeder technischen und organisatorischen Umsetzung**. Sie beschreibt das **wahre fachliche Problem** der Problem-domäne — also **WAS** das System leisten muss und **WOFÜR** — und lässt alle Aussagen darüber weg, **WIE** (mit welcher Technologie, in welcher Reihenfolge aus rein organisatorischen Gründen, durch welche Person) dies geschieht. Alle rein **implementierungsbedingten Ablaufreihenfolgen und Abhängigkeiten werden entfernt**.

„Erläutern Sie den Zweck eines Ereignisreaktionsmodells" (Klausur 2020, 2 Punkte)

Das ERM **dokumentiert für ein einzelnes (nicht-triviales) Ereignis die vollständige Reaktion des Systems**: welche **Funktionen** ausgeführt werden, welche **Datenflüsse** dabei entstehen und welche **Datenspeicher** gelesen/geschrieben werden. Es dient dazu, die **Systemgrenze zu klären** (Reaktion wird **immer von Nachbarsystemen und Datenspeichern eingegrenzt**), die **essentiellen Funktionen und Daten zu identifizieren** und ist damit die **Grundlage für die späteren Product Use Cases und für das Datenmodell**.

E.8 ★ Exkurs: Vom ERM zum Datenmodell/Klassenmodell

Die im ERM identifizierten **Datenspeicher** werden zu den **Entitäten/Klassen** des Datenmodells, ihre Verknüpfungen zu den **Beziehungen**.



▼ ? Selbsttest Teil E

1. Zeichnen Sie das RE-Vorgehensmodell mit seinen 3 Ebenen aus dem Gedächtnis.
2. Wozu existieren Informationssysteme? (Grundsatzaussage!)
3. Was produziert jedes Ereignis? Wie identifiziert das System das Ereignis?
4. Was ist eine „Reaktion“?
5. Wovon wird die Reaktion im ERM **immer** eingegrenzt?
6. Nennen Sie die 2 Ereignisarten mit ihren Namensregeln.
7. Warum sind „personelle“ und „echte“ Ereignisse keine Ereignisarten?
8. Was steht in einer Ereignisliste? Was bedeuten (I) und (O)?
9. Modelliert man ERM immer oder nur bei Bedarf? Warum?
10. Warum ist die Aussage „Der Auslöser eines Ereignisses muss auch die Reaktion erhalten“ falsch?
11. Können Akteur und Nachbarsystem identisch sein?
12. Erläutern Sie, was man unter Essenz versteht. (Klausurfrage!)
13. Ist „Kontaktaufnahme über Microsoft Teams“ essentiell? Begründen Sie in 5 Sätzen. (4 Punkte!)
14. Was wird bei der Essenzbildung mit Ablaufreihenfolgen ohne fachlichen Hintergrund gemacht?

TEIL F — LE 6: ERHEBUNGSTECHNIKEN

F.1 ★★ Warum Systemanalyse/Anforderungsanalyse? (3 Ziele)

- ① **Gemeinsames Verständnis** aller betroffenen Personengruppen über den **automatisierenden Anteil** im betroffenen Arbeitssystem herstellen

② Wissenserwerb:

- **Wer** wird das System **wofür** nutzen?
- Welchen **Vorteil** soll das System bringen?
- Was sind die **Randbedingungen** der Entwicklung?

③ Konsensbildung:

- Zwischen **Auftraggeber und Auftragnehmer** („Vertrag“, „Angebot“, „Ausschreibung“)
- Zwischen **Auftraggeber und Benutzer**
- Zwischen den **verschiedenen Nutzergruppen**

F.2 ★ Informationssammlung

Identifikation:

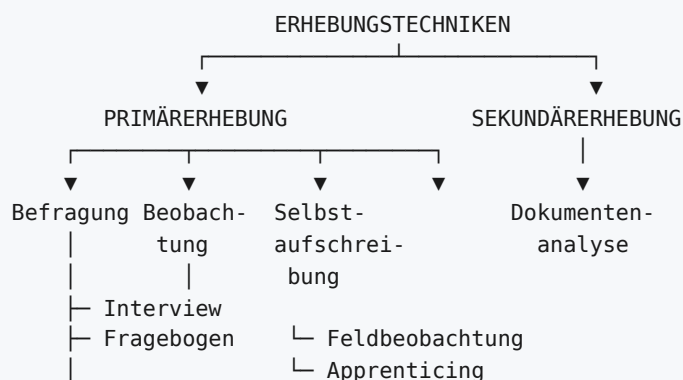
- **Problemabgrenzung** — *Welches Problem soll gelöst werden?*
- **Kontext des Problems** — *Wo ist das Problem?*
- **Ziele der Beteiligten** — *Warum muss es gelöst werden?*
- **Szenariensammlung** — *Wie könnte das neue System helfen?*

Machbarkeit und Risiko — *Was könnte uns daran hindern, das Problem zu lösen?*

Expertenwissen im Anwendungsgebiet:

- Erfassung des **Problemereichs**
- 🔑 **Nutzung der anfänglichen UNWISSENHEIT**, um Fragen zu stellen
- **Sachverstand der Gesprächspartner anerkennen**

F.3 ★★ ★ DIE TAXONOMIE DER ERHEBUNGSTECHNIKEN (auswendig!)



Kategorie	Techniken
PRIMÄRerhebung (Daten werden neu erhoben)	Befragung (Interview, Fragebogen), Beobachtung (Feldbeobachtung, Apprenticing), Selbstaufschreibung
SEKUNDÄRerhebung (auf vorhandene Daten zurückgreifen)	Dokumentenanalyse

F.4 ★ Befragungstypen nach Modus

1. **Schriftlich/Postalisch**
2. **Telefonisch**
3. **Face to Face**
4. **Online**

Schriftliche Befragung

✓ Vorteile	✗ Nachteile
Hoher Grad an Anonymität	Verlust an Spontaneität
Geringe Interviewfehler/Reaktivität	Unkontrollierte Ausfüllsituation (wer füllt aus, mit wem?)
Möglichkeit zu komplexen Fragen mit Diagrammen	Erhöhte Ausfallquote
Überlegte und bewusste Antworten	

Rücklaufquote: zwischen 10 % und 90 %. Was ist wichtig dafür?

- **Thema** der Untersuchung
- **Knapp formulierte, leicht verständliche Fragen**
- **Ansprechendes grafisches Layout**
- **Ankündigungsschreiben — besser telefonische Vorankündigung**
- **Briefaufmachung von einer Werbesendung unterscheiden**
- **Persönlich abgefasstes Anschreiben**
- **Nutzen des Ausfüllers herausstellen**
- **Vertraulichkeit versichern**
- **Angabe eines Rücksendedatums**
- **Frankierter Rückumschlag**
- **Finanzielle Anreize**

★ Fragebogenkonstruktion (8 Regeln)

1. Prüfen, ob es **bereits Fragebögen zum Thema** gibt
2. **Sprachliche Gestaltung immer auf die Zielgruppe** ausrichten
3. **Verständliche, eindeutig anleitende Instruktion**
4. **Letzter Teil des Fragebogens einfach halten**
5. ★ **Platzierungseffekt, Ausstrahlungseffekt beachten** — *nachfolgende Fragen werden von vorhergehenden Fragen beeinflusst*
6. **Lieber mehr Seiten und weniger Fragen pro Seite**
7. **Dauer hängt von der Zielgruppe ab (10 Minuten)** — in einem Unternehmen kann es länger sein (**bis 1 Stunde**)
8. **Grafisch anschauliche Darstellung**

★★ Formulierung von Fragen

- Entscheidung: **offene oder geschlossene** Fragen?
- **Art der Formulierung:**
 - Eine **FRAGE** ist üblicherweise **allgemein formuliert** und hält das Problem **prinzipiell offen**
 - Eine **BEHAUPTUNG** eignet sich zur **Erkundung von Positionen, Meinungen und Einstellungen**
- **Einfach formulieren:** keine Fachausdrücke, Fremdwörter, Abkürzungen, Slang; **kurz halten, konkret formulieren, nicht suggestiv, neutral**
- ★ **Alle Antwortmöglichkeiten in der Frage ausformulieren!**
 - ✗ „Männer sind für die Politik besser geeignet als Frauen.“ Stimme zu / lehne ab
 - ✓ „Würden Sie sagen, dass die meisten Männer für die Politik besser geeignet sind als Frauen, dass Frauen und Männer in dieser Beziehung gleich sind, oder dass Frauen besser für die Politik geeignet sind?“

Telefonische Befragung

- Von einem meist **zentralen Institut** werden **zufällig** Teilnehmer angerufen; der Interviewer **führt aktiv das Gespräch**
- **Fragen mit hoher Ausfallquote können WÄHREND der Befragung gestrichen** werden
- Mittels **kurzer Vorgespräche** können auch **längere Interviews** gemacht werden
- Befragte können durch die Interviewer **motiviert** werden
- ✗ **Komplexe Fragen mit vielen Antwortmöglichkeiten** können nur **mehrstufig** gestellt werden
- ✗ **Keine visuelle Hilfestellung** möglich

Online-Befragung

Zwei Arten: ① **E-Mail-Umfragen** (wenig gebräuchlich, ähnlich der postalischen Befragung) · ② **Umfragen im WWW** (Interaktion möglich, Sammeln von **Metainformationen, Multimedia-Elemente**)

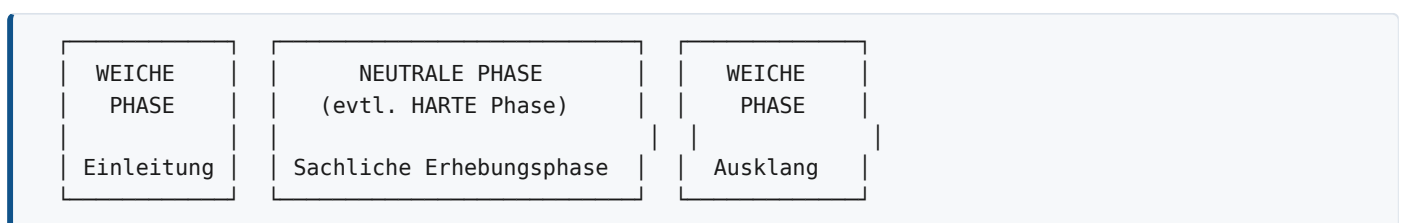
✓	✗
Sehr kostengünstig — geringe Abhängigkeit zur Größe der Stichprobe	Grundgesamtheit der Internetnutzer ist nicht bekannt
Daten liegen elektronisch vor	Kleinere Stichproben sind nicht repräsentativ
Extrem kurze Rücklaufzeiten	Nutzung von Multimediainhalten kann Nutzer ausschließen

F.5 ★★★★★ DIE VIER INTERVIEWFORMEN (große Vergleichstabelle)

Kriterium	Gespräch	Interview i. e. S.	Standardisiertes Interview mit OFFENEN Fragen	Standardisiertes Interview mit GESCHLOSSENEN Fragen
Kurzbeschreibung	lockere Form, weitgehend ungeplant, generelle Fragen vorbereitet	Fragenkatalog vorhanden, aber noch nicht vollständig	Fragen sind festgelegt, hinsichtlich der Antworten ist der Interviewte FREI	Fragen sind festgelegt, Interviewte entscheidet zwischen begrenzter Anzahl von Antworten
Zweck	Ergründung von Absichten, Ziel- und Wertvorstellungen	Detaillierte Erfassung von Aussagen, Überprüfung & Absichern von Aussagen	Absichten und Meinungen einer größeren Personengruppe	Meinungen einer großen Gruppe; Vergleichbarkeit der Antworten ist wichtig
Betroffener Personenkreis	sehr klein	relativ klein	groß	sehr groß
Art der Aussage	allgemein, überwiegend qualitativ	konkreter, noch überwiegend qualitativ	konkret, qualitativ und quantitativ	konkret, qualitativ und quantitativ
Spielraum im Ablauf	sehr groß, Rahmen offen	noch groß, Rahmen nur grob vorgegeben	eingeeengt	klein
Anforderungen an den Interviewer	sehr hoch	hoch	mittel (Fragen interpretieren, situationsbedingt ergänzen)	gering
Vorbereitungsaufwand	gering	mittel (Vorbereitung des Fragenkatalogs)	hoch (Testläufe zweckmäßig)	sehr hoch (Testläufe notwendig)
Auswertung	sehr schwierig (Problemfeld muss strukturiert werden)	schwierig (Problemfeld muss um-/detaillierter strukturiert werden)	mittelschwer (Vergleichbarkeit muss generiert werden)	einfach, maschinelle Auswertung

🔑 **Die Systematik:** Je **standardisierter**, desto **höher der Vorbereitungsaufwand**, aber desto **geringer die Anforderung an den Interviewer** und desto **einfacher die Auswertung** — und desto **größer die erreichbare Personengruppe**.

F.6 ★★★★★ Die 3 Phasen eines Interviews



Phase	Ziele
① Weiche Phase (Einleitung)	Sympathiefeld aufbauen, Kooperationsbereitschaft wecken, Verunsicherung abbauen
② Neutrale Phase (sachliche Erhebung)	7 Schritte: ① Begründung ② Sammlung allgemeiner Infos ③ Probleme/Ziele erläutern ④ Ursachen ermitteln ⑤ Lösungsvorstellungen entwickeln ⑥ Bewertung erstellen ⑦ Ergebnisse zusammenfassen
③ Weiche Phase (Ausklang)	Entspannen, Sympathiefeld pflegen, Verständnis für weiteres Vorgehen fördern

⚠ Über die gesamte neutrale Phase gilt: **KEINE** Stellungnahme, **KEINE** Bewertung des Interviewers!

F.7 ★★ Die 9 Grundregeln für Interviewer (auswendig!)

1. **Keine Suggestivfragen**
2. **Interviewer soll ZUHÖREN können**
3. **Kein Verhör-Stil**
4. **Keine Positionierung des Interviewers** — weder positive noch negative Kritik
 - ⚠ **Ausnahme: provokative Wertung**, um Aussagen herauszufordern, die sonst nicht erhältlich wären
5. **Kurze, offene Fragen stellen** — eine Frage soll i. d. R. **nur EINEN Sachverhalt** betreffen (**keine Unterfragen und Kettenfragen**)
6. **Wahl einfacher, leicht verständlicher Formulierungen**
7. **Ausschweifungen „dezent eindämmen“** (Vielredner auch bei offenen Fragen bremsen)
8. **Vermeidung von Hast und Eile** — sonst Gefahr **unüberlegter Antworten**
9. **Lange Interviews ermüden** — **maximal EINE STUNDE**; wenn nötig **Fortsetzungstermin**

★ Bewertung der Interviewmethode

✓	✗
Erfassung auch QUALITATIVER Einflussgrößen	Erfordernis qualifizierter Interviewer
Abbau negativer Einstellungen während des Gesprächs möglich	Zeitaufwendiges Verfahren
Angaben für die Entwicklung können erfragt werden	Starke Beeinflussung des Betriebsablaufs
Erfassung der Unternehmensstruktur und des Arbeitsablaufs	Angaben zu Mengen und Bearbeitungszeiten sind SUBJEKTIV

F.8 ★★ Die 7 Fragetypen (mit ihren Gefahren)

#	Fragetyp	Definition	Beispiel	Gefahren
1	Sachfragen	Fragen nach Sachverhalten , nicht nach Meinungen oder psychischem Befinden	„Besitzen Sie ein Smartphone?“	Überforderung vermeiden; Recherchefragen („Wie viel Geld geben Sie monatlich für Ihre Hobbys aus?“); Gedächtnisfragen, die zu weit in der Vergangenheit liegen; doppelte Sachfragen („Essen Sie morgens und/oder zum Frühstück Schinken?“)
2	Wissensfragen	Fragen zum individuellen Informationsstand	„Wer ist deutscher Bundeskanzler?“	Frustration durch zu viele/zu schwierige Fragen → Lösung: leichte und schwere Fragen mischen
3	Einstellungs- und Meinungsfragen	Fragen nach persönlichen Meinungen	„Wie beurteilen Sie das Asylgesetz?“ (sozial/unsozial, gerecht/ungerecht)	★ NON-OPINIONS : Viele Befragte machen sich erst im Moment der Fragestellung Gedanken über das Problem (Pseudo-Meinung) · Frageformulierung/Wortwahl haben großen Einfluss · „Weiß-nicht“ als gültige Antwort nicht vergessen! Auch das Nicht-Vorliegen einer Einstellung ist ein gültiger/messbarer Wert
4	Verhaltensfragen	Sollen Auskunft über konkrete Verhaltensweisen geben	„Wie lange haben Sie gestern ferngesehen?“	Frage sollte nicht zu weit in die Vergangenheit reichen → ERINNERUNGSVERZERRUNG , weil dem Befragten eher die EINSTELLUNG zum eigenen Verhalten als das tatsächliche Verhalten bewusst ist
5	Eisbrecherfragen	Zu Beginn : Hemmschwellen abbauen, auflockern, Interesse wecken; muss mit dem Thema nichts zu tun haben , wird oft gar nicht ausgewertet		Thema sollte jeder kennen , sollte zum Reden ermuntern ; ⚠ keine Ja/Nein-Frage stellen , die der Befragte vielleicht auch noch verneint
6	Überleiterfragen	Überleitung zwischen Themenbereichen	„Und jetzt zu etwas anderem – haben Sie sich schon einmal Gedanken über antiautoritäre Erziehung gemacht?“	Dienen der gedanklichen Abgrenzung , dem Öffnen für das neue Thema , der Gedankenpause / gedanklichen Rüstzeit ; meist keine Relevanz für das eigentliche Erkenntnisinteresse
7	Kontrollfragen	Doppelte Fragen mit veränderter Formulierung zur Feststellung von Inkonsistenzen		Gründe für Inkonsistenzen : ① Zwiespalt beim Befragten ② nicht ehrliche Beantwortung ③ Ermüdung (Befragter antwortet zufällig) ④ Antwort nach SOZIALER ERWÜNSCHTHEIT bei kritischen Themen

F.9 ★ Konkrete Beispielfragen des Dozenten

Zum System/Produkt

- **Welches Problem soll das neue System lösen?**
- **Könnte es sein, dass dieses System neue Probleme schafft?**
- **In welchem Umfeld soll das System zum Einsatz kommen?**
- **Gibt es Vorgaben für die geforderte Leistungsfähigkeit?**
- **Sind bestimmte Vorgaben des Gesetzgebers zu beachten?**

Falls schriftlicher Kontakt bestand

- „Ich habe das Material studiert, das Sie mir geschickt haben. Es war nützlich, aber manche Punkte kann man besser im persönlichen Gespräch klären. Können wir einen Termin ausmachen?“
- „Gibt es weitere Mitarbeiter, die mir in dieser Sache helfen könnten?“
- „Können wir uns die Umgebung ansehen, in der die Software später eingesetzt werden soll?“

★ Am Ende des Interviews (die drei Standardfragen!)

- „Gibt es wichtige Punkte, die ich vergessen habe?“
- „Gibt es SONDERFÄLLE, die zwar selten auftreten, aber angesprochen werden müssen?“
- „Darf ich zurückkommen, wenn mir weitere Fragen einfallen?“

F.10 ★★ Beobachtung, Selbstaufschreibung, Dokumentenanalyse

Beobachtung

Erfassung von Fakten durch den Analytiker OHNE DIREKTEN KONTAKT mit dem beobachteten Aufgabenträger/Arbeitsprozess.

✓	✗
Arbeitsablauf wird nicht behindert	Hoher Zeitaufwand
Ermittlung von Arbeitsauslastungen , Fehlerquellen im Arbeitsablauf und Auswirkungen von Umwelteinflüssen	Mengengerüst der Daten wird nicht erfasst

Selbstaufschreibung

Es wird eine ausführliche Beschreibung der in Frage kommenden Arbeitsgebiete verlangt — Ausfüllen von **Erhebungsbögen** (Statistik).

✓	✗
Genaue Beschreibung der Arbeitsgebiete	Individuelle Erfassung (subjektiv, uneinheitlich)
Betroffene sind AKTIV beteiligt und können Lösungsvorschläge und Anregungen unterbreiten	Mengen-/Zeitangaben sind kritisch zu betrachten

Dokumentenanalyse (= Sekundärerhebung!)

Auswertung vorhandener Unterlagen: Organisations- und Aufgabenpläne, **Stellen- und Arbeitsplatzbeschreibungen**, **Arbeitsablaufdiagramme**, **ausgefüllte Vordrucke**, **Geschäftsberichte/Statistiken**, ...

✓	✗
Keine Störung des Betriebsablaufs	⚠ Beschreibung = Realität? (Bsp.: Abläufe/Prozesse)
Gute Erfassung von MENGENGERÜSTEN	Gültigkeit für die Zukunft unsicher
Objektive Aussagen	Kein Abbau negativer Einstellungen der Belegschaft möglich
Kostengünstig	Qualifiziertes Untersuchungsteam erforderlich

★★ Die Auswahlmatrix (Merksystematik)

Ich brauche ...	dann nehme ich ...
qualitative Einflussgrößen, Ziele, Meinungen	Interview
Meinungen einer sehr großen Gruppe, vergleichbar	standardisiertes Interview mit geschlossenen Fragen / Fragebogen
objektive MENGengerüste	Dokumentenanalyse
tatsächliche Arbeitsabläufe, Fehlerquellen, Auslastung	Beobachtung
detaillierte Beschreibung eines Arbeitsgebiets mit Beteiligung der Betroffenen	Selbstaufschreibung
Beeinflussung des Betriebsablaufs vermeiden	Dokumentenanalyse oder Beobachtung

▼ ? Selbsttest Teil F

1. Nennen Sie die 3 Ziele der Systemanalyse.
2. Zeichnen Sie die Taxonomie der Erhebungstechniken.
3. Welche Technik ist die einzige Sekundärerhebung?
4. Nennen Sie 4 Vor- und 3 Nachteile der schriftlichen Befragung.
5. Was sind Platzierungs- und Ausstrahlungseffekt?
6. Nennen Sie die 4 Interviewformen und ordnen Sie sie nach Vorbereitungsaufwand.
7. Welche Interviewform hat die höchsten Anforderungen an den Interviewer? Welche die niedrigsten?
8. Nennen Sie die 3 Phasen eines Interviews mit ihren Zielen.
9. Nennen Sie die 7 Schritte der neutralen Phase.
10. Nennen Sie 6 der 9 Grundregeln für Interviewer.
11. Wie lange darf ein Interview maximal dauern?
12. Nennen Sie die 7 Fragetypen mit je einem Beispiel.
13. Was sind „Non-Opinions“?
14. Was ist Erinnerungsverzerrung?
15. Nennen Sie 4 Gründe für Inkonsistenzen bei Kontrollfragen.
16. Welche Technik nutzen Sie, um Mengengerüste objektiv zu erfassen? Warum?

TEIL G — LE 7: USE CASES ★★★ (DAS HERZSTÜCK DER KLAUSUR!)

In JEDER der 5 Klausuren ist die Modellierung eines Product Use Case die größte Einzelaufgabe (15 von 60 bzw. 15 von 100 Punkten).

G.1 ★★★★★ Grundsätzlich: Was ist ein Use Case?

- Ein Use Case besteht aus mehreren **ZUSAMMENHÄNGENDEN AUFGABEN**, die von einem **AKTEUR** durchgeführt werden, um ein **ZIEL** zu erreichen bzw. ein gewünschtes **ERGEBNIS** zu erstellen.

G.2 ★★★★★ AKTEUR — Definition (Klausurfrage 3×!)

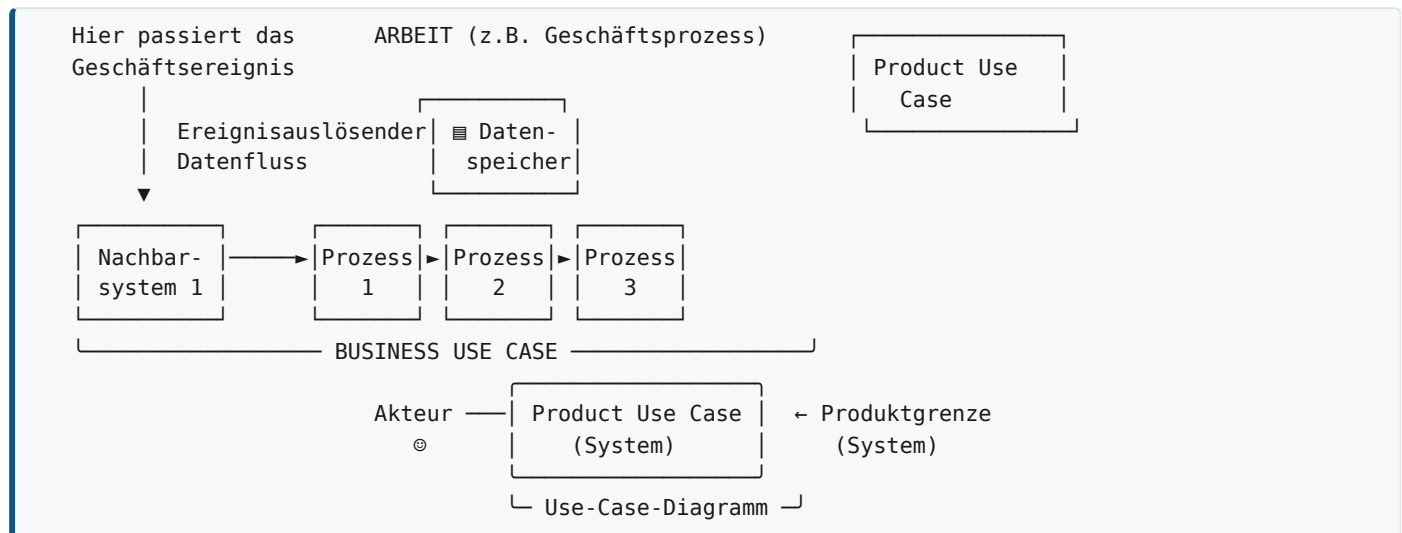
- Ein Akteur ist eine **ROLLE**, die ein Benutzer des Systems einnimmt. Jeder Akteur hat einen gewissen **EINFLUSS** auf das System.
- Ein Akteur ist häufig eine Person. Es kann sich aber ebenso um eine **ORGANISATIONSEINHEIT** oder ein **EXTERNES SYSTEM** handeln, das mit dem zu modellierenden System kommuniziert.
- ★★★★★ **AKTEURE BEFINDEN SICH STETS AUSSERHALB DES SYSTEMS!**

★★★★★ „Nennen und erläutern Sie die ARTEN VON AKTEUREN eines Product Use Cases" (Klausurfrage 3×, 3 Punkte!)

Art	Erläuterung	 Beispiel
① Person / menschlicher Akteur	Eine Rolle , die ein Benutzer einnimmt (nicht die konkrete Person!)	Kundensachbearbeiter, Lagersachbearbeiter, Bankkunde, Mitarbeiter im First-Level-Support
② Organisationseinheit	Eine Abteilung/Stelle des Unternehmens, die als Ganzes mit dem System interagiert	Buchhaltung , Second-Level-Support, Vertrieb
③ Externes System / Nachbarsystem	Ein anderes IT-System , das mit dem zu modellierenden System kommuniziert	Zahlungssystem , E-Mail-Server, Wissensdatenbank , CMS

⚠ ★★★★★ **DIE WICHTIGSTE ABGRENZUNG (Klausur 2024 SC + bäm.pdf „AHA-Moment!"):** „Ist das zu modellierende System ein Akteur bei der Modellierung seiner Product Use Cases?" → **NEIN!** ✅ **Das „System Under Design" ist KEIN Akteur.** Es ist das, was modelliert wird — Akteure stehen **stets außerhalb**. Das System **selbst** taucht in der Beschreibung zwar als handelndes Subjekt auf („Das System validiert ..."), wird aber **nicht in der Zeile ‚Akteure‘ aufgeführt** und **nicht als Strichmännchen** gezeichnet.

G.3 ★★★★★ Business Use Case vs. Product Use Case (Klausurfrage!)



★★★★ „Erläutern Sie das SYSTEMVERSTÄNDNIS in einem Geschäftsprozessmodell und in einem Product Use Case" (Klausur 2021, 4 Punkte!)

	Business Use Case / Geschäftsprozessmodell	Product Use Case
Was ist das „System“?	Das Unternehmen / der Geschäftsprozess als Ganzes — inklusive aller beteiligten Menschen, Abteilungen und IT-Systeme	Nur das zu entwickelnde IT-Produkt — alles andere liegt außerhalb der Produktgrenze
Systemgrenze	Grenze der PROBLEMDOMÄNE	PRODUKTGRENZE (Grenze der Lösungsdomäne)
Auslöser	ein Geschäftsereignis	ein auslösendes Ereignis an der Produktgrenze
Wer sind die Akteure?	die Nachbarsysteme des Geschäftsprozesses (Kunde, Lieferant, Behörde)	die Rollen und Systeme, die auf das Produkt zugreifen — auch Mitarbeiter, die im Business UC innerhalb des Systems lagen!
Ergebnis	Verständnis der PROBLEMDOMÄNE (RE-Vorgehensmodell, Ebene ①)	Verständnis des PRODUKTS (RE-Vorgehensmodell, Ebene ②)
Enthält	mehrere Prozessschritte, davon werden nur einige automatisiert	genau den automatisierten Ausschnitt

🔑 **Der Kernsatz:** Ein **Business Use Case** beschreibt das Geschäft; ein oder mehrere **Product Use Cases** beschreiben den Teil davon, den das zu bauende Produkt übernimmt. **Ein Mitarbeiter, der im Business UC noch Teil des Systems war, wird im Product UC zum AKTEUR** — weil er von außen auf das Produkt zugreift.

📝 **Beispiel Vorlesungsplanung:** Business UC = „Semesterplanung" · Product UC = „VVZ erstellen" · Akteurin = **Frau Estler (Vorlesungsplanerin)** · Produktgrenze = **VVZ-System**

Die Fallstudie „Vorlesungsplanung“ (das Beispiel des Dozenten)

Der Studiengangsleiter gibt die zentralen Planungsdaten (Kurs, Semester, Dozenten) und wichtige Restriktionen (z. B. Sporttag der STUV, Erstsemesterbegrüßung durch den Rektor) an die Vorlesungsplanerin, **Frau Estler**. Dies erfolgt in einem **Kick-Off-Meeting ca. 4 Monate vor Semesterstart**. [...] Dann beginnt Frau Estler mit der **Anfrage der Dozenten**: Sie ruft an, mailt oder spricht Dozenten direkt an. Dazu kann sie auf die Datenbank „**Dozentenpool**“ zugreifen. Nach Kontaktaufnahme fragt sie die **Dozentendaten** ab (Termine, Raum- und Materialbedarf). Liegen alle Dozentendaten vor und gibt es keine Terminkonflikte, wird das **VVZ erstellt**, ausgedruckt und an Dozenten und Unternehmen **per Post verschickt**. Für die Studierenden wird ein **PDF auf der Webseite** eingestellt.

Die daraus abgeleiteten Prozessschritte (Business-Ebene): Planungsdaten → Dozenten anfragen → VVZ erstellen → VVZ verschicken / VVZ veröffentlichen Nachbarsysteme/Datenspeicher: Studiengangsleiter, Dozenten, Studienhandbuch, Dozenten-CMS, Unternehmen

G.4 ★★★★★ DIE USE-CASE-SCHABLONE (AUSWENDIG — 15 Klausurpunkte!)

Bei **einfachen** Use Cases reicht eine **umgangssprachliche Beschreibung** aus. Bei **komplexen** Use Cases wird eine **Use-Case-Schablone (use case template)** verwendet.

1. Use Case:	Name, bestehend aus ZWEI ODER DREI WÖRTERN (was wird getan?)
2. Ziel:	globale Zielsetzung bei erfolgreicher Ausführung des Use Case
3. Vorbedingung:	Erwarteter Zustand VOR Beginn
4. Nachbedingung Erfolg:	Erwarteter Zustand NACH erfolgreicher Ausführung = Ergebnis des Prozesses
5. Nachbedingung Fehlschlag:	Erwarteter Zustand, WENN DAS ZIEL NICHT ERREICHT werden kann
6. Akteure:	Rollen von Personen oder ANDERE SYSTEME, die den UC auslösen oder daran beteiligt sind
7. Auslösendes Ereignis:	Wenn dieses Ereignis eintritt, dann wird der Use Case initiiert
8. Beschreibung:	
1 Erste Aktion	
2 Zweite Aktion	
Erweiterungen:	
1a Erweiterung des Funktionsumfangs der ersten Aktion	
Alternativen:	
1a Alternative Ausführung der ersten Aktion	
1b Weitere Alternative zur ersten Aktion	

!★ Der SC-Klassiker: „Muss es eine Nachbedingung Fehlschlag im Use Case geben?“

→ **JA!** ☒ Die Schablone hat **Punkt 5 „Nachbedingung Fehlschlag“** als **festen Bestandteil**. Auch wenn kein Fehlschlag denkbar ist, muss das Feld **ausgefüllt** werden (dann eben mit einer Begründung) — es einfach mit „**keiner**“ zu füllen, ist in der Regel ein **formaler Fehler**, weil es fast immer einen Fehlschlagzustand gibt (siehe Klausuraufgabe 6 im Teil L!).

📝⭐ Vergleich: umgangssprachlich vs. Schablone

(a) Umgangssprachlich (UC „Kundenauftrag ausführen“):

Eine Kundenbestellung kommt in der Versandabteilung an. Neukunden werden im System registriert und der Versand an diese Kunden erfolgt ausschließlich per Nachnahme oder per Bankeinzug. Für alle lieferbaren Artikel wird die Rechnung erstellt und als Auftrag an das Lager weitergegeben. Sind einige der gewünschten Artikel nicht lieferbar, so wird der Kunde informiert und die Nachlieferung im System vorgemerkt. Alle erstellten Rechnungen werden an die Buchhaltung weitergegeben. An diesem Use Case sind folgende Akteure beteiligt: Kundensachbearbeiter, Lagersachbearbeiter und Buchhaltung.

(b) ⭐ Mit UC-Schablone (dieselbe Sache):

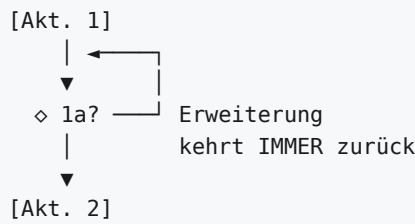
Use Case: #002 Kundenauftrag ausführen
 Ziel: Ware an Kunden geliefert
 Vorbedingung: -
 Nachbedingung Erfolg: Ware ausgeliefert (auch Teillieferungen),
 Rechnungskopie bei Buchhaltung
 Nachbedingung Fehlschlag: Kunde wurde mitgeteilt, dass nichts lieferbar ist
 Akteure: Kundensachbearbeiter, Lagersachbearbeiter, Buchhaltung
 Auslösendes Ereignis: Bestellung des Kunden liegt vor
 Beschreibung:
 1 Die Kundendaten werden auf Aktualität geprüft
 2 Lieferbarkeit prüfen
 3 Rechnung erstellen: UC #020 Rechnung erstellen
 4 Auftrag vom Lager ausführen lassen
 5 Rechnungskopie an Buchhaltung geben
 Erweiterung:
 1a Wenn die gespeicherten Kundendaten nicht aktuell sind:
 UC #004 Kundendaten aktualisieren
 2a Für nachlieferbare Artikel: UC #003 Nachlieferung veranlassen
 Alternativen:
 1a Wenn der Besteller noch nicht Kunde ist: UC #001 Neukunde erfassen
 3a Wenn ein Artikel nicht lieferbar: Kunde mitteilen, dass nicht
 geliefert werden kann. UC beenden.

G.5 ⭐⭐⭐ ERWEITERUNG vs. ALTERNATIVE (Der wichtigste formale Unterschied!)

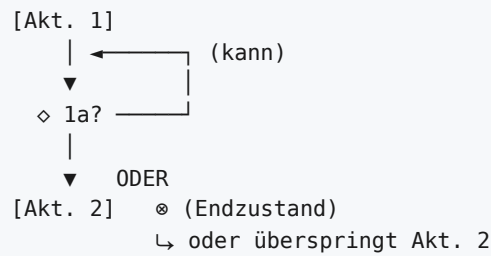
📖 Aus der Studierendenpräsentation „Use Cases Lessons Learned“, Lesson Learned 3:

	ERWEITERUNG	ALTERNATIVE
Definition	Erweitert den Funktionsumfang einer Aktivität	Ersetzt die Ausführung einer Aktivität durch einen anderen Weg
⭐ Rückkehrverhalten	KEHRT VOR DER NÄCHSTEN AKTIVITÄT ZUM HAPPY PASS ZURÜCK	Kann zum Happy Pass zurückkehren, ENDET aber MINDESTENS EINMAL IN EINEM ENZZUSTAND oder UMGEHT Aktivitäten des Happy Pass
Im UC-Diagramm	<<extend>>	<<extend>> (siehe SC-Frage unten!)
Nummerierung	1a , 1b , ... unter „Erweiterungen:“	1a , 1b , ... unter „Alternativen:“

ERWEITERUNG



ALTERNATIVE

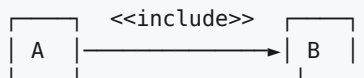


!☆☆ Die Single-Choice-Fallen dazu

Frage	Antwort
„Welche Aussage stimmt bezüglich Alternativen in einem Product Use Case?“ A. Eine Alternative ergänzt eine Aktivität · B. Zu einer Alternative kann es keine Erweiterungen geben · C. Nichts davon	C. Nichts davon ✓ — A ist falsch (das tut die Erweiterung), B ist falsch (zu einer Alternative KANN es sehr wohl Erweiterungen geben)
„Eine Extend-Beziehung in einem Use-Case-Diagramm ist nur möglich für...“ A. den Erweiterungsteil · B. den Alternativenteil · C. den Erweiterungs- UND den Alternativenteil	C ✓ — im Diagramm wird beides über <<extend>> dargestellt!
„Welche Aussage ist FALSCH?“ A. Nach „prüfen“, „validieren“ etc. kommt immer genau eine Alternative · B. „extend“ wird für Erweiterungen und Alternativen eines Use Cases benutzt	A ist falsch ✓ — nach einer Validierung kann es mehrere Alternativen geben (bzw. muss es nicht genau eine sein). B ist richtig.
„Welche Aussage zu den Beziehungsarten eines Product Use Cases ist richtig?“ A. „include“ erweitert einen UC unter bestimmten Bedingungen · B. „extend“ wird für Erweiterungen und Alternativen benutzt · C. Nichts davon	B ✓ — A ist falsch, das ist die Beschreibung von extend

G.6 ☆☆☆ include vs. extend (Die große Vergleichstabelle)

<<include>>



<<extend>>



Kriterium	<<include>>	<<extend>>
Bedeutung	Ablauf von A schließt IMMER Ablauf von B ein	Ablauf von A KANN, MUSS ABER NICHT durch den Ablauf von B erweitert werden
Definition	Liegt vor, wenn zwei Use Cases A und B ein GEMEINSAMES VERHALTEN besitzen , das in Use Case C spezifiziert ist. C wird analog zu einem UNTERPROGRAMM aufgerufen	Liegt vor, wenn ein Use Case B ähnlich einem Use Case A ist, aber „ETWAS MEHR ALS A TUT“ . Ermöglicht, einen komplexen UC zunächst vereinfacht zu spezifizieren und komplexe Sonderfälle in die Erweiterungen auszulagern
Wann genutzt?	Ablauf von B kann in VERSCHIEDENEN UC genutzt werden	A besitzt neben Normalverhalten einen in B ausgelagerten SONDERFALL
Bedeutung für A	A ist meist UNVOLLSTÄNDIG und wird erst durch Inklusion von B zu einem vollständigen Ablauf	A ist meist VOLLSTÄNDIG und kann durch B optional erweitert werden
Abhängigkeiten	A muss B bei der Modellierung berücksichtigen. B wird unabhängig von A modelliert , um die Nutzung durch weitere UC sicherzustellen (Wiederverwendbarkeit). B muss in sich NICHT vollständig sein („B weiß nicht, durch wen er inkludiert wird“)	A muss durch Angabe von ERWEITERUNGSPUNKTEN (Extension Points) auf B vorbereitet werden. B wird in sich VOLLSTÄNDIG und unabhängig von A modelliert („B weiß nicht, wen er erweitert“)
Pfeilrichtung	A → B	B → A ⚠ (der erweiternde UC zeigt auf den erweiterten!)

Die Beispiele des Dozenten

<<include>>: UC #002 Kundenauftrag ausführen —<<include>>→ UC #020 Rechnung erstellen
„UC #002 inkludiert IMMER den UC #020“

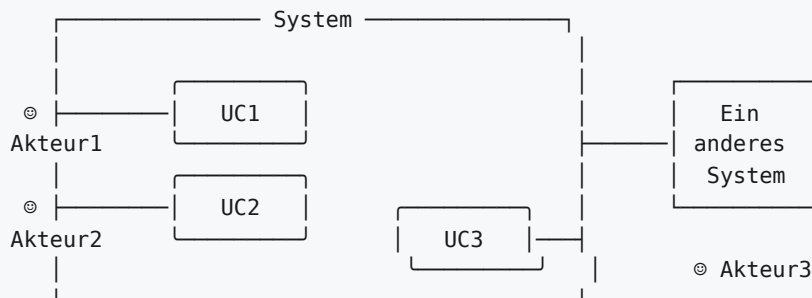
<<extend>>: UC #002 Kundenauftrag ausführen ←<<extend>>— UC #004 Kundendaten aktualisieren
Extension Point: „Kundendaten nicht aktuell“
„UC #002 wird – UNTER BESTIMMTEN BEDINGUNGEN – durch UC #004 erweitert“

 **Der Extension Point** ist die **Bedingung**, unter der die Erweiterung greift. Er wird **im erweiterten UC (A)** notiert.

G.7 ★★ Das Use-Case-Diagramm

Definition und Notationsregeln

- Das **Zusammenspiel mehrerer Product Use Cases untereinander und mit den Akteuren** wird im Use-Case-Diagramm modelliert
- Es gibt auf **hohem Abstraktionsniveau** einen guten **Überblick über das System und seine Schnittstellen zur Umwelt**
- **Die Akteure werden als STRICHMÄNNCHEN eingezeichnet.** Externe Systeme können auch als **Rechtecke** dargestellt werden
- **Use Cases sind IMMER OVALE**
- Eine **LINIE** zwischen Akteuren und Use Cases bedeutet, dass eine **KOMMUNIKATION** stattfindet
- Das System wird als **RECHTECK** gezeichnet
- Das UC-Diagramm fasst die Product Use Cases **VISUELL** zusammen. Es zeigt also, **welche Business Use Cases durch das System realisiert werden sollen**



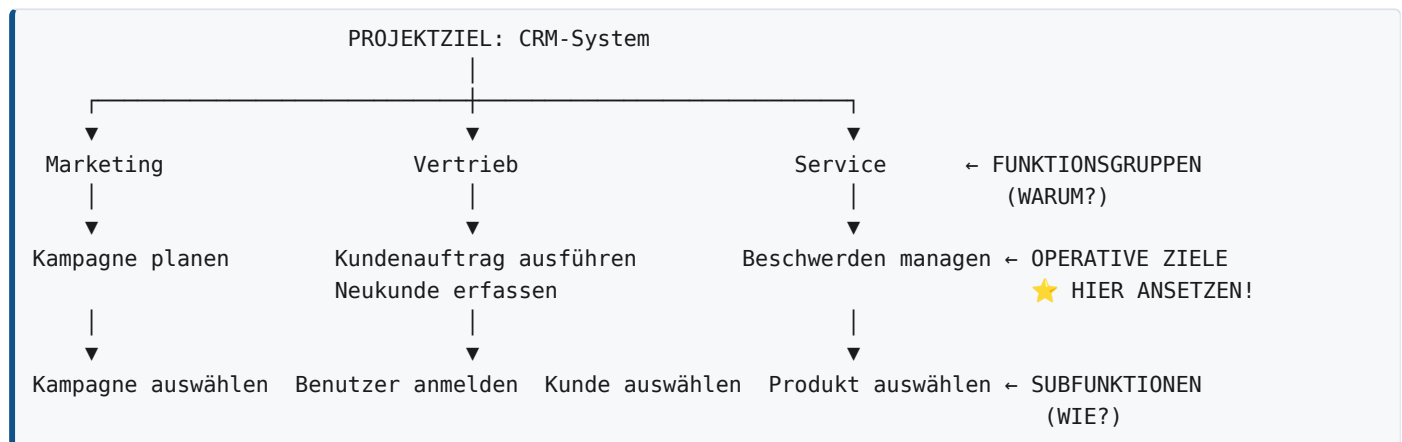
!★★ DIE SINGLE-CHOICE-FALLE (2× wörtlich identisch!)

„Ein Use-Case-Diagramm ist..." A. eine grafische Darstellung **EINES** Product Use Case des zu modellierenden Systems **B. eine grafische ZUSAMMENFASSUNG ALLER Product Use Cases UND AKTEURE** des zu modellierenden Systems ☒ C. nichts davon

🔑 Merken: Das UC-Diagramm zeigt **ALLE UCs + ALLE Akteure** — nicht einen einzelnen UC!

G.8 ★★★★★ WIE DETAILLIERT MUSS EIN USE CASE SEIN? (Der Coffee-Break-Test!)

Die Zielhierarchie



★★★★★ DIE 4 MERKSÄTZE (auswendig!)

1. Ausgangspunkt sind die Use Cases der **OPERATIVEN ZIELE!**
2. Die Abgrenzung der Use Cases ist **NIE EINDEUTIG RICHTIG**, kann aber sehr wohl **EINDEUTIG FALSCH** sein!
3. Ein Use Case für operative Ziele muss den **COFFEE BREAK TEST** aus Sicht des Akteurs bestehen:
„After I get done with this, I can take a coffee break!“
4. Typischerweise dauert diese Aktivität **2-20 MINUTEN**, betrifft **1 PERSON** und wird in **1 SITZUNG** erledigt

Die Beispiele des Dozenten:

- Sich 30 Mal anzumelden ist **KEIN** operatives Ziel. Einen Neukunden erfassen hingegen **SCHON**.
- Eine Kampagne auszuwählen ist **KEIN** operatives Ziel. Eine Kampagne planen hingegen **SCHON**.

!★★★★★ Die Single-Choice-Falle (4× in Klausuren!)

„Was meint der ‚Coffee-Break-Test?‘“ / „Der Coffee-Break-Test ist eine Daumenregel für...“

- A. Den sinnvollen Umfang **einer Aktivität** B. Den sinnvollen Umfang eines **PRODUCT USE CASES** ✓
 C. die angemessene Anzahl von Product Use Cases / Nichts davon

🔑 **Merken: Coffee-Break-Test → Umfang eines ganzen USE CASE (nicht einer Aktivität!)** Für den Umfang **einer Aktivität** gilt stattdessen **Regel R4** (signifikanter Zwischenerfolg, siehe unten).

G.9 ★★★★★ DIE 8 REGELN FÜR GUTE USE CASES (R1-R8 — Klausur-

Gold!)

✗ **So sicher NICHT** (das Negativbeispiel des Dozenten für UC #001 Neukunde erfassen): Reiter "Person" wählen → Felder "Person" füllen → Person speichern

★ R1: Einfache Grammatik

Subjekt	Verb	Objekt	Präpositionalphrase
Der Bankkunde	hebt	den Geldbetrag	von seinem Bankkonto ab.

★★★★ R2: Es muss klar sein, wer etwas macht — „WHO HAS THE BALL?“

✗	✓
„ER hebt den Geldbetrag von seinem Bankkonto ab.“	„Der BANKKUNDE hebt den Geldbetrag von seinem Bankkonto ab.“

!★★★★ Die Single-Choice-Falle (3× in Klausuren!)

„Was bedeutet die Regel ‚Who has the ball?‘“ A. Immer aus Sicht des Akteurs schreiben (→ **das ist R3!**) B. Immer angeben, **WER die Aktivität AUSFÜHRT** ✓ C. Immer angeben, für wen etwas getan wird
 🔑 **Achtung, die Verwechslungsgefahr mit R3 ist genau die Falle!** „Who has the ball“ = **WER handelt** (keine Pronomen, kein Passiv!). „Aus Perspektive des Akteurs schreiben“ ist R3.

★ R3: Aus der PERSPEKTIVE DES AKTEURS schreiben

✗	✓
1. Authentifizierungsdaten lesen. 2. Authentifizierungsdaten prüfen. 3. Zahle Geld von Bankkonto an Kunden aus.	1. Der Kunde authentifiziert sich gegenüber dem System. 2. Der Kunde hebt den Geldbetrag von seinem Bankkonto ab.

★★ R4: Eine Aktivität sollte einen SIGNIFIKANTEN ZWISCHENERFOLG für den Akteur liefern

Daumenregel: MEHR ALS ca. 9–10 AKTIVITÄTEN HAT KEIN GUTER USE CASE! Der Zwischenerfolg kann durch folgende Frage ermittelt werden: „WARUM macht der Akteur das?“

✗	✓
1. Vornamen eingeben. 2. Nachnamen eingeben. 3. Kontonummer eingeben.	1. Kunde gibt Vor- und Nachnamen sowie Kontonummer für seine AUTHENTIFIZIERUNG an.

★★ R5: Die ABSICHT des Akteurs sollte in der Aktivitätsformulierung erkennbar werden

✗ (7 Schritte Ping-Pong)	✓ (2 Schritte mit Absicht)
1. System erfragt Vornamen. 2. Kunde gibt Vornamen an. 3. System erfragt Nachnamen. 4. Kunde gibt Nachnamen an. 5. System erfragt Kontonummer. 6. Kunde gibt Kontonummer an. 7. System prüft Authentifizierung.	1. Der Kunde gibt Vor- und Nachnamen sowie Kontonummer für seine Authentifizierung an. 2. Das System VALIDIERT den Kunden anhand seiner Authentifizierungsdaten.

★★ R6: EREIGNIS-REAKTIONSSCHEMA vollständig beschreiben

Ein vollständiger UC durchläuft immer diese vier Schritte:

1. **Der Akteur SENDET DATEN** als auslösendes Ereignis.
2. **Das System ÜBERPRÜFT** die Daten.
3. **Das System REAGIERT** auf die Daten.
4. **Das System GIBT DAS ERGEBNIS** der Reaktion **AN DEN AKTEUR**.

🔑 Das ist die häufigste Fehlerquelle in Klausuren: Der letzte Schritt (Rückmeldung an den Akteur!) wird vergessen.

★★★ R7: VALIDIEREN statt ÜBERPRÜFEN

WOZU wird etwas gemacht, nicht einfach: **ETWAS** wird gemacht. ⚠ „Prüfen“, „überprüfen“ führt **FAST IMMER** zur Notwendigkeit einer Folgeaktivität mit Anfangsbedingung „Wenn korrekt, dann...“

✗	✓
1. Das System ÜBERPRÜFT die Authentifizierungsdaten. 2. Wenn korrekt authentifiziert , bekommt der Kunde die Serviceauswahl vom System angeboten. <i>Alternative:</i> 2a. Wenn nicht korrekt authentifiziert: Kunden informieren und abbrechen.	1. Das System VALIDIERT den Kunden anhand seiner Authentifizierungsdaten. 2. Der Kunde bekommt die Serviceauswahl vom System angeboten. <i>Alternative:</i> 2a. Wenn nicht korrekt authentifiziert: Den Kunden über fehlgeschlagene Validierung informieren und abbrechen.

🔑 **Der Unterschied:** „Validieren“ enthält das **Ergebnis** bereits — der Happy Pass läuft danach einfach weiter. „Prüfen“ erzeugt eine **künstliche Bedingung** im Hauptablauf.

★ R8: WIEDERHOLUNGEN

Notation: Wiederhole Aktivität x-y <Endebedingung>

📝 **Beispiel:**

1. Der Kunde wählt eine zu kaufende Aktie anhand ihrer Wertpapiernummer aus.
2. Der Kunde gibt die zu kaufende Stückzahl der ausgewählten Aktie sowie den Börsenplatz für den Kauf an.
 ★ Wiederhole Schritt 1 und 2, bis der Kunde alle zu kaufenden Aktien ausgewählt hat.
3. Der Kunde bestätigt den Kauf der ausgewählten Aktien.
4. Das System kauft die Aktien und bestätigt dem Kunden den Kauf.

G.10 ★★☆☆ DAS PERFEKTE MUSTER-BEISPIEL (auswendig lernen — Formatvorlage für die Klausur!)

1. Use Case: #001 Neukunde erfassen.
2. Ziel: Ein neuer Kunde wurde erfasst.
3. Vorbedingung: Es handelt sich um eine natürliche Person (Endkunde) und die Kundenstammdaten liegen vor.
4. Nachbedingung Erfolg: Die für Auftragsdurchführung und Kundenkommunikation notwendigen Stammdaten des neuen Kunden wurden erfasst und gespeichert.
5. Nachbedingung Fehlschlag: Der Kundenstammdatensatz wurde nicht im CRM-System angelegt. Die Gründe für den Fehlschlag wurden mitgeteilt.
6. Akteur: Kundensachbearbeiter.
7. Auslösendes Ereignis: Ein Kaufauftrag wurde durch einen Neukunden erteilt.
8. Beschreibung:
 1. Der Kundensachbearbeiter VALIDIERT die Kundenstammdaten (Vorname, Nachname, Lieferadresse, Rechnungsadresse, Geburtsdatum) auf Plausibilität und Vollständigkeit.
 2. Der Kundensachbearbeiter legt den neuen Kundenstammsatz im CRM-System an.
 3. Das CRM-System generiert die Kundennummer und speichert sie im Kundenstammsatz.

Alternativen:

 - 2a. Die Validierung war nicht erfolgreich: Unter Angabe der Gründe für den Fehlschlag bricht der Kundensachbearbeiter den UC ab.

★ Was macht dieses Beispiel vorbildlich? (Checkliste für Ihre Klausuraufgabe!)

✓	Regel
Nummer + 2-3-Wort-Name („#001 Neukunde erfassen“)	Schablonenpunkt 1
Ziel als ZUSTAND im Perfekt („Ein neuer Kunde wurde erfasst “)	Punkt 2
Vorbedingung als Zustand , nicht als Aktion	Punkt 3
Nachbedingung Fehlschlag ist AUSGEFÜLLT (nicht „keiner“!)	Punkt 5
Nur der handelnde Akteur (Kundensachbearbeiter) — NICHT das CRM-System!	Punkt 6, „System is not an actor“
Auslösendes Ereignis ist ein EREIGNIS , kein Klick	Punkt 7
„VALIDIERT“ statt „prüft“	R7
Jeder Satz: Subjekt-Verb-Objekt, Akteur genannt	R1, R2
Nur 3 Aktivitäten (< 9-10)	R4
Absicht erkennbar („auf Plausibilität und Vollständigkeit“)	R5
System reagiert und liefert Ergebnis (Schritt 3)	R6
Alternative bezieht sich auf die richtige Aktivität (2a) und endet mit „bricht den UC ab“	Alternative = Endzustand

▼ ? Selbsttest Teil G

1. Was ist ein Use Case? (Definition!)
2. Was ist ein Akteur? Nennen Sie die 3 Arten mit je einem Beispiel. (*Klausurfrage 3x!*)
3. Wo befinden sich Akteure immer?
4. Ist das zu modellierende System ein Akteur? Begründen Sie.
5. Erläutern Sie das Systemverständnis in Geschäftsprozessmodell vs. Product Use Case. (*4 Punkte!*)
6. Nennen Sie alle 8 Punkte der Use-Case-Schablone in der richtigen Reihenfolge.
7. Muss es eine Nachbedingung Fehlschlag geben?
8. Erklären Sie den Unterschied Erweiterung vs. Alternative über das Rückkehrverhalten.
9. Kann es zu einer Alternative Erweiterungen geben?
10. Was ist include, was extend? Nennen Sie je 3 Unterscheidungsmerkmale.
11. In welche Richtung zeigt der extend-Pfeil? Warum?
12. Was ist ein Extension Point und wo wird er notiert?
13. Was ist ein Use-Case-Diagramm? (*SC-Falle!*) Welche Notation gilt für Akteure, UCs, System?
14. Was besagt der Coffee-Break-Test? Wofür ist er die Daumenregel?
15. Wie lange dauert ein UC für ein operatives Ziel typischerweise?
16. Nennen Sie die 8 Regeln R1-R8 für gute Use Cases.
17. Was besagt „Who has the ball“? Warum ist das nicht dasselbe wie R3?
18. Wie viele Aktivitäten hat maximal ein guter Use Case?
19. Nennen Sie die 4 Schritte des Ereignis-Reaktionsschemas (R6).
20. Warum „validieren“ statt „prüfen“?
21. Wie notiert man Wiederholungen?
22. Schreiben Sie den Muster-UC #001 Neukunde erfassen aus dem Gedächtnis auf.

TEIL H — LE 8: AKTIVITÄTSDIAGRAMME UND BPMN

H.1 ★★ Warum überhaupt Aktivitätsdiagramme?

⚠ **Nachteil der Use-Case-Schablone:** Die Use-Case-Schablone SUGGERIERT EINE SEQUENZ der Abarbeitung, die u. U. gar nicht gegeben ist.

📝 **Beispiel (UC #002 Kundenauftrag ausführen):** Aktivität 4 („Auftrag vom Lager ausführen lassen“) und 5 („Rechnungskopie an Buchhaltung geben“) müssen NICHT sequentiell ablaufen — sie können **parallel** erfolgen.

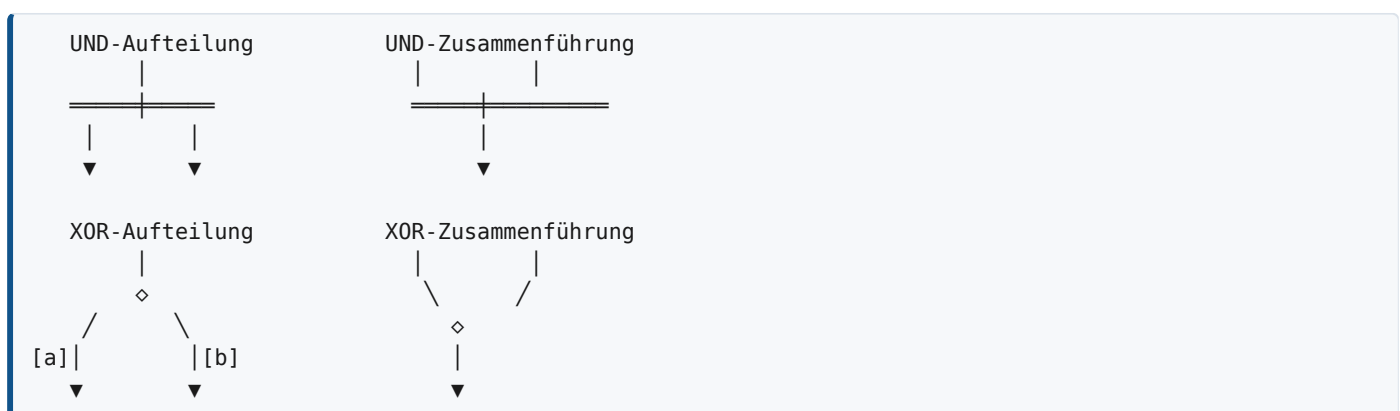
✅ **Dieser Nachteil kann durch AKTIVITÄTSDIAGRAMME beseitigt werden.**

H.2 ★★★★★ Die Notationssymbole eines Aktivitätsdiagramms (UML)

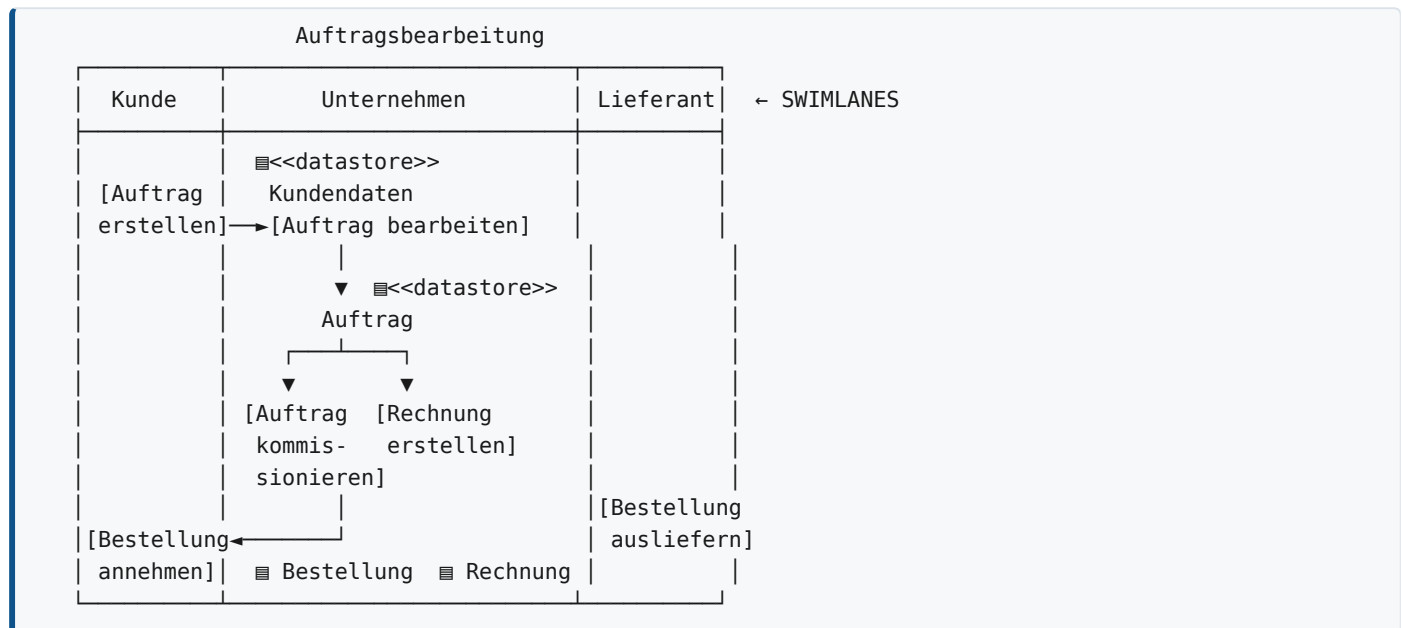
Symbol	Bedeutung
▭ (abgerundetes Rechteck)	Aktivität — Aktivitätsschritt aus dem UC
◇ (Raute)	Entscheidungsknoten
▬ (dicker Balken)	Synchronisationsbalken
● (ausgefüllter Kreis)	Startknoten
⦿ (Kreis mit Ring)	Endeknoten
—[Bedingung]→	Kontrollfluss mit Bedingungsangabe

★★★★ Die logischen Operatoren (auswendig!)

Operator	Symbol/Knoten
Aufteilendes UND	Parallelisierungsknoten (Balken, 1 Eingang → n Ausgänge)
Zusammenführendes UND	Synchronisationsknoten (Balken, n Eingänge → 1 Ausgang)
Aufteilendes XOR	Entscheidungsknoten (Raute, 1 Eingang → n Ausgänge mit Bedingungen)
Zusammenführendes XOR	Zusammenführungsknoten (Raute, n Eingänge → 1 Ausgang)



H.3 ★★ Weiterführende Elemente: Swimlanes und Datenspeicher



NB des Dozenten: „ <<datastore>> kann man auch weglassen. Es kennzeichnet lediglich die **persistente Speicherung**.“

★ Exkurs: Gedanklicher Zusammenhang von SA und AD

Ein **Datenspeicher im Datenflussdiagramm (SA)** entspricht einem <<datastore>> -Objektknoten im Aktivitätsdiagramm; ein **Datenfluss** entspricht einem **Objektfluss** (mit **Pin**).

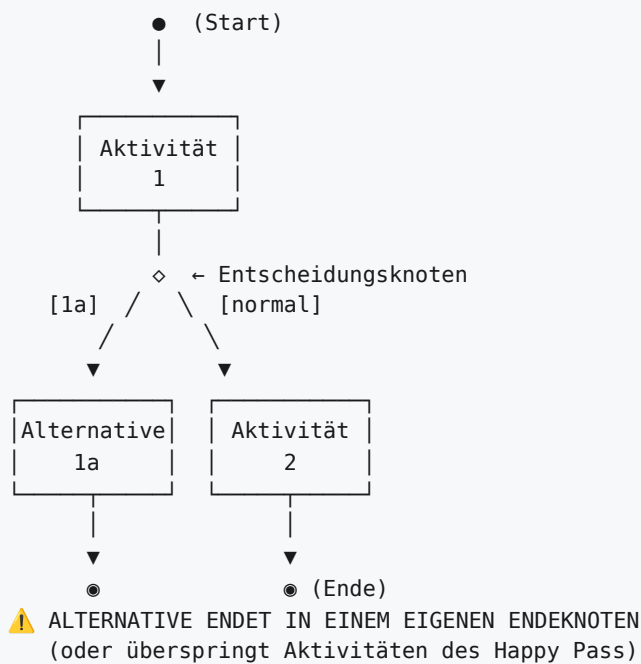
SA: Artikeldata —Artikelnr→ [Preis finden] —Preis→

AD: <<datastore>> Artikeldata —(Pin: Artikelnr)→ [Preis finden] —(Pin: Preis)→

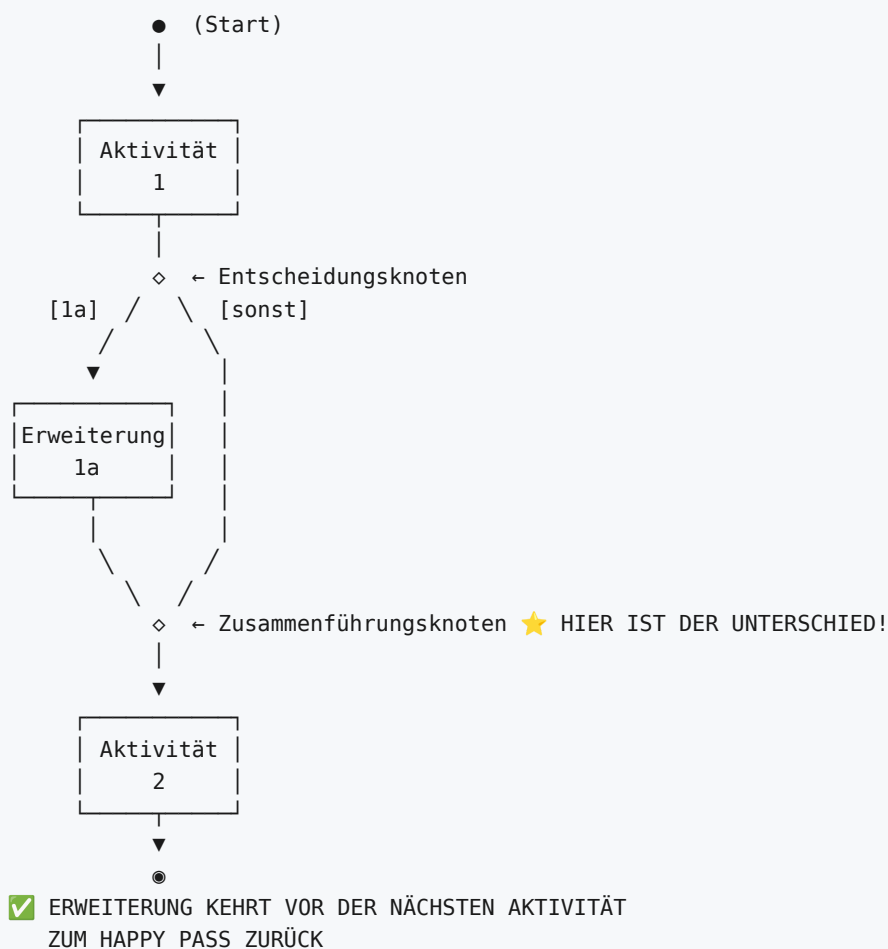
H.4 ★★★ DIE MUSTER FÜR ALTERNATIVEN UND ERWEITERUNGEN (Klausuraufgabe 2024, 10 Punkte!)

Klausuraufgabe: „Erstellen Sie mittels der Methode der Aktivitätsdiagramme das **MUSTER** für (a) **ALTERNATIVEN** und (b) **ERWEITERUNGEN** in einem Product Use Case.“

(a) ★★★ Muster für ALTERNATIVEN

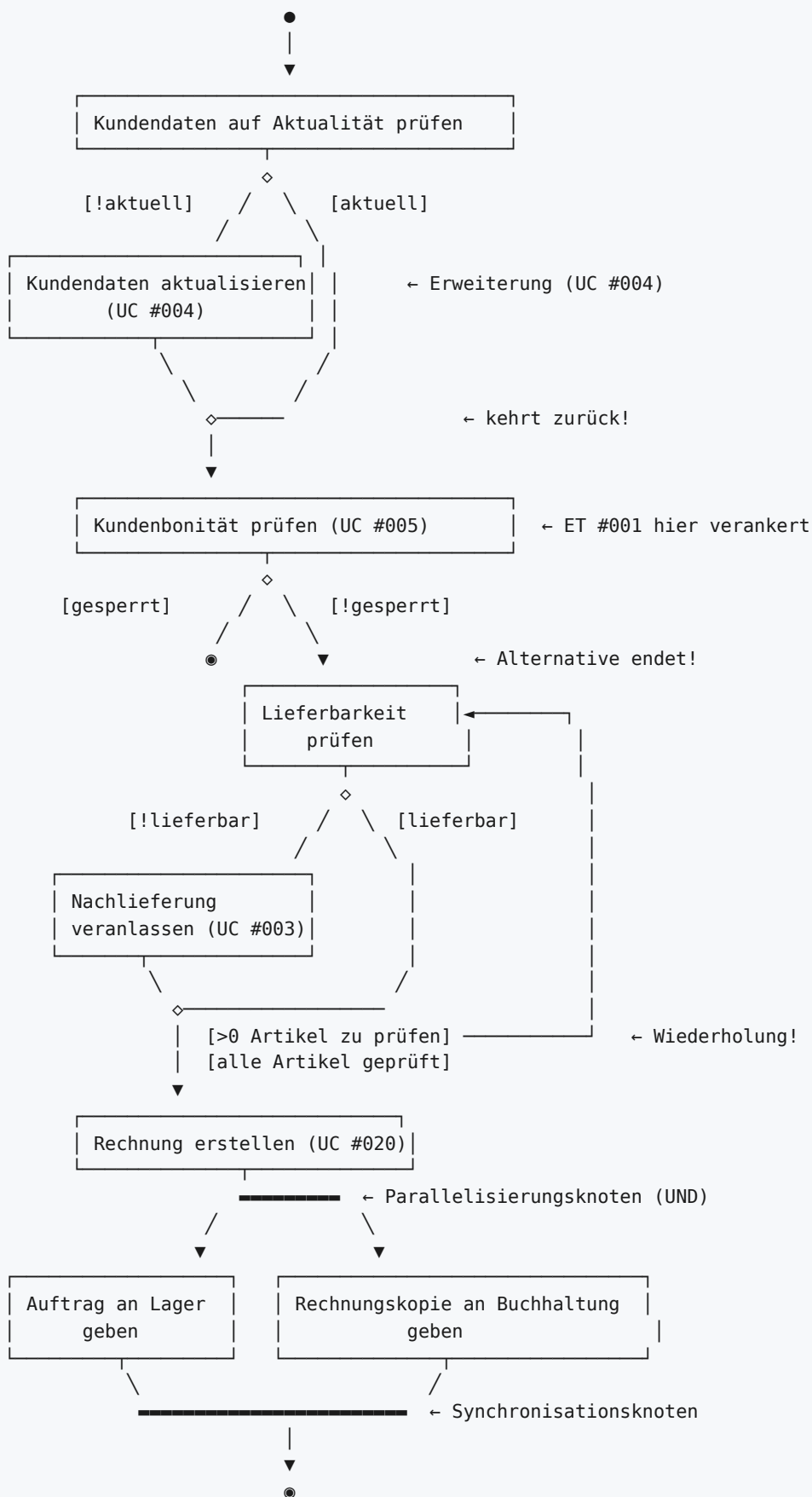


(b) ★★★ Muster für ERWEITERUNGEN



🔑 **Der EINE Unterschied, der die 10 Punkte bringt:** Die **Erweiterung** hat einen **Zusammenführungsknoten** und kehrt in den Hauptablauf zurück — die **Alternative** hat **einen eigenen Endeknoten** (oder überspringt Aktivitäten).

H.5 Das vollständige AD-Beispiel des Dozenten (UC #002 mit ET-Integration)



⚠ **Vergessen Sie nicht die Anpassungen im AD!** — Wenn der Use Case geändert wird (z. B. um die Bonitätsprüfung), **muss das AD nachgezogen werden.**

H.6 ★★ BPMN 2.0

💬 **Der Dozent:** „BPMN 2.0 — die ELEGANTERE Lösung, aber halt nicht UML...“

★ Die wichtigsten BPMN-Notationssymbole (Grundwissen für die 5-Punkte-Aufgabe)

Kategorie	Symbol	Bedeutung
Ereignisse (Events)	○ dünner Kreis	Start-Ereignis
	⊙ doppelter Kreis	Zwischen-Ereignis
	● dicker Kreis	End-Ereignis
Aktivitäten	▭ abgerundetes Rechteck	Task / Aufgabe
	▭ mit „+“	Sub-Prozess (aufklappbar)
Gateways	◇ mit ×	exklusives Gateway (XOR) — genau ein Pfad
	◇ mit +	paralleles Gateway (AND) — alle Pfade
	◇ mit ○	inklusives Gateway (OR) — ein oder mehrere Pfade
Verbindungen	→ durchgezogen	Sequenzfluss
	---→ gestrichelt	Nachrichtenfluss (zwischen Pools!)
	···· gepunktet	Assoziation (zu Datenobjekten/Annotationen)
Artefakte	≡ Blattsymbol	Datenobjekt
	▭ Zylinder	Datenspeicher
Container	großes Rechteck	Pool (ein Beteiligter/Organisation)
	Zeile im Pool	Lane (Rolle/Abteilung)

★★★★ DIE 3 LESSONS LEARNED (aus der Studierendenpräsentation — Prüfungsgold!)

Lesson Learned 1: BPMN — VERMEIDE KREUZUNGEN so gut es geht

Lesson Learned 2: BPMN — IMMER GATEWAYS EINFÜGEN zum ZUSAMMENFÜHREN nach Erweiterungen und Alternativen. *Ein Gateway kann auch **mehr als 2 Eingänge** haben.*

Lesson Learned 3: KLARE UNTERSCHIEDUNG Erweiterung vs. Alternative

- **Erweiterung:** kehrt VOR DER NÄCHSTEN AKTIVITÄT zum Happy Pass zurück
- **Alternative:** kann zum Happy Pass zurückkehren, aber ENDET MINDESTENS EINMAL IN EINEM ENZZUSTAND oder UMGEHT Aktivitäten des Happy Pass

⚠ Der Klausurhinweis

In der Klausur WDSx23 lautet die Aufgabe: *„...und erstellen Sie das zugehörige **BPMN-Modell (OHNE SWIMLANES, bitte).**“* 🗝 **Also: keine Pools/Lanes zeichnen — nur den reinen Ablauf mit Start-Event, Tasks, Gateways und End-Events!**

H.7 ★★ Grundsätze ordnungsgemäßer Modellierung (GoM) — die 6 Grundsätze

? Die Abschlussfrage des Dozenten: „Was kennzeichnet eine gute Modellierung Ihrer Meinung nach?“

#	Grundsatz	Kerninhalt
1	Grundsatz der RICHTIGKEIT	Das Modell muss den zu repräsentierenden Sachverhalt korrekt wiedergeben = SEMANTISCHE Richtigkeit . ⚠ Die Richtigkeit von Modellen ist grundsätzlich NICHT BEWEISBAR, sondern ergibt sich aus dem KONSENS der Fach- und Methodenexperten. Davon abzugrenzen: SYNTAKTISCHE Richtigkeit = Einhaltung der (ggf. individuell definierten) Notationsregeln
2	Grundsatz der RELEVANZ	Es sollen nur die Sachverhalte modelliert werden, die für den Modellierungszweck relevant sind. Dafür müssen die Ziele der Modellierung EXPLIZIERT werden → daran werden Abstraktionsniveau und Modellierungstechniken entschieden
3	Grundsatz der WIRTSCHAFTLICHKEIT	Die Modellierungsaktivitäten müssen in einem angemessenen Kosten-Nutzen-Verhältnis stehen. ⚠ Die Modellierungskosten dürfen den Nutzen NICHT ÜBERKOMPENSIEREN
4	Grundsatz der KLARHEIT	Ein Modell ist nur von Nutzen, wenn es vom ADRESSATEN VERSTANDEN wird. Abhängig vom Modellnutzer braucht es einen adäquaten Grad an intuitiver Lesbarkeit → nutzeradäquate Technikwahl + klare, lesbare Darstellung
5	Grundsatz der VERGLEICHBARKEIT	In realen Situationen existieren mehrere Modelle nebeneinander , die vergleichbar sein müssen. Besondere Bedeutung: Vergleich von IST- und SOLL-Modellen , damit Gestaltungsempfehlungen abgeleitet werden können. Auch Modelle verschiedener Techniken sind vergleichbar zu gestalten
6	Grundsatz des SYSTEMATISCHEN AUFBAUS	Ein Sachverhalt wird aus unterschiedlichen SICHTEN dargestellt (Daten-, Funktions-, Organisations- und Steuerungssicht bzw. Struktur- und Verhaltenssicht). Gefordert wird eine SICHTENÜBERGREIFENDE Modellerstellung — gefördert durch ein sichtenübergreifendes METAMODELL und konsistente Modellinhalte

📖 Quelle: Enzyklopädie der Wirtschaftsinformatik, „Grundsätze ordnungsgemäßer Modellierung“

▼ ? Selbsttest Teil H

1. Was ist der Nachteil der Use-Case-Schablone und wie wird er behoben?
2. Nennen Sie die 6 Notationssymbole eines Aktivitätsdiagramms.
3. Ordnen Sie zu: aufteilendes UND / zusammenführendes UND / aufteilendes XOR / zusammenführendes XOR → welcher Knoten?
4. Zeichnen Sie das Muster für Alternativen. (5 Punkte!)
5. Zeichnen Sie das Muster für Erweiterungen. (5 Punkte!)
6. Was ist der zeichnerische Unterschied zwischen beiden?
7. Was bedeutet <<datastore>> ? Kann man es weglassen?
8. Nennen Sie die BPMN-Symbole für Start-, Zwischen- und End-Ereignis.
9. Welche 3 Gateway-Typen gibt es in BPMN und wofür stehen sie?
10. Was ist der Unterschied zwischen Sequenzfluss und Nachrichtenfluss?
11. Nennen Sie die 3 BPMN-Lessons-Learned.
12. Nennen und erläutern Sie die 6 Grundsätze ordnungsgemäßer Modellierung.
13. Was ist der Unterschied zwischen semantischer und syntaktischer Richtigkeit? Warum ist semantische Richtigkeit nicht beweisbar?

TEIL I — LE 9: ENTSCHEIDUNGSTABELLEN ★★ ★

(in JEDER Klausur, 10 Punkte!)

I.1 ★ Wann braucht man eine Entscheidungstabelle?

Bei STARK REGELBEHAFTETEN Anforderungen.

Die Ausgangssituation des Dozenten (das didaktische Szenario)

Freitag-Nachmittag. Sie freuen sich auf das Wochenende... Im letzten Meeting der Woche mit dem Kunden Ihres CRM-Systems führen Sie ein Review zu **UC #020 (Rechnung erstellen)** durch, dabei erfahren Sie, dass **vor der Rechnungsstellung die Bonität des Kunden positiv geprüft sein muss. Was tun?**

★★ Der Ablauf bei einer NEUEN Anforderung (3 Schritte, auswendig!)

① ANFORDERUNG ERFASSEN - ERM erstellen - Erst einmal in PROSA - Auf Alternativen und Erweiterungen achten - ABGRENZUNG: muss das noch ins System?	② AUSWIRKUNG AUF BESTEHENDES MODELL KLÄREN - Ist die neue Anforderung ein EIGENER Product Use Case? - Welche Auswirkungen auf bestehende Product Use Cases gibt es?	③ MODELL ANPASSEN - Änderungen am Modell vornehmen - Dabei auf mögliche SEITENEFFEKTE achten
---	---	--

Der Kunde im O-Ton (das typische unstrukturierte Anforderungsgespräch)

„Wenn ein neuer Kundenauftrag ausgeführt wird, dann schreiben wir eine Rechnung – schon richtig in Ihrem UC #020. Aber wenn so ein Auftrag reinkommt, dann müssen wir erst einmal klären, ob das einer von unseren **schrägen Vögeln** ist, die immer bestellen, aber dann eine schlechte Zahlungsmoral haben. **DHBW-Studenten – zum Beispiel – sind echt ein Problem für uns.** Deshalb prüfen wir das gleich zu Beginn des Auftragsprozesses. Wenn das nämlich ein schräger Vogel ist, verweigern wir die Ausführung des Auftrags.“

Und die Bonitätsprüfung im Detail: „Wir prüfen da mehrere Sachen. Also zunächst einmal, ob der **Kunde überhaupt noch bei uns kaufen darf**. Denn ansonsten hat sich das gleich erledigt. Ansonsten prüfen wir, ob er noch **offene Rechnungen** hat und wenn ja, wie hoch die sind. Wenn die über 750, nein, falsch, das wurde ja mittlerweile auf **1000 EUR** geändert, also, wenn die über den Betrag sind, dann geht nur **Bankeinzug. Nachnahme** lassen wir übrigens nur bei Kunden mit **einwandfreiem Zahlungsverhalten** zu, also wenn in der Kundenhistorie **nicht mehr als zwei 2. Mahnungen** stehen, denn wissen Sie, wir haben da schon erlebt, dass...“

? „Alles kapiert?“ → Genau hierfür braucht man eine Entscheidungstabelle!

1.2 ★★★★★ DER AUFBAU EINER ENTSCHEIDUNGSTABELLE

ET #001	R1	R2	R3	R4	← REGELTEIL
B1: <Bedingung 1>	J	J	N	N	↑
B2: <Bedingung 2>	J	N	J	N	BEDINGUNGSTEIL
B3: <Bedingung 3>	...	...	...	...	↓
A1: <Aktion 1>	X		X		↑
A2: <Aktion 2>		X		X	AKTIONSTEIL
A3: <Aktion 3>				X	↓

Bereich	Inhalt
Bedingungsteil	Die Bedingungen B1...Bn mit den Werten J (Ja), N (Nein) oder - (irrelevant/beliebig)
Aktionsteil	Die Aktionen A1...Am , markiert mit X , wenn sie bei dieser Regel ausgeführt werden
Regelteil	Die Spalten R1...Rk — jede Spalte ist eine REGEL

★★★★★ DIE GOLDENE FORMEL

Anzahl Regeln der VOLLSTÄNDIGEN Tabelle = 2^n (n = Anzahl der Bedingungen)

n	Regeln
2	4
3	8
4	16
5	32

★★★★★ DAS AUSFÜLLSCHEMA (immer gleich — so vergisst man keine Kombination!)

Bei $n = 3$ Bedingungen (8 Spalten):

B1:	J	J	J	J	N	N	N	N	← 4× J, 4× N	($2^{(n-1)} = 4$ er-Blöcke)
B2:	J	J	N	N	J	J	N	N	← 2× J, 2× N	(2er-Blöcke)
B3:	J	N	J	N	J	N	J	N	← 1× J, 1× N	(Wechsel jede Spalte)

🔑 **Merkregel:** Die oberste Bedingung wechselt am langsamsten, die unterste am schnellsten — genau wie beim Binärzählen!

I.3 ★★★★★ DAS DURCHGERECHNETE BEISPIEL DES DOZENTEN (ET #001 Bonitätsprüfung)

Schritt 1 — die VOLLSTÄNDIGE Tabelle ($n = 3 \Rightarrow 2^3 = 8$ Regeln)

ET #001	R1	R2	R3	R4	R5	R6	R7	R8
B1: Kunde gesperrt	J	J	J	J	N	N	N	N
B2: Offene Rechnungen > 1000 EUR	J	J	N	N	J	J	N	N
B3: Einwandfreies Zahlungsverhalten	J	N	J	N	J	N	J	N
A1: Verweigerung	X	X	X	X				
A2: Bankeinzug					X	X	X	
A3: Nachnahme								X

(Anmerkung: Der Dozent nummeriert die dritte Bedingung auf der Folie als „B4“ — inhaltlich ist es die dritte.)

Schritt 2 — KONSOLIDIERUNG

Eine Entscheidungstabelle ist **KONSOLIDIERBAR**, wenn **MEHRERE REGELN ZU EINER ZUSAMMENGEFASST** werden können. Hier: R1–R4 (denn wenn der Kunde gesperrt ist, sind B2 und B3 irrelevant)

ET #001 (konsolidiert)	R1	R5	R6	R7	R8
B1: Kunde gesperrt	J	N	N	N	N
B2: Offene Rechnungen > 1000 EUR	–	J	J	N	N
B3: Einwandfreies Zahlungsverhalten	–	J	N	J	N
A1: Verweigerung	X				
A2: Bankeinzug		X	X	X	
A3: Nachnahme					X

? „Frage: Kann man noch weitere Regeln zusammenfassen?“ → **Ja!** R5 und R6 unterscheiden sich nur in B3 und haben **dieselbe Aktion** → zu (N, J, –) → **A2** zusammenfassbar. Ebenso wären dann (N, J, –) und (N, N, J) **nicht** weiter zusammenfassbar (zwei Unterschiede). **Maximal konsolidierte Fassung:**

	R1	R2	R3	R4
B1: Kunde gesperrt	J	N	N	N
B2: Offene Rechn. > 1000 €	–	J	N	N
B3: Einwandfr. Zahlungsverh.	–	–	J	N
A1: Verweigerung	X			
A2: Bankeinzug		X	X	
A3: Nachnahme				X

★★★★★ DIE KONSOLIDIERUNGSREGEL (auswendig!)

Zwei Regeln lassen sich zusammenfassen, wenn sie sich in **GENAU EINER** Bedingung unterscheiden **UND EXAKT**

I.4 ★★★★★ REDUNDANZFREIHEIT und WIDERSPRUCHSFREIHEIT

★ Redundanz (die Klausuraufgabe 5 der neuesten Klausur!)

	R1	R2
B1	J	J
B2	N	-
B3	N	N
A1	X	X
A2	X	X

⚠ **REDUNDANZ:** R1 braucht es nicht, da R2 diese Regel schon beinhaltet für B2 = „N“. 🔑
Erkennungsmerkmal: Eine Regel mit „-“ DECKT eine spezifischere Regel mit demselben Aktionsmuster bereits AB. Korrektur: R1 STREICHEN.

★ Widerspruch / Inkonsistenz

	R1	R2
B1	J	J
B2	N	N
B3	-	N
A1	X	X
A2	X	

⚠ **WIDERSPRUCH:** R1 steht bei B3 = „N“ mit R2 im Widerspruch, denn R1 hat A1 UND A2 als Aktion, R2 aber NUR A1. 🔑 **Erkennungsmerkmal:** Zwei Regeln decken dieselbe Bedingungskombination ab, schreiben aber **UNTERSCHIEDLICHE** Aktionen vor. Korrektur: R1 auf B3 = „J“ konkretisieren (dann überlappt sie nicht mehr mit R2) oder die Aktionen angleichen.

★★ Die Prüf-Checkliste für „konsolidiert und widerspruchsfrei“

Prüfung	Frage	Wie prüfen?
① VOLLSTÄNDIGKEIT	Sind alle 2ⁿ Kombinationen abgedeckt?	Jede der 2 ⁿ Kombinationen durchgehen und prüfen, ob genau eine Regel passt
② EINDEUTIGKEIT / WIDERSPRUCHSFREIHEIT	Passt auf jede Kombination genau EINE Regel?	Regelpaare auf Überlappung prüfen; bei Überlappung müssen die Aktionen identisch sein
③ REDUNDANZFREIHEIT	Ist keine Regel überflüssig?	Prüfen, ob eine Regel von einer anderen (mit „-“) vollständig abgedeckt wird
④ KONSOLIDIERUNG	Lassen sich noch Regeln zusammenfassen ?	Regelpaare suchen, die sich in genau einer Bedingung unterscheiden und identische Aktionen haben

I.5 ★★★★★ Integration der ET in Use Case und Aktivitätsdiagramm

!★ Die Single-Choice-Falle (Klausur 2020)

„Wie kann eine Entscheidungstabelle in einen Use Case integriert werden?" **A. Auf sie wird IN DER ENTSPRECHENDEN AKTIVITÄT VERWIESEN** ✓ B. Sie wird als Tabelle im Erweiterungsteil eingefügt C. Gar nicht

Der angepasste UC #002 (mit integrierter ET #001)

Use Case: #002 Kundenauftrag ausführen
 Ziel: Ware an Kunden geliefert
 Vorbedingung: -
 Nachbedingung Erfolg: Ware ausgeliefert (auch Teillieferungen),
 Rechnungskopie bei Buchhaltung
 Nachbedingung Fehlschlag: Mitteilung an Kunden, dass nichts lieferbar ist
 Akteure: Kundensachbearbeiter, Lagersachbearbeiter, Buchhaltung
 Auslösendes Ereignis: Bestellung des Kunden liegt vor
 Beschreibung:
 1 Die Kundendaten werden auf Aktualität geprüft
 2 Den Kunden ANHAND ET #001 auf Bonität prüfen ← ★ HIER die ET!
 3 Lieferbarkeit prüfen
 4 Rechnung erstellen: UC #020 Rechnung erstellen
 5 Auftrag vom Lager ausführen lassen
 6 Rechnungskopie an Buchhaltung geben
 Erweiterung:
 1a Wenn die gespeicherten Kundendaten nicht aktuell sind:
 UC #004 Kundendaten aktualisieren
 5a Für nicht-lieferbare Artikel: UC #003 Nachlieferung veranlassen
 Alternativen:
 1a Wenn der Besteller noch nicht Kunde ist: UC #001 Neukunde erfassen
 2a Wenn der Kunde gesperrt ist: Auftragsannahme verweigern (UC beenden)
 4a Rechnung mit Nachnahme erstellen (UC #020 Rechnung erstellen)
 4b Rechnung mit Bankeinzug erstellen (UC #020 Rechnung erstellen)

 **So wird die ET integriert:** In der **Aktivität** wird auf die ET **verwiesen** („anhand ET #001"), und die **Aktionen der ET** werden zu **Alternativen** des UC (2a = Verweigerung, 4a = Nachnahme, 4b = Bankeinzug).

 „Vergessen Sie nicht die Anpassungen im AD!"

I.6 ★★★★★ ALLE KLAUSUR-ENTSCHEIDUNGSTABELLEN — VOLLSTÄNDIG GELÖST

ET-Aufgabe 1: Buchhandlung „BäMs Lesefreuden" (Klausur 2020, 10 Punkte)

„Muss in der Buchhandlung ‚BäMs Lesefreuden‘ für einen Kunden ein Buch bestellt werden, weil es nicht vorrätig ist, wird zunächst geprüft, **ob das Buch lieferbar ist**. Für **nicht-lieferbare** Bücher wird die **Bestellung abgelehnt**. Bei lieferbaren Büchern wird die **Bonität des Kunden** geprüft. Wenn diese **nicht in Ordnung** ist, kann **nur auf Vorkasse** bestellt werden. Ist die Bonität **in Ordnung**, wird **immer auf Rechnung** bestellt und der Kunde zahlt erst nach Erhalt des Buchs."

Schritt 1 — Bedingungen und Aktionen identifizieren:

- **B1: Buch lieferbar?**
- **B2: Bonität in Ordnung?**
- A1: Bestellung ablehnen · A2: Bestellung auf Vorkasse · A3: Bestellung auf Rechnung

Schritt 2 — Vollständige Tabelle ($2^2 = 4$ Regeln):

ET	R1	R2	R3	R4
B1: Buch lieferbar	J	J	N	N
B2: Bonität in Ordnung	J	N	J	N
A1: Bestellung ablehnen			X	X
A2: Bestellung auf Vorkasse		X		
A3: Bestellung auf Rechnung	X			

Schritt 3 — Konsolidieren (R3 + R4, da bei nicht-lieferbar die Bonität irrelevant ist):

✓ ET (konsolidiert)	R1	R2	R3
B1: Buch lieferbar	J	J	N
B2: Bonität in Ordnung	J	N	-
A1: Bestellung ablehnen			X
A2: Bestellung auf Vorkasse		X	
A3: Bestellung auf Rechnung	X		

ET-Aufgabe 2: Krankenhaus-Besuchsregelung (Klausuren 2021 UND 2022, je 10 Punkte!)

*„Die Patientin kann **ohne Einschränkungen** innerhalb der Besuchszeit besucht werden, sofern **keine ansteckende Krankheit** bei ihr vorliegt und sie **kein Fieber** hat. **Außerhalb der Besuchszeit** ist grundsätzlich eine **Krankenschwester als Begleitung** erforderlich. Falls die Patientin eine **ansteckende Krankheit** hat, werden Besuche **ganz abgelehnt**. Wenn die Krankheit **nicht ansteckend** ist, die Patientin aber **Fieber** hat, darf der Besuch **innerhalb der Besuchszeit maximal 30 Minuten** betragen. **Außerhalb der Besuchszeiten dürfen Patienten mit Fieber NICHT besucht werden.**“*

Schritt 1 — Bedingungen und Aktionen:

- **B1: Ansteckende Krankheit?**
- **B2: Fieber?**
- **B3: Innerhalb der Besuchszeit?**
- A1: Besuch ablehnen · A2: Besuch ohne Einschränkung · A3: Besuch max. 30 Minuten · A4: Begleitung durch Krankenschwester

Schritt 2 — Vollständige Tabelle ($2^3 = 8$ Regeln):

ET	R1	R2	R3	R4	R5	R6	R7	R8
B1: Ansteckende Krankheit	J	J	J	J	N	N	N	N
B2: Fieber	J	J	N	N	J	J	N	N
B3: Innerhalb Besuchszeit	J	N	J	N	J	N	J	N
A1: Besuch ablehnen	X	X	X	X		X		
A2: Besuch ohne Einschränkung							X	
A3: Besuch max. 30 Minuten					X			
A4: Begleitung Krankenschwester								X

Schritt 3 — Konsolidieren (R1–R4: bei ansteckender Krankheit sind B2 und B3 irrelevant):

✓ ET (konsolidiert & widerspruchsfrei)	R1	R2	R3	R4	R5
B1: Ansteckende Krankheit	J	N	N	N	N
B2: Fieber	–	J	J	N	N
B3: Innerhalb der Besuchszeit	–	J	N	J	N
A1: Besuch ablehnen	X		X		
A2: Besuch ohne Einschränkung				X	
A3: Besuch max. 30 Minuten		X			
A4: Begleitung durch Krankenschwester					X

⚠ **Häufiger Fehler:** R1 und R3 haben zwar beide „Besuch ablehnen“, lassen sich aber **NICHT** zusammenfassen — sie unterscheiden sich in **mehr als einer** Bedingung (B1 und B2 und B3).

📝☆☆☆ ET-Aufgabe 3: Onlineshop-Rabatt (Klausur WDSx23 2024/12, 10 Punkte — die NEUESTE!)

Ein Onlineshop bietet Kunden Rabatt, abhängig von drei Bedingungen: 1. **Kundenstatus:** Stammkunde oder Neukunde · 2. **Bestellwert:** über 100 € oder nicht · 3. **Saisonales Angebot:** Sonderangebot aktiv oder nicht

Regeln für die Rabattvergabe:

- Wenn der Kunde ein **Stammkunde** ist **und** der Bestellwert **über 100 €** liegt, erhält er **immer** einen Rabatt, **unabhängig davon, ob ein Sonderangebot aktiv ist**.
- **Neukunden** erhalten **nur dann** einen Rabatt, wenn der Bestellwert **über 100 €** liegt **und** ein **Sonderangebot aktiv** ist.
- **Alle anderen** Kombinationen führen zu **keiner Rabattgewährung**.

Schritt 1 — Bedingungen und Aktionen:

- **B1: Kunde ist Stammkunde?** (J = Stammkunde, N = Neukunde)
- **B2: Bestellwert > 100 €?**
- **B3: Sonderangebot aktiv?**
- A1: Rabatt gewähren · A2: Kein Rabatt

Schritt 2 — Vollständige Tabelle ($2^3 = 8$ Regeln):

ET	R1	R2	R3	R4	R5	R6	R7	R8
B1: Stammkunde	J	J	J	J	N	N	N	N
B2: Bestellwert > 100 €	J	J	N	N	J	J	N	N
B3: Sonderangebot aktiv	J	N	J	N	J	N	J	N
A1: Rabatt gewähren	X	X			X			
A2: Kein Rabatt			X	X		X	X	X

Schritt 3 — Konsolidieren:

- **R1 + R2** → (J, J, -) → **Rabatt** (Stammkunde + >100 € → immer Rabatt)
- **R3 + R4** → (J, N, -) → kein Rabatt
- **R7 + R8** → (N, N, -) → kein Rabatt
- **(J, N, -) + (N, N, -)** → ★ (-, N, -) → **kein Rabatt** (unterscheiden sich nur in B1!)
- **R5** → (N, J, J) → **Rabatt**
- **R6** → (N, J, N) → kein Rabatt

✓ ENDERGEBNIS — konsolidierte und widerspruchsfreie Entscheidungstabelle:

ET #001 „Rabattgewährung“	R1	R2	R3	R4
B1: Kunde ist Stammkunde	J	N	-	N
B2: Bestellwert > 100 €	J	J	N	J
B3: Sonderangebot aktiv	-	J	-	N
A1: Rabatt gewähren	X	X		
A2: Kein Rabatt			X	X

Verifikation (alle 8 Kombinationen genau einmal abgedeckt):

Kombination	passende Regel	Aktion
(J,J,J)	R1	Rabatt ✓
(J,J,N)	R1	Rabatt ✓
(J,N,J)	R3	kein Rabatt ✓
(J,N,N)	R3	kein Rabatt ✓
(N,J,J)	R2	Rabatt ✓
(N,J,N)	R4	kein Rabatt ✓
(N,N,J)	R3	kein Rabatt ✓
(N,N,N)	R3	kein Rabatt ✓

✓ Vollständig, widerspruchsfrei, redundanzfrei, maximal konsolidiert.

📝★ ET-Aufgabe 4: „Geld einzahlen“ am Geldautomaten (Übungsaufgabe ET #001)

„Ist die **Karte gültig** und wird die **korrekte PIN** eingegeben, dann wird der **Schacht für die Geldeingabe geöffnet**. Ist die **Karte nicht gültig**, wird sie **zurückgewiesen**. Ist die **PIN falsch**, wird sie **erneut angefordert**. Bei der **dritten fehlerhaften PIN-Eingabe** wird die **Karte einbehalten**. Ist der eingezahlte **Betrag nach seiner Prüfung akzeptabel** (kein Falschgeld, keine Fremdwährung), wird das **Geld auf dem Konto des Kunden verbucht**. Ansonsten wird der **Schacht zur Geldentnahme geöffnet**.“

Schritt 1 — Bedingungen und Aktionen:

- **B1: Karte gültig?**
- **B2: PIN korrekt?**
- **B3: Dritte fehlerhafte PIN-Eingabe?**
- **B4: Betrag akzeptabel?**
- A1: Karte zurückweisen · A2: PIN erneut anfordern · A3: Karte einbehalten · A4: Schacht für Geldeingabe öffnen · A5: Geld auf Konto verbuchen · A6: Schacht zur Geldentnahme öffnen

Vollständig wären $2^4 = 16$ Regeln — nach Konsolidierung bleiben 5:

ET #001 „Geld einzahlen“	R1	R2	R3	R4	R5
B1: Karte gültig	N	J	J	J	J
B2: PIN korrekt	-	N	N	J	J
B3: 3. fehlerhafte PIN-Eingabe	-	N	J	-	-
B4: Betrag akzeptabel	-	-	-	J	N
A1: Karte zurückweisen	X				
A2: PIN erneut anfordern		X			
A3: Karte einbehalten			X		
A4: Schacht für Geldeingabe öffnen				X	X
A5: Geld auf Konto verbuchen				X	
A6: Schacht zur Geldentnahme öffnen					X

🔑 Die drei „-“-Muster erklärt:

- **R1:** Ist die Karte ungültig, sind **PIN, Versuchsähler und Betrag völlig irrelevant**
- **R2/R3:** Bei falscher PIN wird **gar kein Geld eingezahlt** → **B4 irrelevant**
- **R4/R5:** Bei korrekter PIN ist der **Fehlversuchsähler irrelevant** → **B3 irrelevant**

I.7 ★★ Die Klausur-Arbeitsanleitung für ET-Aufgaben (Schritt für Schritt!)

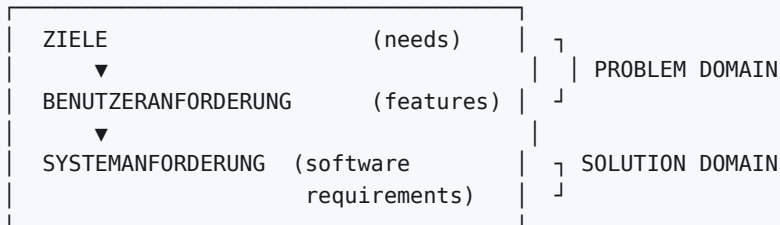
- ① **BEDINGUNGEN** identifizieren
 - Suchen Sie im Text nach allen **ENTSCHEIDUNGSPUNKTEN** (ja/nein-Fragen).
 - Formulieren Sie sie als **AUSSAGE**, die mit J/N beantwortbar ist.
 - Vorsicht: „Stammkunde ODER Neukunde“ ist **EINE** Bedingung, nicht zwei!
- ② **AKTIONEN** identifizieren
 - Alle möglichen **ERGEBNISSE/Reaktionen** auflisten.
 - Auch „nichts tun“ / „ablehnen“ ist eine Aktion!
- ③ **VOLLSTÄNDIGE** Tabelle mit 2^n Spalten aufstellen
 - Ausfüllschema verwenden (oberste Bedingung wechselt am langsamsten).
- ④ Für **JEDE** Spalte die Aktion(en) mit X eintragen
 - Text noch einmal Satz für Satz durchgehen.
 - ⚠ Prüfen: Ist wirklich **JEDE** Spalte belegt? (Vollständigkeit!)
- ⑤ **KONSOLIDIEREN**
 - Regelpaare suchen: genau **EINE** Bedingung unterschiedlich + **IDENTISCHE** Aktionen
 - Zusammenfassen, die unterschiedliche Bedingung wird „–“
 - Solange wiederholen, bis nichts mehr zusammenfassbar ist
 - ⚠ Auch **NACH** einer Zusammenfassung nochmal prüfen (Kaskaden!)
- ⑥ **PRÜFEN**
 - Vollständig? Alle 2^n Kombinationen durch genau eine Regel abgedeckt?
 - Widerspruchsfrei? Keine Überlappung mit unterschiedlichen Aktionen?
 - Redundanzfrei? Keine Regel vollständig durch eine andere abgedeckt?
- ⑦ **ÜBERSCHRIFT** nicht vergessen: „ET #001 <Name>“ und Zeilenbeschriftung B1..., A1...

▼ ? Selbsttest Teil I

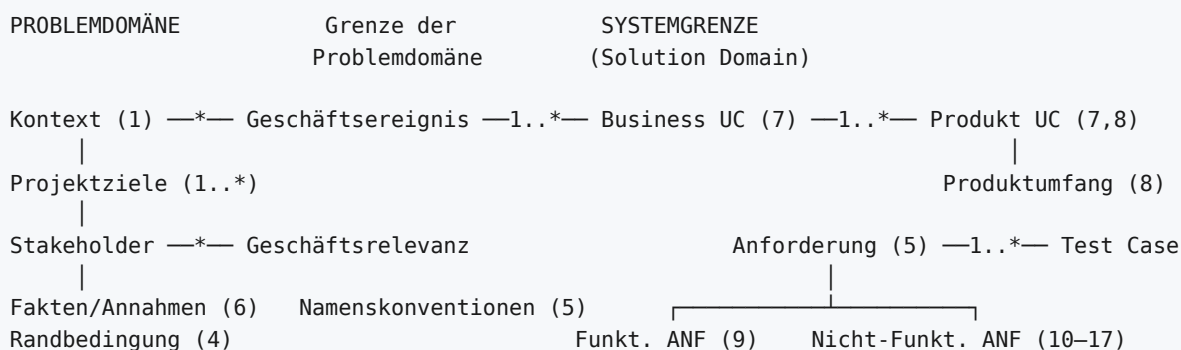
1. Wann setzt man Entscheidungstabellen ein?
2. Nennen Sie die 3 Schritte beim Erfassen einer neuen Anforderung.
3. Aus welchen drei Teilen besteht eine Entscheidungstabelle?
4. Wie viele Regeln hat die vollständige Tabelle bei 4 Bedingungen? Bei 5?
5. Wie lautet die Konsolidierungsregel?
6. Woran erkennt man Redundanz? Wie korrigiert man sie?
7. Woran erkennt man einen Widerspruch? Wie korrigiert man ihn?
8. Wie wird eine ET in einen Use Case integriert? (*SC-Falle!*)
9. Lösen Sie die Buchhandlungs-Aufgabe aus dem Gedächtnis.
10. Lösen Sie die Krankenhaus-Aufgabe. Warum lassen sich R1 und R3 nicht zusammenfassen?
11. Lösen Sie die Onlineshop-Rabatt-Aufgabe. Welche zwei Regeln lassen sich über B1 hinweg zusammenfassen?
12. Erstellen Sie die ET für „Geld einzahlen“. Welche Bedingungen werden wann irrelevant?

TEIL J — LE 10: ANFORDERUNGEN FORMULIEREN

J.1 ★ Was muss modelliert werden? Die Ebenenhierarchie



★ Das Metamodell (Volere-Kapitelnummern in Klammern)



📖 Volere Template: <http://www.volere.co.uk/template.htm> — jede Nummer entspricht dem zugehörigen Kapitel in der Spezifikationsvorlage

J.2 ★★★ DIE 3 ATOMAREN ANFORDERUNGSARTEN (Klausuraufgabe 3, 8 Punkte!)

„Erläutern Sie die Arten von Anforderungen und nennen Sie jeweils ein Beispiel.“

① Funktionale Anforderungen (FANF)

Beschreiben, WELCHE DIENSTE das Produkt zur Verfügung stellt, damit das Geschäftsziel / die Aufgabe erfüllt werden kann.

📝 Beispiele des Dozenten:

- „Das System **muss die Kundenstammdaten auf Plausibilität prüfen.**“
- „Das System **muss dem Sachbearbeiter die Möglichkeit bieten, einen Auftrag für einen Kunden anzulegen.**“

② Nicht-funktionale Anforderungen (NFANF)

Beschreiben, WIE sich das System verhalten soll, z. B. Antwortzeitverhalten, Ergonomieanforderungen, Sicherheitsanforderungen etc. ⇒ **QUALITÄTSANFORDERUNGEN**

Beispiele des Dozenten:

- „Die **Plausibilitätsprüfung** muss nach **maximal 5 Sekunden** erfolgreich abgeschlossen sein.“
- „Die **Bildschirmmasken** für die Sachbearbeiter müssen **ergonomisch EN ISO 9241 entsprechen**.“

③ Randbedingungen (RANF)

Beschreiben, welche **WIRTSCHAFTLICHEN, ZEITLICHEN, RECHTLICHEN, TECHNISCHEN** Restriktionen etc. zu beachten sind. ★ **GEHÖREN ZU DEN NICHT-FUNKTIONALEN ANFORDERUNGEN.**

Beispiele des Dozenten:

- „Das System **muss auf der bestehenden IT-Infrastruktur lauffähig sein**.“
- „Die **SW-Lösung darf nicht mehr als EUR 50.000 kosten**.“

★★ Die Merkregel

WAS liefert das System?	WIE GUT liefert es?	WELCHE GRENZEN gelten?
FANF	NFANF	RANF (Teilmenge der NFANF!)

 **Klausur-Tipp:** Bei der Frage „Erläutern Sie die Arten von Anforderungen“ immer die **drei** nennen **und** dazusagen, dass **Randbedingungen zu den nicht-funktionalen Anforderungen gehören** — das ist die Feinheit, auf die es ankommt.

J.3 ★★★★★ DIE SOPHIST-NORMSPRACHE (Satzschablone)

[<Wann?>	Das System	muss	TYP 1: -
<Unter	Die	sollte	TYP 2: <wem?> die Möglichkeit bieten
welcher	Komponente	wird	TYP 3: fähig sein
Bedingung?>	<Name>		
]			
<Prozesswort> [<Objekt & Ergänzung des Objekts>]			

★★★★ Die 3 Typen (auswendig!)

Typ	Bezeichnung	Bedeutung	Schablonen-Kern
Typ 1	Selbständige Systemaktivität	OHNE Benutzerinteraktion — das System handelt von sich aus	Das System muss <Prozesswort> <Objekt>
Typ 2	Benutzerinteraktion	Das System stellt dem NUTZER eine Funktionalität zur Verfügung	Das System muss <wem?> DIE MÖGLICHKEIT BIETEN , <Objekt> zu <Prozesswort>
Typ 3	Schnittstellenanforderung	Ein EINGEHENDES EREIGNIS aus einem EXTERNEN SYSTEM löst eine Aktion aus	Das System muss FÄHIG SEIN , <Objekt> zu <Prozesswort>

 Rupp, Ch. & die SOPHISTen: Requirements-Engineering und -Management, 5. Aufl., S. 159 ff.

Die Beispiele des Dozenten (aus UC #001 Neukunde erfassen)

Ermittlung der Anforderungen aus Aktivität 1:

FANF#001: Das System **muss dem Kundensachbearbeiter die Möglichkeit bieten**, die Kundenstammdaten eines Neukunden einzugeben. → **TYP 2**

FANF#002: Das System **muss** die Kundenstammdaten des Neukunden auf Plausibilität überprüfen. Plausibilitätskriterien: **ET#001**. → **TYP 1**

FANF#003: Wenn mindestens 1 Plausibilitätskriterium aus FANF#002 nicht erfüllt ist, **muss** das System den Kundensachbearbeiter zur Korrektur auffordern und die Speicherung des Kundenstammdatensatzes verweigern. → **TYP 1 mit Bedingung**

Beispiel Typ 1 — Selbständige Systemaktivität:

FANF #004: Wenn die Plausibilitätsprüfung nach FANF#002 erfolgreich war, **muss das System** den Kundenstammdatensatz des Neukunden in der Datenbank „CRM-DB“ **speichern**.

FANF#005: Wenn ein Neukunde nach FANF#004 erfolgreich angelegt wurde, **muss das System** eine Kundennummer für den Neukunden **generieren**. Regel: Letzte Kundennummer + 1.

FANF #006: Sobald die Kundennummer (FANF#005) erfolgreich erzeugt wurde, **speichert das System** die Kundennummer im Stammdatensatz des Neukunden (vgl. Data Dictionary) in der Datenbank „CRM-DB“ ab.

Beispiel Typ 3 — Schnittstellenanforderung:

NFANF#099: Solange das System online ist, **muss es in der Lage sein**, Softwareupdates von einem zentralen Administrationsrechner über das LAN zu **empfangen und einzuspielen**.

🔑 **Beachte die Verkettung:** Jede Anforderung **referenziert die vorhergehende** („nach FANF#004“, „aus FANF#002“) — das ist die geforderte **Verfolgbarkeit/Traceability**!

J.4 ★★★★★ SCHLÜSSELWÖRTER UND IHRE RECHTSVERBINDLICHKEIT (auswendig!)

Schlüsselwort	Verbindlichkeit	Erläuterung
muss / shall	VERPFLICHTEND	- Die Erfüllung der Anforderung im Produkt ist verpflichtend ⚠️ Die ABNAHME des Produkts kann VERWEIGERT werden, wenn einer <i>muss</i>-Anforderung nicht entsprochen wurde
sollte / soll / should	WÜNSCHENSWERT	Wünsche - sind nicht verpflichtend müssen nicht erfüllt werden - dienen der besseren Zusammenarbeit von Stakeholdern und Entwicklern erhöhen die Zufriedenheit der Stakeholder
wird / will	ZUKÜNFTIG — ABER VERPFLICHTEND!	Anforderungen, die in der Zukunft integriert werden. ★ Zukünftige Anforderungen sind VERPFLICHTEND - Sie helfen, in der aktuellen Lösung Vorbereitungen zu treffen, um Zukünftiges später optimal zu integrieren

⚠️ ★ **Die Falle:** „wird“ klingt unverbindlich, ist aber **verpflichtend** — es verpflichtet dazu, die **Architektur bereits heute so zu bauen**, dass die Zukunftsanforderung integrierbar ist!

✎ ★ Die Übungsaufgabe des Dozenten — Welcher Typ liegt jeweils vor?

Satz	✓ Lösung
1. „Der Kunde will, dass das System Deutsch, Englisch und Chinesisch als Benutzersprache unterstützt.“	MUSS — eine verpflichtende Anforderung an das aktuelle Produkt
2. „Der Kunde möchte, dass das System auch beliebige Sprachen in der Zukunft unterstützen kann.“	WIRD — eine zukünftige, aber verpflichtende Anforderung: die Architektur muss heute schon die spätere Erweiterung ermöglichen (z. B. Internationalisierungs-Framework)
3. „Das System muss nicht mandantenfähig sein, soll diese Fähigkeit aber zukünftig nicht unmöglich machen .“	SOLLTE/WIRD-Mischform — der erste Teil ist eine explizite Nicht-Anforderung , der zweite Teil eine wünschenswerte (bzw. bei strenger Lesart zukünftige) Architekturanforderung

J.5 ★ ★ Die SNOWCARD-Schablone (Volere)

Für JEDE atomare ANF wird die Schablone ausgefüllt!

Feld	Inhalt
ANF #	Eindeutige Nummer
ANF-Typ	Nr. (aus Volere-Template) — z. B. FANF, NFANF, RANF
UC #	Zugehöriger Use Case
Definition	Hier kommt der die ANF beschreibende Satz — ★ Regelfall: 1 SATZ!
Grund	Begründung für die ANF
Quelle	Von wem stammt die ANF?
Abnahme-Kriterium	Messgröße, mit der beim Test ermittelt werden kann, ob die ANF erfüllt ist
Kundenzufriedenheit	Wert auf Zufriedenheitsskala 1 (gleichgültig) bis 5 (extrem glücklich) — wenn die ANF erfolgreich implementiert wurde
Kundenunzufriedenheit	Wert auf Unzufriedenheitsskala 1 (gleichgültig) bis 5 (extrem unzufrieden) — wenn die ANF NICHT Teil des Endprodukts ist
Priorität	Ratingwert des Kundennutzens
Zusatzmaterial	Dokumente, Formulare etc., die für das Verständnis wichtig sind
Änderungshistorie	Erstellungsdatum, Ersteller · Änderungsdatum, Ändernder

Beispiel-Snowcard für FANF#005

Feld	Wert
ANF #	005 · ANF-Typ: FANF · UC #: 001
Definition	Wenn ein Neukunde nach FANF#004 erfolgreich angelegt wurde, muss das System eine Kundennummer für den Neukunden generieren (Regel: Letzte Kundennummer + 1).
Grund	Dient der eindeutigen Identifizierung eines Kunden.
Quelle	Auftragsabteilung
Abnahme-Kriterium	Kundennummer wird fortlaufend vergeben: letzteKundenNr + 1
Kundenzufriedenheit	3
Kundenunzufriedenheit	5
Priorität	5
Zusatzmaterial	Auftragserfassungsformular
Änderungshistorie	2012-05-15, BÄM

★ **Erweiterte Snowcard mit Kano-Index:** Die Felder *Kundenzufriedenheit* und *Kundenunzufriedenheit* werden durch **eine** Zeile ersetzt: **Kano-Merkmalssklasse: M (Basismerkmal)**

J.6 ★★★★★ DAS KANO-MODELL (Klausuraufgabe 2, 6 Punkte — neuestes Format!)

„Erläutern Sie, welche Bedeutung das Kano-Modell in der PRIORISIERUNG von Anforderungen hat und wie Basis-, Leistungs- und Begeisterungsfaktoren voneinander unterschieden werden.“

 Rupp, Chris & die SOPHISTen: Requirements-Engineering und -Management, 5. Aufl., S. 82.

★★★★ Die drei Faktoren (Definitionen wörtlich!)

Faktor	Definition (wörtlich vom Dozenten)	Verhältnis Erfüllung ↔ Zufriedenheit	Beispiel Auto
BASISfaktor (Must-be, M)	„Eine Anforderung ist ein Basisfaktor, wenn das System diese Anforderung als SELBSTVERSTÄNDLICHKEIT aufweisen MUSS , um den MARKTEINTRITT/AKZEPTANZ zu ermöglichen.“	Erfüllung erzeugt KEINE Zufriedenheit — Nicht-Erfüllung erzeugt MASSIVE UNZUFRIEDENHEIT (Kurve unten links, sättigt bei „neutral“)	Bremse n, Lenkrad
LEISTUNG Sfaktor (One-dimensional, O)	„Eine Anforderung ist ein Leistungsfaktor, wenn die Kunden diese Anforderung im System BEWUSST FORDERN . Eine Steigerung der Leistungsmerkmale führt i. d. R. zu einer Steigerung der Kundenzufriedenheit .“	LINEAR: mehr Erfüllung = mehr Zufriedenheit (Diagonale)	Verbrauch, Motorleistung
BEGEISTERUNG Sfaktor (Attractive, A)	„Eine Anforderung ist ein Begeisterungsfaktor, wenn den Kunden diese Anforderung NICHT BEWUSST war bzw. sie die Umsetzung NICHT ERWARTET haben und die Kundenzufriedenheit ÜBERPROPORTIONAL WÄCHST , falls das System eine solche Anforderung umsetzt.“	Nicht-Erfüllung schadet NICHT — Erfüllung begeistert überproportional (Kurve oben rechts)	Massagesitz, Panoramadach



★★★ Bedeutung für die PRIORISIERUNG (das ist die eigentliche Klausurfrage!)

Das Kano-Modell liefert die Reihenfolge, in der Anforderungen umgesetzt werden müssen:

1. **Basisfaktoren zuerst und VOLLSTÄNDIG** — ohne sie ist das Produkt **nicht marktfähig**; jede Lücke erzeugt massive Unzufriedenheit, jede zusätzliche Investition darüber hinaus bringt aber **keinen Zufriedenheitsgewinn**
2. **Leistungsfaktoren als Nächstes und nach WIRTSCHAFTLICHKEIT dosiert** — hier zahlt sich jede zusätzliche Investition **linear** in Zufriedenheit aus; hier findet der **Wettbewerbsvergleich** statt
3. **Begeisterungsfaktoren gezielt und sparsam** — sie sind der **Differenzierer gegenüber dem Wettbewerb** und erzeugen **überproportionale Zufriedenheit bei geringem Umfang**. Sie dürfen aber **niemals zulasten der Basisfaktoren** umgesetzt werden

★ **Der dynamische Aspekt:** Begeisterungsfaktoren werden im Zeitablauf zu Leistungsfaktoren und schließlich zu Basisfaktoren (Beispiel: Klimaanlage im Auto, Kamera im Handy). Deshalb muss die Kano-Einstufung **regelmäßig überprüft** werden.

★★ Der Kano-Ablauf (4 Schritte)

Schritt	Inhalt
① Identifikation	Identifikation von Systemmerkmalen, die klassifiziert werden sollen
② Fragebogenentwurf	Entwurf eines Fragebogens, in dem für jedes Systemmerkmal erfragt wird, wie der Stakeholder empfindet, wenn (1) das Merkmal im System ENTHALTEN ist → FUNKTIONALE Frage (2) das Merkmal NICHT im System enthalten ist → DYSFUNKTIONALE Frage
③ Auswertung	Auswertung der Befragungsergebnisse und Ermittlung der Durchschnittswerte
④ Klassifizierung	Identifizierung der zugehörigen MERKMALSKLASSE (über die Auswertungsmatrix)

★★★★ DIE KANO-AUSWERTUNGSMATRIX (die 5 Antwortoptionen und die 6 Klassen)

Die 5 Antwortmöglichkeiten je Frage:

1. gewünscht · 2. muss · 3. interessiert nicht · 4. damit kann ich leben · 5. unerwünscht

funktional ↓ / dysfunktional →	1. gewünscht	2. muss	3. interessiert nicht	4. damit kann ich leben	5. unerwünscht
1. gewünscht	Q	A	A	A	O
2. muss	R	I	I	I	M
3. interessiert nicht	R	I	I	I	M
4. damit kann ich leben	R	I	I	I	M
5. unerwünscht	R	R	R	I	Q

Kürzel	Bedeutung
A	Attractive — BEGEISTERUNGSfaktor
O	One-dimensional — LEISTUNGSfaktor
M	Must-be — BASISfaktor
I	Indifferent — indifferente Antwort
Q	Questionable — fragliche Antwort (widersprüchlich → Frage prüfen)
R	Reverse — inkonsistente Antwort (Kunde will das Gegenteil)

★★ Das durchgerechnete Beispiel für FANF #005

Funktionale Frage: „Das CRM-System erzeugt automatisch die KundenNr“ → Antwort: **2. muss** X

Dysfunktionale Frage: „Das CRM-System kann die KundenNr nicht automatisch erzeugen“ → Antwort: **5. unerwünscht** X

Ablezen in der Matrix: Zeile „2. muss“ × Spalte „5. unerwünscht“ → **M**

✓ **Ergebnis:** FANF #005 ist ein **BASISFAKTOR (Must-be, M)** — wird so in die erweiterte Snowcard eingetragen.

▼ ? Selbsttest Teil J

1. Nennen und erläutern Sie die 3 Anforderungsarten mit je einem Beispiel. (8 Punkte!)
2. Wozu gehören Randbedingungen?
3. Nennen Sie die 3 Typen der SOPHIST-Satzschablone und formulieren Sie je ein Beispiel.
4. Welche 3 Schlüsselwörter gibt es und was bedeuten sie rechtlich?
5. Warum ist „wird“ verpflichtend?
6. Nennen Sie 8 Felder der Snowcard-Schablone.
7. Was ist der Unterschied zwischen „Kundenzufriedenheit“ und „Kundenunzufriedenheit“ in der Snowcard?
8. Definieren Sie Basis-, Leistungs- und Begeisterungsfaktor. (6 Punkte!)
9. Welche Bedeutung hat das Kano-Modell für die Priorisierung? Nennen Sie die Reihenfolge.
10. Was passiert mit Begeisterungsfaktoren im Zeitablauf?
11. Nennen Sie die 4 Schritte des Kano-Ablaufs.
12. Was sind funktionale und dysfunktionale Fragen?
13. Was bedeuten A, O, M, I, Q, R?
14. Ein Kunde antwortet funktional „gewünscht“ und dysfunktional „damit kann ich leben“. Welche Klasse? (A)

TEIL K — EXKURS: SPEZIFIKATIONSVORLAGEN

K.1 ★★★★★ LASTENHEFT vs. PFLICHTENHEFT (Klausurfrage 4× als SC!)

Zwei Spezifikationsdokumente entstehen im RE:

	LASTENHEFT	PFLICHTENHEFT
Inhalt	eine ZUSAMMENFASSUNG ALLER BENUTZERANFORDERUNGEN , die das System aus Sicht des Auftraggebers erfüllen muss	eine DETAILLIERTE DARSTELLUNG ALLER SYSTEMANFORDERUNGEN , die das System aus Sicht des Auftraggebers erfüllen muss
Verwendung	Kann als AUSSCHREIBUNGSUNTERLAGE dienen	Kann als VERTRAGSGRUNDLAGE dienen
Zeitpunkt	PROJEKTINITIIERUNGSPHASE	ANALYSEPHASE
Ebene	Benutzeranforderungen (Problemdomäne)	Systemanforderungen (Lösungsdomäne)
Wer erstellt es (typisch)?	Auftraggeber	Auftragnehmer (auf Basis des Lastenhefts)

⚠️ ★★★ DIE SINGLE-CHOICE-FALLE (4× wörtlich identisch!)

„Was ist die korrekte Definition eines Lastenhefts?" **A. Im Lastenheft wird definiert WAS und WOFÜR zu lösen ist** ✓ B. Im Lastenheft wird definiert WAS WIE zu lösen ist

🔑 DIE MERKFORMEL:

- **LASTENHEFT: WAS und WOFÜR** (Anforderung + Zweck — Problemdomäne, ESSENTIELL!)
- **PFLICHTENHEFT: WAS und WIE** (Anforderung + Umsetzung — Lösungsdomäne)

📝 „Nennen Sie die Aufgaben eines Lastenhefts" (Klausur 2020, 2 Punkte):

1. **Zusammenfassung aller Benutzeranforderungen** aus Sicht des Auftraggebers
2. **Grundlage für die Ausschreibung** — Basis, auf der Anbieter ein Angebot erstellen können
3. **Definition von WAS und WOFÜR** — also des fachlichen Bedarfs, **ohne** die technische Umsetzung vorwegzunehmen
4. **Herstellung eines gemeinsamen Verständnisses** zwischen Auftraggeber, Benutzern und potenziellen Auftragnehmern

K.2 ★ Die beiden Standard-Vorlagen

Standard	Bezeichnung	Datei im Kursmaterial
IEEE/ANSI 830-1998	<i>Recommended Practice for Software Requirements Specifications (SRS)</i>	IEEE-STD-830-1998.pdf , srs_template.doc
VOLERE	Volere Requirements Specification Template (Robertson & Robertson)	Volere_templateV901.doc

K.3 ★★ Aufbau eines Pflichtenhefts (analog IEEE/ANSI 830-1998)

Kapitel	Beschreibung
Deckblatt, Verzeichnisse	Deckblatt umfasst Titel, Datum, Version, Autoren und Kundenname . Wichtige Verzeichnisse: Inhalts-, Abbildungs-, Tabellen-, Abkürzungs- und Anhangverzeichnis
Vorwort	Legt die erwartete Leserschaft fest und beschreibt eine Versionsgeschichte mit Begründung für die Entwicklung einer neuen Version und Zusammenfassung der Veränderungen
Einleitung	Beschreibt die Notwendigkeit des Systems , legt kurz seine Funktionen dar und erklärt, wie es mit anderen Systemen zusammenarbeitet . Erläutert, wie das System mit den Unternehmenszielen übereinstimmt bzw. diese unterstützt
Begriffe	Legt die im Dokument verwendeten technischen Fachbegriffe fest. ⚠ Dabei dürfen keine Erfahrung oder Fachwissen beim Leser vorausgesetzt werden
Definition der BENUTZERanforderungen	Die für den Benutzer bereitgestellten Dienste und die nicht-funktionalen Anforderungen . Beschreibung in natürlicher Sprache, Diagrammen oder anderen für die Kunden verständlichen Notationen . Auch Produkt- und Entwicklungsstandards hier festlegen. ★ Die Anforderungen sind mit einer ID zu versehen
Systemarchitektur	Grober Überblick über die erwartete Systemarchitektur , der die Verteilung der Funktionen auf Systemmodule/-pakete zeigt
Spezifikation der SYSTEManforderungen	Detaillierte Beschreibung der funktionalen und nicht-funktionalen Anforderungen . ★ Die Beschreibungen sind mit einer ID zu versehen
Systemmodelle	Eines oder mehrere Systemmodelle , die die Beziehungen zwischen den Systemkomponenten und dem System und seiner Umgebung aufzeigen: Klassen-, Datenfluss- und semantische Datenmodelle (ERM)
Systementwicklung	Die grundlegenden Voraussetzungen , auf denen das System basiert, und erwartete Veränderungen aufgrund der Hardwareentwicklung, Veränderungen der Benutzeranforderungen etc.
Anhänge	Genauere spezifische Informationen: Hardware- und Datenbankbeschreibungen . HW-Anforderungen legen die minimale und optimale Konfiguration fest. DB-Anforderungen definieren die logische Verwaltung der Daten und deren Beziehungen
Index	Das Pflichtenheft kann mehrere Indizes enthalten

🔑 **Die Struktur folgt der Ebenenhierarchie aus J.1:** erst **Benutzeranforderungen** (Problemdomäne), dann **Systemarchitektur** und **Systemanforderungen** (Lösungsdomäne).

▼ ? Selbsttest Teil K

1. Lastenheft vs. Pflichtenheft: Inhalt, Verwendung, Zeitpunkt?
2. Wie lautet die Merkformel WAS/WOFÜR/WIE?
3. Nennen Sie die Aufgaben eines Lastenhefts.
4. Welche zwei Standard-Vorlagen gibt es?
5. Nennen Sie 8 Kapitel eines Pflichtenhefts nach IEEE 830.
6. Was muss in beiden Anforderungskapiteln zwingend vergeben werden? (IDs!)
7. Welche Systemmodelle nennt IEEE 830 explizit?

TEIL L — ALLE 5 ALTKLAUSUREN MIT MUSTERLÖSUNGEN ★★ ★

Die wichtigste Vorbereitung überhaupt. Rechnen Sie jede Klausur **zuerst ohne Lösung** durch!

L.1 ★★ ★ DER KOMPLETTE SINGLE-CHOICE-LÖSUNGSSCHLÜSSEL

⚠ **Erinnerung: falsche Antwort = -1 Punkt!**

Klausur WWIDS119 (2020)

#	Frage	✓ Antwort	Begründung
a	Nachbarsystem/Akteur	C. Akteur und Nachbarsystem können identisch sein	Verschiedene Begriffswelten (ERM vs. UC), überlappen sich, sind aber nicht deckungsgleich
b	Welche Aussage ist falsch ?	A. „Nach ‚prüfen‘, ‚validieren‘ etc. kommt immer genau EINE Alternative“	Es können mehrere oder gar keine sein — B (extend für Erweiterungen und Alternativen) ist richtig
c	Kein Grund für FISCUS-Scheitern	A. Personalmangel	Nicht die Menge, sondern die Qualifikation („disponibles statt gutes Personal“)
d	Projektmanagement bei Scrum	B. hilft dem Team	Scrum kennt keine klassische PL-Rolle
e	Kein Punkt des Agilen Manifests	B. „Team skills over financial incentives“	Frei erfunden
f	Keine Ereignisart	A. Personelle Ereignisse	Es gibt nur externe und zeitliche Ereignisse
g	ET in UC integrieren	A. Auf sie wird in der entsprechenden Aktivität verwiesen	„Den Kunden anhand ET #001 auf Bonität prüfen“
h	Extend-Beziehung möglich für	C. den Erweiterungs- UND den Alternativenteil	Im Diagramm wird beides über <<extend>> dargestellt
i	Coffee-Break-Test	A. den angemessenen Umfang eines PRODUCT USE CASES	Nicht einer Aktivität!
j	ERM modelliert man	B. nur bei Bedarf	Nur für nicht-triviale Ereignisse

Klausuren WWIDS120 (2021) und WWIDS121 (2022) — identischer Fragensatz!

#	Frage	✓ Antwort	Begründung
a	Ein Use-Case-Diagramm ist...	B. eine grafische Zusammenfassung ALLER Product Use Cases UND Akteure	Nicht ein einzelner UC
b	„Who has the ball“	B. Immer angeben, WER die Aktivität ausführt	R2 — nicht zu verwechseln mit R3 (Perspektive des Akteurs)
c	Coffee-Break-Test	B. Den sinnvollen Umfang eines PRODUCT USE CASES	
d	Welche Aussage zum auslösenden Datenfluss ist falsch ?	A. „Das Nachbarsystem, das ein Ereignis auslöst, muss auch immer dasjenige sein, das die Reaktion erhält“	Auslöser ≠ zwingend Empfänger
e	Keine Aufgabe des Scrum Masters	A. Die fachliche LEITUNG des Teams	SM unterstützt und moderiert, leitet nicht
f	Essenzbildung: „Abläufe OHNE fachlichen Hintergrund betrachten“	B. Die Aussage ist FALSCH	Genau die werden entfernt , nicht betrachtet
g	Korrekte Lastenheft-Definition	A. WAS und WOFÜR zu lösen ist	Pflichtenheft = WAS und WIE
h	Keine Ereignisart	A. Echte Ereignisse	Nur extern + zeitlich
i	Alternativen im PUC	C. Nichts davon — A und B sind beide falsch	A beschreibt die Erweiterung; B ist falsch, denn zu einer Alternative KANN es Erweiterungen geben
j	Beziehungsarten	B. „extend“ wird für Erweiterungen UND Alternativen benutzt	A beschreibt fälschlich extend als include

Klausur WWIDS222 (WKL, 2024)

#	Frage	✓ Antwort	Begründung
a	Arten/Prinzipien der Zerlegung	A. ereignisorientierte Zerlegung, funktionsorientierte Zerlegung	Es gibt keine „klassische“/„agile“ Zerlegung
b	„Who has the ball“	B. Immer angeben, wer die Aktivität ausführt	
c	Coffee-Break-Test	B. Den sinnvollen Umfang eines Product Use Cases	
d	Ist das zu modellierende System ein Akteur?	B. nein	★ „System Under Design ist kein Akteur“
e	Keine Aufgabe des Scrum Masters	B. Die ARBEITSPLANUNG des Teams	Selbstorganisation! SM moderiert Retrospektiven und unterstützt bei Problemen
f	Keine Dimension der Methodenauswahl	A. Innovationsgrad des Projekts	Die 5 Dimensionen: Personal, Kritikalität, Dynamik, Teamgröße, Unternehmenskultur
g	Korrekte Lastenheft-Definition	A. WAS und WOFÜR zu lösen ist	
h	Muss es eine Nachbedingung Fehlschlag geben?	A. ja	Fester Bestandteil der Schablone (Punkt 5)
i	Sprint Retrospektive ist Maßnahme der...	B. KONSTRUKTIVEN Qualitätssicherung	Verbessert den Prozess (a priori), nicht das Produkt
j	Definition of Ready legt Kriterien fest für...	B. die ÜBERNAHME eines Features aus dem Product Backlog in das Sprint Backlog	DoD wäre die Fertigstellung eines Inkrements

L.2 ★★★★★ KLAUSURAUFGABE: „FORMALE FEHLER IM PRODUCT USE CASE“ (WDSx23, 15 Punkte)

Aufgabenstellung: „Leider hat er mehrere formale Fehler. Bitte identifizieren Sie FÜNF davon und korrigieren Sie diese Fehler. Geben Sie auch eine kurze BEGRÜNDUNG für Ihre jeweilige Korrektur an.“

Der fehlerhafte Use Case

Use Case: Bestellung abschließen
 Ziel: Der Kunde gibt eine Bestellung erfolgreich auf und erhält eine Bestätigung per E-Mail.
 Vorbedingung: Der Kunde ist authentifiziert und hat Produkte im Warenkorb.
 Zustand Erfolg: Die Bestellung ist erfolgreich abgeschlossen, und der Kunde erhält eine Bestätigung.
 Zustand Fehlschlag: keiner
 Akteure: Kunde, Zahlungssystem, Onlineshop
 Auslösendes Ereignis: Der Kunde klickt auf „Bestellung abschließen“.
 Normaler Ablauf:
 1. Der Kunde klickt auf „Bestellung abschließen“, um den Bestellvorgang zu starten.
 2. Der Kunde gibt Lieferadresse und Rechnungsadresse ein, um die Adressen im System zu speichern.
 3. Der Kunde wählt die Zahlungsart und gibt die Zahlungsinformationen zur Verarbeitung ein.
 4. Das Zahlungssystem verarbeitet die Zahlung und bestätigt diese an den Onlineshop.
 5. Der Onlineshop zeigt dem Kunden eine Bestellbestätigung an und schickt ihm die Bestellbestätigung als E-Mail über den E-Mailserver.
 Erweiterungen:
 4a. Der Kunde wird zur zusätzlichen Sicherheitsüberprüfung aufgefordert.
 Alternativen:
 5a. Die Verarbeitung im Zahlungssystem schlug fehl: Der Onlineshop bricht den Bestellprozess unter Angabe des Grundes ab.

✓ DIE FEHLER — DIE 5 STÄRKSTEN ZUERST

#	Fehler	Korrektur	Begründung
1 ★ ★ ★	Der Onlineshop ist als AKTEUR aufgeführt	Akteure: Kunde, Zahlungssystem, E-Mailserver — Onlineshop streichen	Der Onlineshop ist das zu modellierende SYSTEM (System Under Design). Akteure befinden sich STETS AUSSERHALB des Systems — das System selbst kann kein Akteur seiner eigenen Use Cases sein
2 ★ ★ ★	Zustand Fehlschlag: keiner	Nachbedingung Fehlschlag: Die Bestellung wurde nicht abgeschlossen; dem Kunden wurde der Grund für den Abbruch mitgeteilt.	Es GIBT einen Fehlschlag! Die Alternative 5a beschreibt ihn explizit („Der Onlineshop bricht den Bestellprozess unter Angabe des Grundes ab“). Die Nachbedingung Fehlschlag ist Pflichtbestandteil der Schablone (Punkt 5) und muss den Zustand beschreiben, wenn das Ziel NICHT erreicht wird
3 ★ ★	Der E-Mailserver fehlt bei den Akteuren	Akteure: ... , E-Mailserver ergänzen	In Aktivität 5 wird die Bestätigung über den E-Mailserver verschickt. Ein externes System, mit dem das zu modellierende System kommuniziert, ist ein Akteur und muss aufgeführt werden
4 ★ ★	Falsche Feldbezeichnungen: Zustand Erfolg / Zustand Fehlschlag / Normaler Ablauf	Nachbedingung Erfolg: / Nachbedingung Fehlschlag: / Beschreibung:	Die Use-Case-Schablone gibt die Feldbezeichnungen VOR (Punkte 4, 5, 8). Abweichende Bezeichnungen verletzen die syntaktische Richtigkeit und erschweren die Vergleichbarkeit von Use Cases (GoM)
5 ★ ★	Aktivität 1 ist mit dem auslösenden Ereignis identisch (redundant); zudem ist der Klick ein Implementierungsdetail (nicht essentiell)	Auslösendes Ereignis: Der Kunde will seine Bestellung abschließen. Aktivität 1 streichen , der Ablauf beginnt mit der Adresseingabe	Das auslösende Ereignis initiiert den Use Case — es darf nicht zusätzlich als erste Aktivität erscheinen. Ein „Klick auf einen Button“ ist eine GUI-Implementierungsentscheidung und damit nicht essentiell

+ Weitere Fehler (Reserve — falls einer der obigen nicht anerkannt wird)

#	Fehler	Korrektur	Begründung
6	Die UC-Nummer fehlt	Use Case: PUC #001 Bestellung abschließen	Jeder Use Case braucht eine eindeutige ID für Referenzierung und Verfolgbarkeit (siehe Snowcard „UC #“)
7	Ziel und Nachbedingung Erfolg sind vermischt/redundant	Ziel: Die Bestellung des Kunden ist aufgegeben.	Das Ziel ist die globale Zielsetzung , die Nachbedingung Erfolg der konkrete Zustand danach . Das Ziel beschreibt hier fälschlich bereits einen Ablauf („gibt auf und erhält“)
8	Erweiterung 4a ist im PASSIV formuliert — „Der Kunde wird aufgefordert“	4a. Das Zahlungssystem fordert vom Kunden eine zusätzliche Sicherheitsüberprüfung an; nach erfolgreicher Überprüfung wird mit Aktivität 5 fortgefahren.	Verletzt R2 „Who has the ball“ (es ist unklar, WER handelt) und R1 (einfache Grammatik: Subjekt-Verb-Objekt) . Zudem fehlt die Rückkehr zum Happy Pass , die eine Erweiterung definitionsgemäß hat
9	Alternative 5a ist der falschen Aktivität zugeordnet	4a. Die Verarbeitung im Zahlungssystem schlug fehl: ...	Der Fehlschlag tritt in Aktivität 4 auf (dort verarbeitet das Zahlungssystem die Zahlung), nicht in 5
10	Aktivität 5 enthält ZWEI Aktivitäten (Bestätigung anzeigen + E-Mail versenden)	Aufteilen in 5. und 6.	Eine Aktivität soll genau EINEN signifikanten Zwischenerfolg liefern (R4)
11	Aktivität 2 nennt den Grund als Implementierungsdetail („um die Adressen im System zu speichern“)	2. Der Kunde gibt Liefer- und Rechnungsadresse für die Zustellung und Fakturierung der Bestellung an.	Die Absicht des Akteurs soll erkennbar sein (R5) — „im System speichern“ ist die Systemsicht , nicht die Akteursabsicht

 **Klausur-Strategie:** Schreiben Sie zu **jedem Fehler drei Zeilen: (1) Was ist falsch, (2) wie lautet die Korrektur, (3) warum**. Die Aufgabe verlangt ausdrücklich eine Begründung — **ohne Begründung gibt es maximal die Hälfte der Punkte**.

L.3 ★★☆☆ MUSTERLÖSUNG: PUC #001 „Ticket im First-Level-Support bearbeiten“ (15 Punkte, 3× in Klausuren!)

Der Aufgabentext (gekürzt)

Die **All-for-You AG** ist ein IT-Dienstleister, der das **Outsourcing von ERP-Systemen als Software-as-a-Service** großen Unternehmenskunden anbietet. Für den Support wird ein **neues Ticketsystem** eingeführt.

„Damit ein Mitarbeiter ein Ticket im First-Level-Support bearbeiten kann, muss **durch den Kunden im Ticketsystem ein Ticket mit niedriger Priorität angelegt** und anschließend **automatisch durch das Ticketsystem einem Mitarbeiter zugewiesen** worden sein.

Ist das passiert: Der Mitarbeiter **schaut sich das Ticket** im Ticketsystem an und **informiert sich über das vom Kunden beschriebene Problem**. Dann **sucht** der Mitarbeiter nach gleichen/ähnlichen Problemen **in der Wissensdatenbank**. Wenn **keine Lösung** in der Wissensdatenbank dokumentiert ist, dann **schiebt der Mitarbeiter das Ticket zum Second-Level-Support** weiter. Damit wäre **der Fall für ihn erledigt**. **Hat er aber Erfolg** und findet einen Lösungsvorschlag, dann **nimmt er Kontakt mit dem Kunden auf**, um ihm den Lösungsvorschlag mitzuteilen. Der **Kunde testet den Lösungsvorschlag** und **wenn alles klappt, schließt der Mitarbeiter das Ticket**. Sollte das Problem nicht gelöst werden können, dann **schiebt der Mitarbeiter das Ticket zum Second-Level-Support**.“

✓ MUSTERLÖSUNG (a) Product Use Case

1. Use Case: PUC #001 Ticket im First-Level-Support bearbeiten
2. Ziel: Das vom Kunden gemeldete Problem ist gelöst und das Ticket geschlossen.
3. Vorbedingung: Der Kunde hat im Ticketsystem ein Ticket mit niedriger Priorität angelegt und das Ticketsystem hat dieses Ticket automatisch einem Mitarbeiter des First-Level-Supports zugewiesen.
4. Nachbedingung Erfolg: Der Lösungsvorschlag wurde vom Kunden erfolgreich getestet und das Ticket ist im Ticketsystem geschlossen.
5. Nachbedingung Fehlschlag: Das Ticket wurde an den Second-Level-Support weitergegeben; der First-Level-Support ist für dieses Ticket nicht mehr zuständig.
6. Akteure: Mitarbeiter (First-Level-Support), Kunde, Wissensdatenbank, Second-Level-Support
7. Auslösendes Ereignis: Dem Mitarbeiter wurde ein Ticket zur Bearbeitung zugewiesen.
8. Beschreibung:
 1. Der Mitarbeiter informiert sich im Ticketsystem über das vom Kunden beschriebene Problem.
 2. Der Mitarbeiter recherchiert in der Wissensdatenbank nach gleichen oder ähnlichen Problemen und findet einen Lösungsvorschlag.
 3. Der Mitarbeiter teilt dem Kunden den Lösungsvorschlag mit.
 4. Der Kunde testet den Lösungsvorschlag und meldet dem Mitarbeiter das erfolgreiche Ergebnis zurück.
 5. Der Mitarbeiter schließt das Ticket im Ticketsystem.

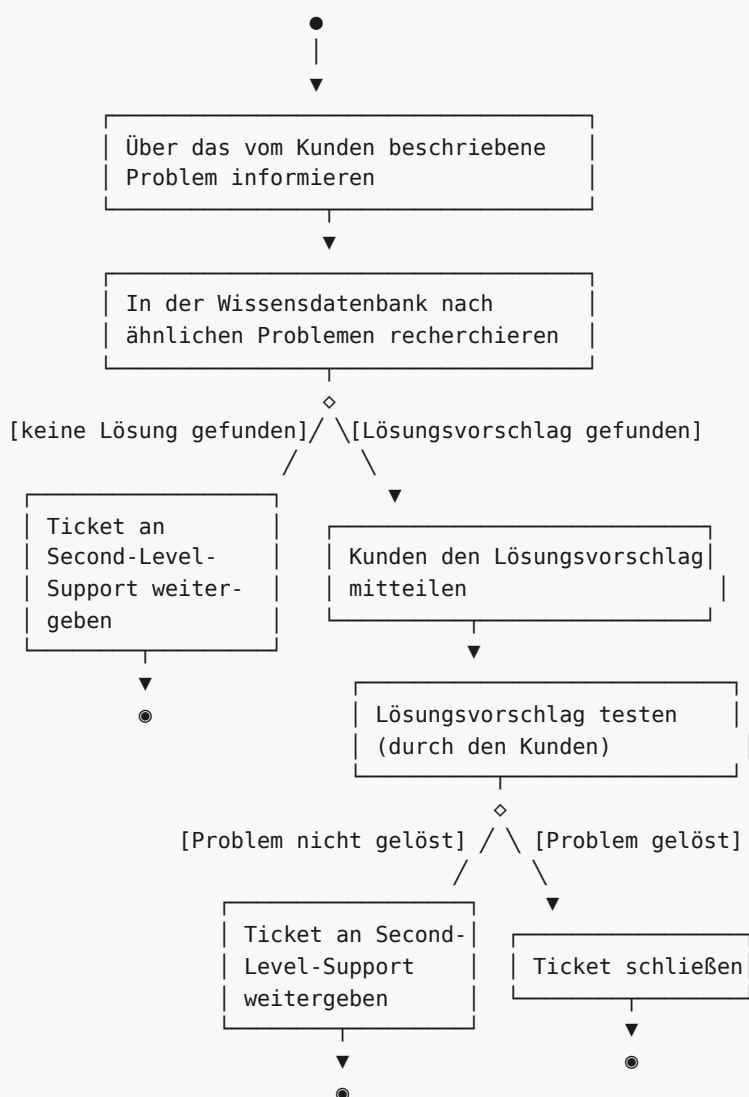
Alternativen:

 - 2a. In der Wissensdatenbank ist keine Lösung für das Problem dokumentiert: Der Mitarbeiter gibt das Ticket an den Second-Level-Support weiter. UC beenden.
 - 4a. Das Problem des Kunden konnte mit dem Lösungsvorschlag nicht gelöst werden: Der Mitarbeiter gibt das Ticket an den Second-Level-Support weiter. UC beenden.

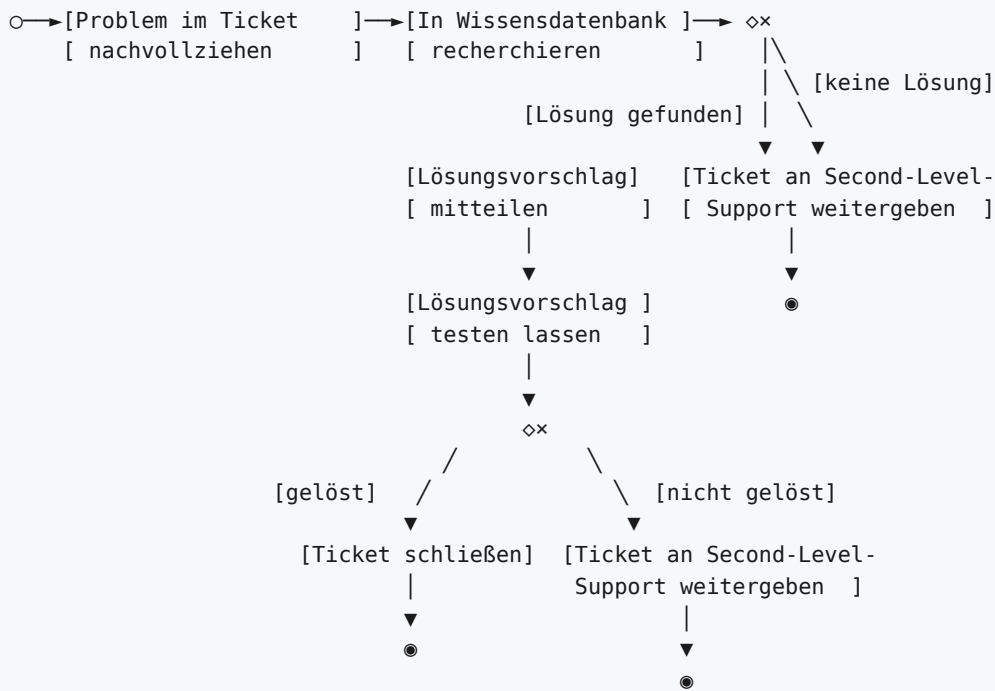
⚠️★★★★ DIE 5 FALLEN DIESER AUFGABE (hier werden die meisten Punkte verschenkt!)

Fälle	Warum
① Das TICKETSYSTEM als Akteur aufführen ✗	Das Ticketsystem ist das zu modellierende System („Für den Support wird ein neues Ticketsystem eingeführt . Sie sollen dafür die Product Use Cases modellieren “). System Under Design ≠ Akteur!
② Das Anlegen und Zuweisen des Tickets als Aktivität 1/2 modellieren ✗	Das steht im Text als „ muss ... worden sein “ → das ist die VORBEDINGUNG , nicht Teil des Ablaufs!
③ Nachbedingung Fehlschlag vergessen oder „keiner“ schreiben ✗	Der Text nennt zwei Fehlschlagpfade (keine Lösung gefunden / Lösung funktioniert nicht) — beide enden mit der Weitergabe an den Second-Level-Support
④ Die beiden Weitergaben als ERWEITERUNGEN modellieren ✗	Sie kehren NICHT zum Happy Pass zurück , sondern beenden den Use Case → es sind ALTERNATIVEN
⑤ Die Wissensdatenbank und den Second-Level-Support als Akteure vergessen ✗	Beide sind Nachbarsysteme/Organisationseinheiten außerhalb des Ticketsystems, mit denen kommuniziert wird

✅ MUSTERLÖSUNG (b) — Aktivitätsdiagramm



✓ MUSTERLÖSUNG (b') — BPMN-Modell (ohne Swimlanes!)



Legende: ○ Start-Event · □ Task · ◇× exklusives Gateway (XOR) · ● End-Event

⚠ Denken Sie an Lesson Learned 2: Nach Erweiterungen immer ein Gateway zum Zusammenführen — hier nicht nötig, weil beide Zweige Alternativen sind und in End-Events enden. ⚠ Lesson Learned 1: Kreuzungen vermeiden — die Zweige sauber nach außen führen.

L.4 ★★★★★ MUSTERLÖSUNG: PUC 001 „Neues Mitglied registrieren“ (Fitnessstudio, 15 Punkte, 2×!)

Der Aufgabentext (gekürzt)

Für ein Fitnessstudio wird ein neues **Mitgliederverwaltungssystem** entwickelt. Eine Funktion soll die **Registrierung neuer Mitglieder durch einen Mitarbeiter** ermöglichen.

„Kommt jemand zu uns neu in das Fitnessstudio, dann müssen wir diese Person zuerst im System als neues Mitglied registrieren. Dazu **gibt der Mitarbeiter die Registrierungsdaten** des neuen Mitgliedes in das Mitgliederverwaltungssystem ein. Das **System prüft dann die Daten auf formale Plausibilität**, also ob alle erforderlichen Angaben gemacht wurden. Danach **klärt der Mitarbeiter mit dem neuen Mitglied, ob er Basis- oder Premiummitglied werden will und wählt den entsprechenden Mitgliedsvertrag im System aus**. Dieser **Vertrag wird zweimal ausgedruckt und vom Kunden unterschrieben**. Eine Kopie bekommt das neue Mitglied, eine bekommen wir. **Dies wird im System vermerkt**, damit wir wissen, dass der Vertrag unterschrieben und eine Version ausgehändigt wurde. **Unterschreibt die Person nicht**, weil sie mit dem Vertrag nicht einverstanden ist, dann **wird die Registrierung abgebrochen**. Ansonsten **beschreibt das Mitgliederverwaltungssystem die Mitgliedskarte** des neuen Mitglieds mit dessen Daten und der **Mitarbeiter händigt diese Karte dem Mitglied aus**.“

✓ MUSTERLÖSUNG (a) Product Use Case

1. Use Case: PUC #001 Neues Mitglied registrieren
2. Ziel: Ein neues Mitglied ist registriert und einsatzbereit ausgestattet.
3. Vorbedingung: Eine Person möchte neu Mitglied im Fitnessstudio werden und ihre Registrierungsdaten liegen vor.
4. Nachbedingung Erfolg: Das neue Mitglied ist mit einem unterschriebenen Mitgliedsvertrag im Mitgliederverwaltungssystem registriert, der Vertragsstatus ist vermerkt und die beschriebene Mitgliedskarte wurde ausgehändigt.
5. Nachbedingung Fehlschlag: Die Registrierung wurde abgebrochen; es wurde kein Mitgliedsdatensatz aktiviert und keine Mitgliedskarte ausgehändigt.
6. Akteure: Mitarbeiter des Fitnessstudios, neues Mitglied
7. Auslösendes Ereignis: Eine Person möchte neu Mitglied im Fitnessstudio werden.
8. Beschreibung:
 1. Der Mitarbeiter gibt die Registrierungsdaten des neuen Mitglieds in das Mitgliederverwaltungssystem ein.
 2. Das Mitgliederverwaltungssystem validiert die Registrierungsdaten auf formale Plausibilität (Vollständigkeit aller erforderlichen Angaben).
 3. Der Mitarbeiter wählt gemeinsam mit dem neuen Mitglied den gewünschten Mitgliedsvertrag (Basis oder Premium) im Mitgliederverwaltungssystem aus.
 4. Das Mitgliederverwaltungssystem erzeugt den Mitgliedsvertrag in zweifacher Ausfertigung.
 5. Das neue Mitglied unterschreibt beide Vertragsexemplare und erhält ein Exemplar ausgehändigt.
 6. Der Mitarbeiter vermerkt im Mitgliederverwaltungssystem, dass der Vertrag unterschrieben und ein Exemplar ausgehändigt wurde.
 7. Das Mitgliederverwaltungssystem beschreibt die Mitgliedskarte mit den Daten des neuen Mitglieds.
 8. Der Mitarbeiter händigt dem neuen Mitglied die Mitgliedskarte aus.

Erweiterungen:

 - 2a. Die Validierung war nicht erfolgreich: Der Mitarbeiter ergänzt bzw. korrigiert die fehlenden Angaben; weiter mit Aktivität 2.

Alternativen:

 - 5a. Das neue Mitglied ist mit dem Vertrag nicht einverstanden und unterschreibt nicht: Der Mitarbeiter bricht die Registrierung ab. UC beenden.

⚠ Die Fallen dieser Aufgabe

Falle	Erklärung
Das Mitgliederverwaltungssystem als Akteur listen ✗	Es ist das zu modellierende System
2a als Alternative statt Erweiterung ✗	Die Korrektur kehrt zum Happy Pass zurück (weiter mit 2) → ERWEITERUNG
5a als Erweiterung statt Alternative ✗	Der Abbruch endet in einem Endzustand → ALTERNATIVE
Zu viele Aktivitäten ⚠	8 Aktivitäten sind noch im Rahmen (Daumenregel: max. 9–10)
„prüft“ statt „validiert“ ✗	R7 : validieren statt prüfen

L.5 DIE OFFENEN WISSENSFRAGEN ALLER KLAUSUREN — KURZANTWORTEN

Klausur	Frage	Antwort (Kurzform)
2020 (a)	Was versteht man unter „Essenz“?	→ Teil E.7
2020 (b)	Zweck eines Ereignisreaktionsmodells?	→ Teil E.7
2020 (c)	Analytische und konstruktive QS bei Scrum?	→ Teil C.6
2020 (d)	Klassisches V-Modell erläutern	→ Teil C.5
2020 (e)	Unterschied evolutionär ↔ inkrementell	→ Teil C.9
2020 (f)	Die 3 Arten von Prototypen	→ Teil C.7
2020 (g)	Aufgaben eines Lastenhefts	→ Teil K.1
2020 (h)	4 Bedingungen, unter denen Agile ungeeignet ist	→ Teil D.11
2021 (a)	Systemverständnis Geschäftsprozessmodell vs. PUC	→ Teil G.3
2021 (b)	Grund, warum FISCUS beim zweiten Anlauf scheiterte	→ Teil A.5
2021 (c)	Arten von Akteuren eines PUC	→ Teil G.2
2021 (d)	Komponenten von Scrum (10 P.!)	→ Teil D.8
2022 (a)	Ist „Kontaktaufnahme über MS Teams“ essentiell?	→ Teil E.7
2022 (b)	3 Hauptpunkte für Scheitern (CHAOS-Studie)	→ Teil A.4
2022 (c)	3 der 5 Voraussetzungen für agile Methoden	→ Teil D.11
2022 (d)	Was versteht man unter einem Akteur?	→ Teil G.2
2022 (e)	Unterschied inkrementell ↔ evolutionär	→ Teil C.9
2022 (f)	Die Punkte des Agilen Manifests	→ Teil D.4
2024 (a)	Was ist eine essentielle Anforderung?	→ Teil E.7
2024 (b)	3 Arten von Prototypen (explorativ, experimentell, evolutionär)	→ Teil C.7
2024 (c)	Warum ist klassische SW-Entwicklung schwergewichtig?	→ Teil D.2
2024 (d)	INVEST erläutern	→ Teil D.12
2024 (e)	Das agile Manifest erläutern (8 P.!)	→ Teil D.4
2024/12 (a)	Ist „MS Teams“ essentiell?	→ Teil E.7
2024/12 (b)	2 Gründe für das Scheitern von FISCUS	→ Teil A.5
2024/12 (c)	Arten von Akteuren eines PUC	→ Teil G.2
2024/12 (d)	INVEST (6 P.)	→ Teil D.12
2024/12 (e)	Was tut ein Area Product Owner?	→ Teil D.9
2024/12 (f)	Sprint Retrospektive: konstruktiv oder analytisch?	→ Teil C.6
2024/12 (g)	Arten von Ereignissen im RE	→ Teil E.5
2024/12 (h)	Schablone für eine User Story	→ Teil D.12
2024/12 (i)	Definition of Done und Definition of Ready	→ Teil D.12
2024/12 (j)	3 Dimensionen klassisch vs. agil	→ Teil D.11
2024/12 (2)	Kano-Modell und Priorisierung	→ Teil J.6
2024/12 (3)	Arten von Anforderungen + Beispiele	→ Teil J.2

TEIL M — DIE FALLSTUDIE AIM (All-Inclusive-Maschine)

Die aktuelle Übungs-Fallstudie des Kurses — die Arbeitsaufträge bilden **exakt den Klausurablauf** ab!

M.1 Die Vision

„Unsere Bank bietet ihren Kunden einen intuitiven SINGLE-POINT-OF-ACCESS zu allen zentralen Finanzdienstleistungen — schnell, sicher und jederzeit verfügbar. AIM – die All-Inclusive-Maschine für modernes Banking.“

M.2 ★ Die 7 User Stories des Product Backlog (perfekte Schablonen-Beispiele!)

#	Titel	User Story
1	Bargeldauszahlung	Als Bankkunde möchte ich an der AIM Bargeld abheben können , damit ich jederzeit auf mein Guthaben zugreifen kann .
2	Bargeldeinzahlung	Als Bankkunde möchte ich an der AIM Bargeld einzahlen können , damit das Geld sofort auf meinem Konto verfügbar ist .
3	Kontostandabfrage	Als Kunde einer beliebigen Bank möchte ich meinen aktuellen Kontostand an der AIM einsehen können , um meine Finanzen schnell zu überprüfen .
4	Überweisungen / Terminüberweisungen	Als Bankkunde möchte ich Überweisungen an der AIM durchführen oder terminieren , damit ich Zahlungen flexibel und unabhängig von Öffnungszeiten erledigen kann .
5	PIN-Änderung	Als Bankkunde möchte ich meine PIN an der AIM ändern können , um die Sicherheit meines Kontos zu erhöhen .
6	Mini-Kontoauszug drucken	Als Bankkunde möchte ich einen Mini-Kontoauszug an der AIM ausdrucken , damit ich meine letzten Transaktionen direkt überprüfen kann .
7	Fremdbank-Kunde / Interbank-Funktion	Als Kunde einer anderen Bank möchte ich an der AIM Bargeld mit meiner Karte abheben können , damit ich nicht auf einen Geldautomaten meiner Hausbank angewiesen bin .

🔑 **Beachten Sie:** Jede Story hat exakt die Struktur **Als <Rolle> möchte ich <Aktion>, damit/um <Nutzen>** — und die **Rolle unterscheidet** zwischen „Bankkunde“ (eigene Bank) und „Kunde einer beliebigen/anderen Bank“ (Fremdbank)!

M.3 ★★ Die 4 Arbeitsaufträge = der Klausurablauf

#	Arbeitsauftrag	Entspricht Klausuraufgabe
#0 1	Erstellen Sie für Ihre User Story den Use Case	Aufgabe 7a / 4a (15 Punkte)
#0 2	Erstellen Sie für Ihren UC das Aktivitätsdiagramm UND das BPMN-Modell	Aufgabe 7b / 4b (5 Punkte)
#0 3	Erstellen Sie die konsolidierte Entscheidungstabelle ET #001 „Geld einzahlen“	Aufgabe 4 / 3 (10 Punkte)
#0 4	Übertragen Sie Ihren Use Case in präzise, überprüfbare Anforderungssätze nach der SOPHIST-Satzschablone	Aufgabe 3 (Anforderungsarten)

☺ „P.S.: Es gibt keine schlechten Lösungen und niemand kann sich blamieren.“

M.4 ★ Die „AHA-Momente“ der Systemanalyse (aus dem Follow-Up-Beitrag)

Vier Erkenntnisse, die Studierende aus dem Projekt mitgenommen haben:

1. Präzisere Identifikation der ECHTEN Akteure
2. „System Under Design“ ist KEIN Akteur ★★★
3. Klare und eindeutige Formulierungen notwendig
4. Unpräzise Anforderungen führen schnell zu Missverständnissen

M.5 ★ Die KALM-Projektretrospektive (Retrospektiven-Methode!)

KALM = Keep · Add · More · Less — eine Retrospektiven-Technik

	Inhalt
KEEP (beibehalten)	Feste Scrum-Master-Rolle für Kommunikation mit dem Dozenten · Zentrales gemeinsames Dokument zur Ablage aller Ergebnisse
ADD (hinzufügen)	Kurze regelmäßige Updates (z. B. Mini-Meetings) · Klare Kommunikationsstruktur für Änderungen und neue Infos
MORE (mehr davon)	Gemeinsame Durchsicht der Use Cases vor dem Verschicken · Abstimmung und Absprache innerhalb der Gruppe
LESS (weniger davon)	Unabgestimmtes Arbeiten an einzelnen Use Cases · Spontane, unstrukturierte Änderungen ohne Rückmeldung

TEIL N — SELBSTTESTFRAGEN (120 Fragen)

N.1 Projektscheitern & Softwaretechnik (1–20)

#	Frage	Antwort
1	3 Charakteristika eines Black-Swan-Ereignisses?	Unvorhersehbarkeit, massive Auswirkungen, seltenes Auftreten
2	Schaden bei Levi Strauss?	USD 192,5 Mio.; CIO David Bergen trat zurück
3	Umfang der Capers-Jones-Studie?	8.300 Projekte, 1991–2008
4	Lebensdauer einer Programmiersprache?	15 Jahre
5	Seit wann läuft die CHAOS-Studie, wie viele Projekte?	Seit 1994, über 40.000 Projekte
6 ★	3 Hauptgründe für Projektscheitern (CHAOS)?	Unvollständige Anforderungen, fehlende Nutzereinbindung, fehlende Management-Unterstützung
7	Ziel von FISCUS?	Bundeseinheitliches IT-System für die Steuererhebung
8 ★	Schaden von FISCUS?	ca. 5 Mrd. € , viermal neu aufgesetzt
9 ★	2 Gründe für das Scheitern von FISCUS?	Dezentrale Projektorganisation mit >10 Standorten & eigenen Projektleitungen; politisch besetzte schwache Koordinationsstelle
10	Zitat zum Personal bei FISCUS?	„Es wurde disponibles statt gutes Personal abgestellt“
11	Warum scheiterte Phase 2?	Objektorientierung: Request-Broker-Middleware enttäuschte, zu lange Rechen-/Antwortzeiten
12 ★	Definition Software-Technik?	Zielorientierte Bereitstellung und systematische Verwendung von Prinzipien, Methoden und Werkzeugen für die arbeitsteilige, ingenieurmäßige Entwicklung und Anwendung von umfangreichen Softwaresystemen
13	Das magische Dreieck?	Kosten – Zeit – Qualität
14 ★	Die 4 Prinzipien?	Abstraktion, Strukturierung, Hierarchisierung, Modularisierung
15 ★	Aufgabe von Zerlegungsstrategien?	Komplexität in kleine, handhabbare, stabile Teile zerlegen
16	3 Kriterien für „stabil“?	Einfach strukturierte Schnittstellen, nur lokale Änderungsauswirkungen (keine Seiteneffekte), pragmatische nachvollziehbare Zerlegung
17 ★	Die 6 Komplexitätsarten?	Funktionen, Daten, Algorithmen, zeitabhängiges Verhalten, Systemumgebung, GUI
18	Zu welcher Komplexität gehört die ET?	Komplexität der Algorithmen
19 ★	Was ist die semantische Lücke?	Der Bruch zwischen den Modellen zweier Phasen — SA eignet sich für Analyse, nicht für Entwurf

#	Frage	Antwort
20 ★	Korrekte Zerlegungsarten?	Ereignisorientierte und funktionsorientierte Zerlegung

N.2 Prozessmodelle (21–45)

#	Frage	Antwort
21	6 Inhalte eines Prozessmodells?	Reihenfolge, Aktivitäten, Teilprodukte, Fertigstellungskriterien, Mitarbeiterqualifikationen, Verantwortlichkeiten
22	WHISCY?	„Why isn't Sam coding yet?“
23 ★	3 Nachteile von code&fix → 3 Phasen?	Umstrukturierung teuer → Entwurfsphase ; Anwenderanforderungen zu kurz → Analysephase ; Fehler schwer zu finden → Testphase
24 ★	6 Merkmale des Wasserfallmodells?	Vollständig, dokumentengetrieben, sequentiell, top-down, einfach/wenig Managementaufwand, Anwenderbeteiligung nur in der Definitionsphase
25	4 Nachteile des Wasserfallmodells?	Nicht immer vollständig/sequentiell sinnvoll; Dokumentation wichtiger als Produkt; Risiken werden nicht erkannt
26 ★	Was ist der Beitrag des Wasserfallmodells trotz Nachteilen?	Plan-, steuer- und kontrollierbare Softwareentwicklung (Projektmanagement)
27 ★	Verifikation — Leitfrage?	„Wird ein korrektes Produkt entwickelt?“
28 ★	Validation — Leitfrage?	„Wird das richtige Produkt entwickelt?“
29 ★	Was ist bei der Validation zwingend?	Anwenderbeteiligung
30 ★	Analytische QS?	Diagnostische Maßnahmen zur Messung des Qualitätsniveaus → v. a. Produktqualität
31 ★	Konstruktive QS?	Methoden, Sprachen, Werkzeuge, Richtlinien, Standards, Checklisten, die a priori Eigenschaften sicherstellen → v. a. Prozessqualität
32	Analysierende vs. testende Maßnahmen?	Analysierend: Verifikation, Review, Walk Through, Inspektion · Testend: dynamischer Test, Simulation, Schreibtischtest
33	Technische vs. organisatorische konstruktive Maßnahmen?	Technisch: Methoden, Programmiersprachen, Werkzeuge · Organisatorisch: Richtlinien, Standards, Checklisten
34 ★ ★	Sprint Retrospektive: analytisch oder konstruktiv?	KONSTRUKTIV — verbessert den Prozess
35	Sprint Review?	ANALYTISCH — misst das Inkrement
36 ★	4 Probleme traditioneller Prozessmodelle?	Vollständige Spezifikation zu Projektbeginn; Kooperation nötig; alternative Lösungen zu diskutieren; Realisierbarkeit nur praktisch erprobbar
37 ★ ★	3 Arten von Prototypen?	Experimentell (Labormuster), explorativ (Demonstrationsprototyp), evolutionär (Pilotsystem)
38	Motto des Prototypenmodells?	„ Redo until Right “
39 ★	Horizontaler vs. vertikaler Prototyp?	Horizontal = eine Schicht über die volle Breite · Vertikal = eine Funktion durch alle Schichten
40 ★	Die 5 Schritte des Kreislaufs des Verstehens?	Anwender erzählt → Analytiker notiert in eigener Darstellung → übersetzt in Anwendersprache → zeigt es dem Anwender (Überraschung!) → Änderungen, zurück zu Schritt 2
41	Das Anwender-Zitat?	„I can't tell you what I want, but I'll know it when I see it!“

#	Frage	Antwort
42 ★	Was ist die Nullversion?	Das nur aus den Kern-/Muss-Anforderungen bestehende Kernsystem im evolutionären Modell
43 ★ ★	Unterschied evolutionär ↔ inkrementell?	Evolutionär: partielles Modell (nur Kern), Erweiterungen aus Erfahrung · Inkrementell: vollständiges Modell zuerst, dann stückweise Umsetzung → Erweiterungen passen garantiert
44	2 Risiken des evolutionären Modells?	Kernanforderungen übersehen (Architektur muss überarbeitet werden); Nullversion nicht flexibel genug erweiterbar
45 ★ ★	Primäres Ziel je Modell?	Wasserfall: min. Managementaufwand · V-Modell: max. Qualität · Prototypen: Risikominimierung · Evolutionär: min. Entwicklungszeit · Inkrementell: min. Entwicklungszeit + Risikominimierung

N.3 Agile SE (46–70)

#	Frage	Antwort
46 ★	6 Vorwürfe an schwergewichtige Prozesse?	Rigider Formalismus; Dokumentenfixierung („Schrackware statt Software“); zu wenig Kommunikation; Management nimmt sich zu wichtig; Moving Targets nicht beherrschbar; Time to Market behindert
47 ★	Die 3 Merksätze aus Fazit 1?	Formalismus ≠ Disziplin · Prozess ≠ Fähigkeit · Dokumentation ≠ Verständnis
48 ★	Space-Shuttle-Rechnung?	1 LOC = 1 Personenmonat → 100–200 KLOC = 1–2 Mrd. €, 16 Jahre bei 1000 MA (ohne Brooks' Law)
49 ★	Effektivität/Effizienz-Formel?	Schwergewichtig = Effektivität · Leichtgewichtig = Effektivität + Effizienz = „ doing the right things right “
50 ★ ★	Die 4 Punkte des agilen Manifests?	Individuals and interactions > processes and tools · Working software > comprehensive documentation · Customer collaboration > contract negotiation · Responding to change > following a plan
51 ★	Was ist der wichtige Nachsatz?	Die rechten Werte haben auch Wert — die linken werden nur höher gewichtet
52	5 der 12 agilen Prinzipien?	Kundenzufriedenheit durch schnelle Auslieferung; Änderungen begrüßen; kurze Intervalle; tägliche Zusammenarbeit; Face-to-Face; lauffähige Software als Fortschrittsmaß; Selbstorganisation ...
53	Was bedeutet „Crystal“?	Nichts Besonderes — nur eine Metapher
54 ★	Die Crystal-Achsen?	Kritikalität L/E/D/C (Leben/essenzielles Geld/mäßiges Geld/Komfort) × Teamgröße 6/20/40/80 (Clear/Yellow/Orange/Red)
55 ★ ★	Die 3 Scrum-Rollen?	Product Owner, Scrum Master, Entwicklungsteam
56 ★ ★	Die 3 Scrum-Artefakte?	Product Backlog, Sprint Backlog, Inkrement
57 ★ ★	Die Scrum-Events?	Sprint Planning, Sprint (30 Tage), Daily Scrum (15 min), Sprint Review, Sprint Retrospektive
58 ★ ★ ★	Was ist KEINE SM-Aufgabe?	Die fachliche LEITUNG bzw. die ARBEITSPLANUNG des Teams
59 ★	Was tut ein Area Product Owner?	PO für einen Anforderungsbereich in LeSS; pflegt das Area Product Backlog; stimmt sich mit dem Gesamt-PO ab
60 ★ ★	Die 5 Dimensionen der Methodenauswahl?	Personal, Kritikalität, Dynamik, Teamgröße, Unternehmenskultur
61 ★	Welche ist KEINE Dimension?	Innovationsgrad des Projekts
62	Fähigkeitsebene 3 nach Cockburn?	Kann eine Methode anpassen und deren Regeln brechen , um mit unvorhergesehenen Situationen fertig zu werden
63 ★	Was ist ein Epic?	Große, übergeordnete Anforderung, aus der User Stories abgeleitet werden; nicht in einem Sprint umsetzbar
64 ★	User-Story-Schablone?	Als <Rolle> will ich <Aktion> sodass/weil <Nutzen>

#	Frage	Antwort
★		
65	Aufwand einer User Story?	1-5 Personentage (= eine Iteration)
66 ★ ★	INVEST?	Independent, Negotiable, Valuable, Estimatable, Small, Testable
67 ★ ★	DoD vs. DoR?	DoD: wann ist ein Item fertig (2 Sichten: Produkt + Prozess) · DoR: wann darf es in den Sprint
68 ★	SMART?	Spezifisch, Messbar, Akzeptiert, Realistisch, Terminiert
69 ★	4 Zonen des Agile-Fluency-Modells?	Focusing, Delivering, Optimizing, Strengthening
70	Was ist Fluency?	Eine Teameigenschaft — das Team kann immer die richtigen Personen zur richtigen Zeit einsetzen

N.4 Essenz, Ereignisse, ERM (71–85)

#	Frage	Antwort
71 ★	Wozu existieren Informationssysteme?	Um Dienstleistungen für Dritte anzubieten — als Reaktion auf ein Ereignis außerhalb des Systems
72 ★	Was produziert jedes Ereignis?	Einen eindeutigen Datenfluss — über dessen Inhalt identifiziert das System das Ereignis
73 ★	Was ist eine Reaktion?	Die Zusammenfassung ALLER Aktivitäten , die auf ein Ereignis reagieren
74 ★★	Wovon wird die Reaktion im ERM eingegrenzt?	Immer von Nachbarsystemen und Datenspeichern
75 ★★	Die 2 Ereignisarten?	Externe und zeitliche Ereignisse
76 ★	Namensregel extern?	Name des Nachbarsystems + Grund für den Datenfluss („Kunde gibt Kauforder“)
77 ★	Namensregel zeitlich?	„Zeit für“ + auszuführende Tätigkeit („Zeit für Börsenschluss“)
78	Was steht in der Ereignisliste?	Ereignisname + zugehörige Datenflüsse mit (I)nput/(O)utput
79 ★	Wann erstellt man ein ERM?	Zu jedem nicht-trivialen Ereignis — also nur bei Bedarf
80 ★	Muss der Auslöser die Reaktion erhalten?	NEIN
81 ★	Sind Datenspeicher Nachbarsysteme?	Nein — sie sind immer systemintern
82 ★★	Was ist Essenz?	Menge aller Anforderungen unabhängig von jeder technischen/organisatorischen Umsetzung — das wahre fachliche Problem (WAS und WOFÜR, nicht WIE)
83 ★★	Ist „über MS Teams“ essentiell?	NEIN — konkrete Produktwahl = Implementierungsentscheidung; essentiell wäre „Der Support muss mit dem Kunden Kontakt aufnehmen können“
84 ★	Was passiert bei der Essenzbildung mit nicht-fachlichen Abläufen?	Sie werden ENTFERNT (nicht betrachtet)
85	Können Akteur und Nachbarsystem identisch sein?	Ja, können sie — aber keins impliziert das andere

N.5 Erhebungstechniken (86–95)

#	Frage	Antwort
86 ★	Die Taxonomie?	Primär (Befragung: Interview/Fragebogen · Beobachtung: Feldbeobachtung/Apprenticing · Selbstaufschreibung) vs. Sekundär (Dokumentenanalyse)
87 ★	Welche Technik ist Sekundärerhebung?	Dokumentenanalyse
88	4 Befragungsmodi?	Schriftlich/postalisch, telefonisch, Face-to-Face, online
89 ★	Die 4 Interviewformen?	Gespräch · Interview i. e. S. · standardisiertes Interview mit offenen Fragen · standardisiertes Interview mit geschlossenen Fragen
90 ★	Welche Form hat den höchsten Vorbereitungsaufwand?	Standardisiertes Interview mit geschlossenen Fragen (Testläufe notwendig)
91 ★	Die 3 Interviewphasen?	Weiche Phase (Einleitung) · neutrale Phase (sachliche Erhebung) · weiche Phase (Ausklang)
92 ★	Maximale Interviewdauer?	1 Stunde
93 ★	Die 7 Fragetypen?	Sach-, Wissens-, Einstellungs-/Meinungs-, Verhaltens-, Eisbrecher-, Überleiter-, Kontrollfragen
94	Was sind Non-Opinions?	Pseudo-Meinungen, die erst im Moment der Fragestellung gebildet werden
95 ★	Welche Technik für objektive Mengengerüste?	Dokumentenanalyse

N.6 Use Cases, AD/BPMN, ET, Anforderungen (96–120)

#	Frage	Antwort
96 ★★	Was ist ein Akteur?	Eine ROLLE , die ein Benutzer einnimmt; hat Einfluss auf das System; befindet sich stets AUSSERHALB
97 ★★	3 Arten von Akteuren?	Person/Rolle, Organisationseinheit, externes System
98 ★★	Ist das System ein Akteur?	NEIN — „System Under Design ist kein Akteur“
99 ★★	Die 8 Punkte der UC-Schablone?	Use Case · Ziel · Vorbedingung · Nachbedingung Erfolg · Nachbedingung Fehlschlag · Akteure · Auslösendes Ereignis · Beschreibung (mit Erweiterungen/Alternativen)
100 ★	Muss es eine Nachbedingung Fehlschlag geben?	JA
101 ★★	Erweiterung vs. Alternative?	Erweiterung kehrt vor der nächsten Aktivität zum Happy Pass zurück · Alternative endet mindestens einmal in einem Endzustand oder umgeht Aktivitäten
102 ★	include vs. extend — Bedeutung?	include: A schließt immer B ein · extend: A kann , muss aber nicht durch B erweitert werden
103 ★	Vollständigkeit bei include/extend?	include: A unvollständig , B unvollständig möglich · extend: A vollständig , B vollständig
104 ★	Was ist ein UC-Diagramm?	Grafische Zusammenfassung ALLER PUC und Akteure des Systems
105 ★★	Coffee-Break-Test?	„After I get done with this, I can take a coffee break!“ → Umfang eines PRODUCT USE CASES (2–20 min, 1 Person, 1 Sitzung)
106 ★★	„Who has the ball“?	Immer angeben, WER die Aktivität ausführt (R2)
107 ★	Max. Anzahl Aktivitäten?	9–10
108 ★	Die 4 Schritte von R6?	Akteur sendet Daten → System überprüft → System reagiert → System gibt Ergebnis an Akteur
109 ★	Warum validieren statt prüfen?	„Prüfen“ erzwingt eine Folgeaktivität „Wenn korrekt, dann...“ im Hauptablauf
110 ★	4 logische Operatoren im AD?	Aufteilendes UND = Parallelisierungsknoten · Zusammenführendes UND = Synchronisationsknoten · Aufteilendes XOR = Entscheidungsknoten · Zusammenführendes XOR = Zusammenführungsknoten
111 ★	Zeichnerischer Unterschied Erweiterung/Alternative im AD?	Erweiterung hat einen Zusammenführungsknoten ; Alternative hat einen eigenen Endeknoten
112	Die 3 BPMN-Gateways?	XOR (×), AND (+), OR (○)
113 ★	Die 3 BPMN-Lessons-Learned?	Kreuzungen vermeiden · immer Gateways zum Zusammenführen · klare Unterscheidung Erweiterung/Alternative
114 ★	Die 6 GoM?	Richtigkeit, Relevanz, Wirtschaftlichkeit, Klarheit, Vergleichbarkeit, systematischer Aufbau
115 ★★	Anzahl Regeln einer vollständigen ET?	2ⁿ
116 ★★	Konsolidierungsregel?	Zwei Regeln zusammenfassen, wenn sie sich in genau einer Bedingung unterscheiden und identische Aktionen haben → „–“

#	Frage	Antwort
117 ★	Wie wird eine ET in einen UC integriert?	Es wird in der entsprechenden Aktivität darauf verwiesen
118 ★★	Die 3 Anforderungsarten?	FANF (welche Dienste), NFANF (wie — Qualität), RANF (Restriktionen; gehören zu den NFANF)
119 ★	Die 3 Schlüsselwörter?	muss (verpflichtend, Abnahme kann verweigert werden) · sollte (wünschenswert, nicht verpflichtend) · wird (zukünftig, aber verpflichtend)
120 ★★	Lastenheft vs. Pflichtenheft?	Lastenheft: WAS und WOFÜR , Benutzeranforderungen, Ausschreibungsunterlage, Projektinitiierung · Pflichtenheft: WAS und WIE , Systemanforderungen, Vertragsgrundlage, Analysephase

TEIL O — KLAUSUR-SPICKZETTEL

⚠ **KEINE HILFSMITTEL erlaubt** — alles muss im Kopf sein.

0.1 ★★ ★ DIE 6 DINGE, DIE SIE BLIND KÖNNEN MÜSSEN

① Die Use-Case-Schablone (8 Punkte)

1. Use Case: #NNN <2-3-Wort-Name>
2. Ziel:
3. Vorbedingung:
4. Nachbedingung Erfolg:
5. Nachbedingung Fehlschlag: ← NIE „keiner“!
6. Akteure: ← NIE das System selbst!
7. Auslösendes Ereignis:
8. Beschreibung: 1. ... 2. ... 3. ...
 Erweiterungen: 1a ... (kehrt zurück)
 Alternativen: 1a ... (endet / überspringt)

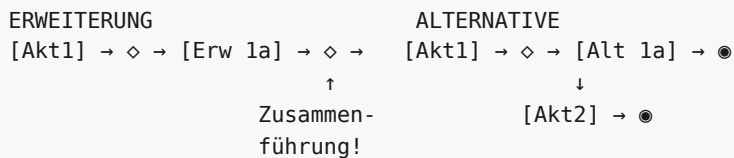
② Die 8 UC-Regeln

- R1 Einfache Grammatik: Subjekt – Verb – Objekt – Präpositionalphrase
- R2 „Who has the ball?“ → IMMER angeben, WER handelt (kein „er“, kein Passiv)
- R3 Aus der Perspektive des AKTEURS schreiben
- R4 Signifikanter Zwischenerfolg → max. 9-10 Aktivitäten
- R5 Die ABSICHT des Akteurs muss erkennbar sein
- R6 Ereignis-Reaktionsschema: senden → überprüfen → reagieren → ERGEBNIS zurück
- R7 VALIDIEREN statt prüfen
- R8 „Wiederhole Aktivität x-y <Endebedingung>“

③ Die Entscheidungstabelle

- 2ⁿ Regeln · Bedingungsteil (J/N/–) · Aktionsteil (X) · Regelteil (R1...Rk)
 Ausfüllschema: B1: JJJJNNNN B2: JJNNJJNN B3: JNJJNJJN
 KONSOLIDIEREN: genau EINE Bedingung unterschiedlich + IDENTISCHE Aktionen → „–“
 PRÜFEN: vollständig? widerspruchsfrei? redundanzfrei? maximal konsolidiert?

④ Die AD-Muster



⑤ Die Prozessmodell-Tabelle

Modell	Ziel	Fokus	Benutzer	Merkmale
Wasserfall	min. Managementaufwand	Dokumente	gering	sequentiell, volle Breite
V-Modell	max. Qualität	Dokumente	gering	sequentiell, volle Breite, QS
Prototypen	Risikominimierung	Code	hoch	nur Teilsysteme
Evolutionär	min. Entwicklungszeit	Code	mittel	sofort nur Kernsystem
Inkrementell	min. Zeit + Risiko	Code	mittel	volle Definition, dann Kern

⑥ Das agile Manifest

Individuals and interactions	over	processes and tools
Working software	over	comprehensive documentation
Customer collaboration	over	contract negotiation
Responding to change	over	following a plan

0.2 ★★ Die Merkformeln

Formel	Bedeutung
LASTENheft = WAS + WOFÜR · PFLICHTENheft = WAS + WIE	Spezifikationsdokumente
Verifikation = KORREKTES Produkt · Validation = RICHTIGES Produkt	V-Modell
analytisch = MESSEN (a posteriori, Produkt) · konstruktiv = SICHERSTELLEN (a priori, Prozess)	QS-Maßnahmen
Retrospektive = KONSTRUKTIV · Review = ANALYTISCH	Scrum-QS
Evolutionär = partielles Modell · Inkrementell = vollständiges Modell	Prozessmodelle
Erweiterung KEHRT ZURÜCK · Alternative ENDET	UC-Struktur
include: A immer B · extend: A kann B	UC-Beziehungen
2 ⁿ Regeln	Entscheidungstabelle
k Ausprägungen → k-1 Dummies	<i>(gilt hier nicht — das ist Modul 15!)</i>
Coffee-Break = ganzer UC · R4 = einzelne Aktivität	Umfangsregeln
RANF ⊂ NFANF	Anforderungsarten
muss = Pflicht · sollte = Wunsch · wird = Zukunft, aber Pflicht	Schlüsselwörter

O.3 DIE 15 TÖDLICHSTEN FEHLER (Checkliste vor der Abgabe!)

1.  **Das zu modellierende System als Akteur aufführen** → NIE!
2.  **„Nachbedingung Fehlschlag: keiner“** → gibt es fast nie
3.  **Ein externes System (E-Mailserver, Wissensdatenbank) bei den Akteuren vergessen**
4.  **Vorbedingungen als Aktivitäten modellieren** („muss ... worden sein“ → Vorbedingung!)
5.  **Das auslösende Ereignis als Aktivität 1 wiederholen**
6.  **Klicks/Buttons/Produktnamen im UC** → nicht essentiell!
7.  **Passiv oder Pronomen** in Aktivitätssätzen → R2 „Who has the ball“
8.  **„prüfen“ statt „validieren“** → R7
9.  **Rückmeldung an den Akteur vergessen** → R6, Schritt 4
10.  **Alternativen als Erweiterungen deklarieren** (und umgekehrt)
11.  **ET nicht konsolidiert** oder **nicht alle 2ⁿ Kombinationen abgedeckt**
12.  **Bei Single-Choice raten** → -1 Punkt!
13.  **„Begründen Sie“ ignorieren** → halbe Punktzahl
14.  **Im BPMN Gateways zum Zusammenführen vergessen** → Lesson Learned 2
15.  **Mehr als 9–10 Aktivitäten im UC** → R4

O.4 ★ Die Zahlen und Zitate

Zahl/Zitat	Kontext
5 Mrd. €	Schaden FISCUS
USD 192,5 Mio.	Schaden Levi Strauss
8.300 Projekte, 1991–2008	Capers-Jones-Studie
40.000 Projekte seit 1994	CHAOS-Studie
15 Jahre	Lebensdauer einer Programmiersprache
1000–1500 Funktionen	Komplexität von Büroanwendungen
2–20 Minuten, 1 Person, 1 Sitzung	Umfang eines Product Use Case
9–10 Aktivitäten	Maximum eines guten UC
1–5 Personentage	Aufwand einer User Story
30 Tage / 15 Minuten	Sprint / Daily Scrum
2–3 Monate	Auslieferungsrhythmus Crystal Clear
max. 1 Stunde	Interviewdauer
10–90 %	Rücklaufquote schriftlicher Befragungen
2 ⁿ	Regeln einer vollständigen ET
„Schrackware statt Software“	Vorwurf an schwergewichtige Prozesse
„Doing the right things right“	Anspruch leichtgewichtiger Prozesse
„Es wurde disponibles statt gutes Personal abgestellt“	FISCUS
„I can't tell you what I want, but I'll know it when I see it!“	Der typische Anwender
„After I get done with this, I can take a coffee break!“	Coffee-Break-Test
„Why isn't Sam coding yet??“	WHISCY-Syndrom
„Redo until Right“	Prototypen-Modell
„Essenz, Essenz, Essenz!!“	Der Dozent

0.5 ★★ Die Prüfungsstrategie (120 Minuten, 100 Punkte)

Minute	0–5	Klausur durchblättern, Seitenzahl prüfen, Punkte scannen → Die 35 Punkte für UC + BPMN (Aufgabe 7) und die 15 Punkte für die Fehlersuche (Aufgabe 6) sind die HÄLFTE der Klausur!
Minute	5–35	AUFGABE 1 (36 P., 10 Wissensfragen) → Das sind die schnellsten Punkte. Reine Reproduktion. → Stichpunkte reichen, wo „Nennen Sie“ steht; ganze Sätze, wo „Erläutern Sie“ steht.
Minute	35–50	AUFGABEN 2 + 3 (14 P., Kano + Anforderungsarten) → Definitionen + je ein Beispiel. Nicht ausschweifen.
Minute	50–70	AUFGABEN 4 + 5 (15 P., Entscheidungstabellen) → IMMER erst die vollständige 2 ⁿ -Tabelle im Schmierbereich, dann konsolidieren. Nie direkt konsolidiert anfangen!
Minute	70–85	AUFGABE 6 (15 P., 5 formale Fehler) → Je Fehler: WAS ist falsch / WIE korrigieren / WARUM. → Die 5 sichersten zuerst (System als Akteur, Fehlschlag „keiner“, fehlender Akteur, falsche Feldbezeichnungen, Ereignis = Aktivität 1).
Minute	85–115	AUFGABE 7 (20 P., PUC + BPMN) → Schablone von oben nach unten AUSFÜLLEN, kein Feld auslassen. → Erst Vorbedingung aus dem „muss ... worden sein“-Satz ziehen! → Akteure: alle Rollen und externen Systeme AUSSER dem SuD. → BPMN erst zeichnen, wenn der UC steht – er ist die Vorlage.
Minute	115–120	KONTROLLE → Checkliste 0.3 durchgehen. → Matrikelnummer auf JEDER Seite? Klausur geheftet?

0.6 ★ Der rote Faden der Vorlesung in einem Bild

LE 1	Warum scheitern Projekte?	→ weil Anforderungen fehlen
↓		
LE 2	Was ist Softwaretechnik?	→ Prinzipien, Methoden, Werkzeuge
↓		
LE 3	Wie strukturiert man Projekte?	→ Prozessmodelle (schwergewichtig)
↓		
LE 4	Oder anders?	→ Agile SE (leichtgewichtig)
↓		
===== AB HIER: DAS RE-VORGEHENSMODELL =====		
↓		
LE 5	① PROBLEMDOMÄNE VERSTEHEN	→ Ereignisse, Ereignisliste, ERM, ESSENZ
↓		
LE 6	Woher kommen die Anforderungen?	→ Erhebungstechniken
↓		
LE 7	② PRODUKT VERSTEHEN	→ Business UC → PRODUCT USE CASES
↓		
LE 8	Ablauf präzisieren	→ Aktivitätsdiagramm, BPMN
↓		
LE 9	Regeln präzisieren	→ ENTSCHEIDUNGSTABELLEN
↓		
LE 10	Anforderungen atomisieren	→ FANF/NFANF/RANF, SOPHIST, Snowcard, KANO
↓		
Exkurs	③ PRODUKT DOKUMENTIEREN	→ Lastenheft, Pflichtenheft (IEEE 830, Volere)

Ende Modul 16 — Systems Engineering, Prof. Dr. Michael Bächle, DHBW Ravensburg WDS224.